Chapter 13

AWT와 애플릿

AWT & Applet

1. AWT

자바로 좀 더 그럴듯한 프로그램을 만들고 싶다면 AWT를 사용하면 된다. AWT는 여러분들이 지금까지와는 다른 보다 멋진 자바 어플리케이션을 개발하도록 도와줄 것이다.

1.1 AWT란?

AWT(Abstract Window Toolkit)는 이름에서 알 수 있듯이 Window프로그래밍(GUI프로그래밍)을 하기 위한 도구이다. AWT는 GUI어플리케이션의 개발에 필요한 여러 개의 관련 패키지와 클래스의 집합으로 구성되어 있으며, 이 들을 이용하면 윈도우와 같은 GUI 어플리케이션을 쉽고 편리하게 작성할 수 있다.

GUI(Graphical User Interface)는 사용자가 그래픽을 통해서 하드웨어와 상호작용하는 환경을 말한다. 지금까지는 텍스트 기반의 어플리케이션을 작성해왔지만, AWT를 이용하면 그래픽 기반(GUI)의 어플리케이션을 작성할 수 있다.

텍스트기반의 사용자 인터페이스(Character User Interface)는 키보드로 명령을 내리고 결과가 문자로 표시되는 단순한 형태의 것이었다.

그러나, 그래픽 유저 인터페이스에서는 마우스 등을 이용하여 화면의 메뉴 중에서 하나를 선택하여 작업을 지시한다.

GUI는 도스(DOS)와 같은 텍스트 기반의 인터페이스와는 대조적이다. GUI의 요소를 살펴보면 윈도(Windows), 스크롤바, 아이콘 이미지, 단추들을 포함한다. 1980년대 후반부터 IBM PC 및 워크스테이션에서도 GUI가 보급되어 현재의 컴퓨터는 GUI를 사용하고 있다. 마이크로소프트사의 윈도, 애플 매킨토시의 GUI가 그 예이다.

AWT로 작성된 GUI어플리케이션 역시 플랫폼(Platform) 독립적이라서 여러 종류 OS에서 코드를 수정하지 않고도 실행이 가능하지만, 윈도우나 버튼과 같은 GUI컴포넌트들을 직접 구현하지 않고 해당 OS의 컴포넌트(native component)를 사용하기 때문에 AWT로 작성된 GUI어플리케이션의 외양(appearance)이 실행되는 OS마다 달라질 수 있다.

또한, 여러 종류의 GUI기반의 OS들이 공통적으로 가지고 있는 컴포넌트만으로 구성해야 하기 때문에 AWT가 제공할 수 있는 GUI컴포넌트의 수가 제한적이다.

대신 OS에서 제공하는 컴포넌트(native component)는 해당 OS에 최적화되어 있기 때문에 자바로 구현한 컴포넌트보다 속도가 더 빠르다.

GUI어플리케이션은 텍스트로만 구성된 어플리케이션 보다 수행해야할 작업이 훨씬 많기 때문에, 속도문제를 안고 있었던 자바로써는 어쩔 수없이 OS독립적인 부분을 다소 포기하면서도 속도를 선택할 수밖에 없었던 것 같다.

[참고] 여기서의 플랫폼은 자바(JVM)가 설치될 수 있는 기반환경을 의미한다. 플랫폼은 하드웨어가 될 수 도 있고, OS가 될 수 도 있다. 즉, OS보다 플랫폼이 더 넓은 의미를 갖는다. 예를 들면, MS Windows용 자바의 플랫폼은 MS Windows(OS)이고, MS Windows의 플랫폼은 IBM호환 컴퓨터(하드웨어)이다.

그러나 자바가 처음 소개되었던 초기상황과는 다르게 해를 거듭할수록 자바의 성능과 하드웨어환경이 크게 향상되면서 자바의 단점으로 지적되던 속도문제가 점점 해결되기 시작하였다.

그래서 새롭게 등장한 것이 Swing이다. Swing은 AWT를 확장한 것이지만, AWT와는 달리 순수한 자바로 이루어져 있어서 AWT보다 다양하고 풍부한 기능의 컴포넌트를 제공함으로써 진정한 GUI어플리케이션을 개발하기 위한 도구로 사용되고 있다.

Swing의 클래스들이 AWT의 클래스들을 기반으로 만들어진 자손 클래스들이므로 AWT와 Swing은 컴포넌트의 종류와 사용법만 조금 다를 뿐 나머지는 거의 같기 때문에 이 책에서는 AWT를 이용한 프로그래밍에 대해서만 살펴봐도 충분하리라 생각한다. 보다 높은 수준의 GUI어플리케이션을 개발하고자 한다면 Swing을 이용한 어플리케이션개발에 대한 책을 별도로 공부하는 것이 좋다.

[참고] 썬에서 제공하는 Swing 온라인 튜터리얼(http://java.sun.com/docs/books/tutorial/uiswing/)이 상당히 잘만들어져 있으니 Swing으로 GUI를 개발할 때 꼭 참고하도록 하자.

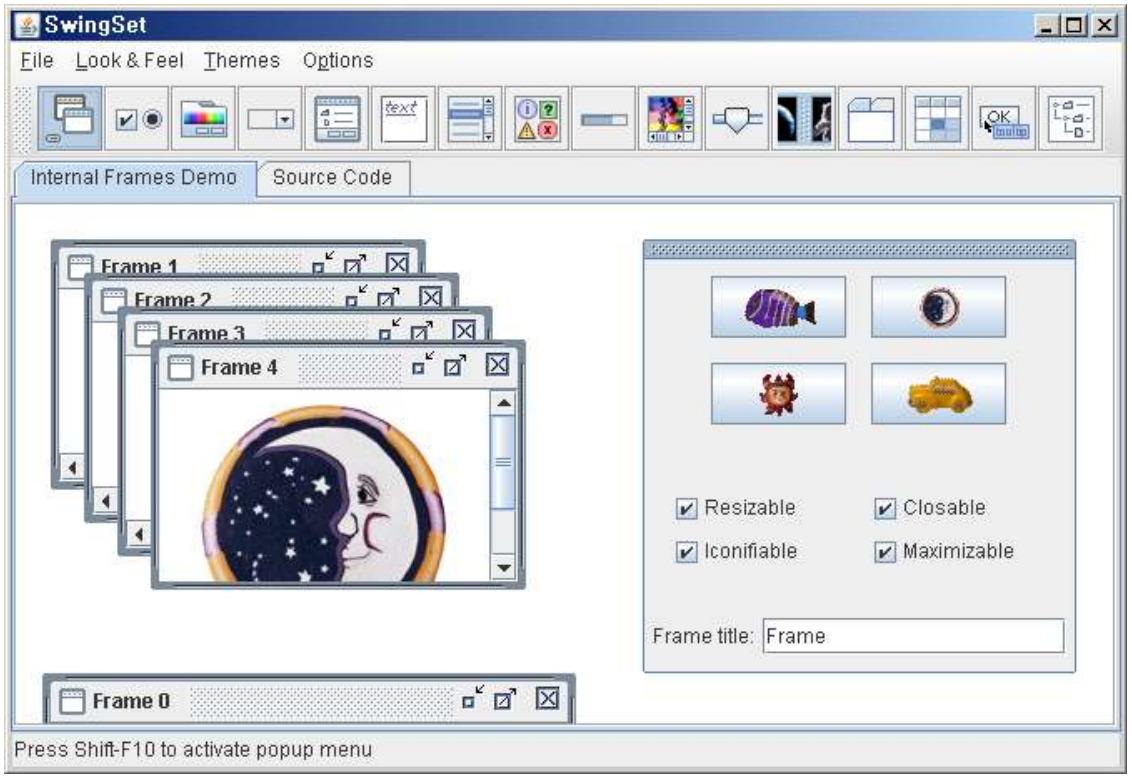
그래도 Swing이 어떤 것인지 구경은 한번 해보고 넘어가야할 것 같다. Swing으로 어떤 GUI어플리케이션을 만들 수 있는지 보려면, JDK에서 제공하는 데모프로그램인 SwingSet2.jar를 실행하면 된다.

JDK의 설치 경로가 C:₩jdk1.5라면, 탐색기로 C:₩jdk1.5₩demo₩jfc₩SwingSet2 디렉토리의 SwingSet2.jar 또는 SwingSet2.html을 찾아서 더블클릭하면 프로그램이 실행된다.

만일 알집이 설치되었다면, SwingSet2.jar를 더블클릭했을 때 프로그램이 실행되는 대신 알집이 실행될 것이다. 이 때는 도스창을 열어서 아래와 같은 명령어를 입력하면 된다.

```
C:\jdk1.5\demo\jfc\SwingSet2>java -jar SwingSet2.jar
```

그러면 다음과 같은 화면이 나타날 것이다.



[그림13-1] SwingSet2.jar의 실행화면

이 데모프로그램을 살펴보면 Swing을 이 어떤 기능을 제공하고, Swing을 이용해서 어떠한 일들을 할 수 있는지 충분히 알 수 있을 것이다.

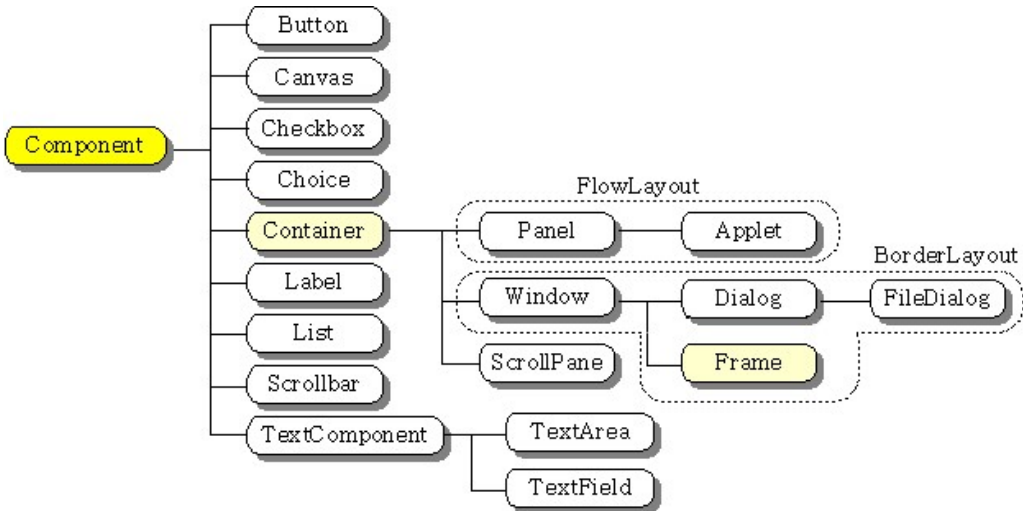
1.2 AWT의 구성

패키지명이 'java.awt'로 시작하는 것은 모두 AWT관련 패키지이다. 아래에 소개한 것 외에도 더 많은 AWT관련 패키지가 있지만 일단 지금은 java.awt와 java.awt.event패키지만 학습해도 충분하다. 나머지는 어떤 것들이 있는지 살펴만 보고 넘어가자.

패키지명	설명
java.awt	AWT를 이용한 GUI어플리케이션을 작성하는데 필요한 기본적인 클래스와 컴포넌트를 제공한다.
java.awt.datatransfer	여러 어플리케이션 사이의, 또는 단일 어플리케이션 내에서의 데이터 전송을 구현하는데 필요한 클래스와 인터페이스를 제공한다.
java.awt.dnd	GUI의 장점 중의 하나인 끌어놓기(Drag and Drop)기능을 구현하는데 필요한 클래스들을 제공한다.
java.awt.event	GUI어플리케이션에서 발생하는 이벤트를 처리하는데 필요한 클래스와 인터페이스를 제공한다.
java.awt.font	폰트와 관련된 클래스와 인터페이스를 제공한다.
java.awt.image	이미지를 생성하거나 변경하는데 사용되는 클래스를 제공한다.
java.awt.print	출력에 관련된 클래스와 인터페이스를 제공한다.

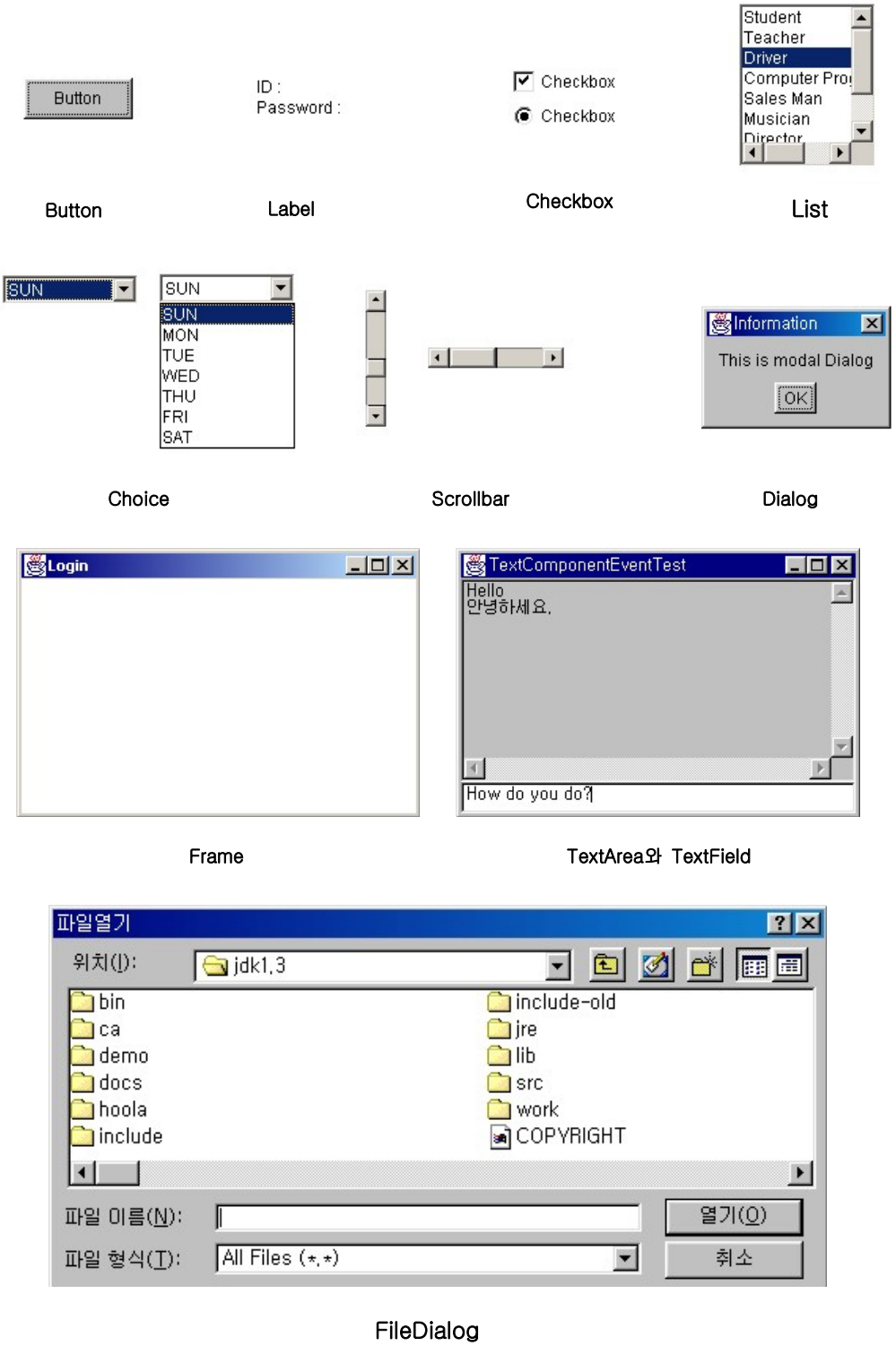
[표13-1] AWT와 관련된 패키지

메뉴와 관련된 컴포넌트들을 제외한 모든 컴포넌트의 조상은 Component클래스이고 Component클래스의 조상은 Object클래스이다. Component클래스와 그 자손 클래스들은 윈도우, 스크롤바, 버튼 등 GUI응용프로그램의 화면을 구성하는데 사용되는 클래스들이다. 이들을 AWT 컴포넌트 또는 줄여서 컴포넌트라고 부른다.

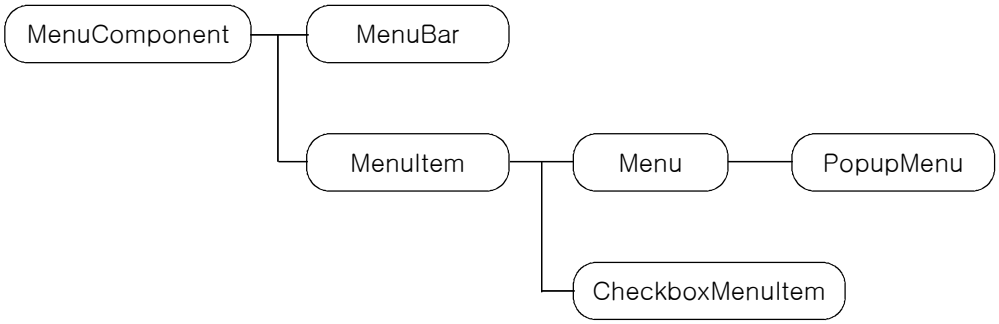


[그림13-2] AWT 컴포넌트의 상속계층도

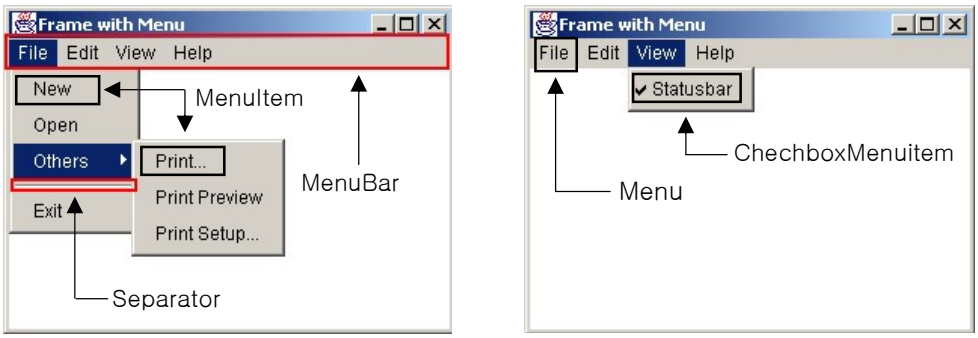
컴포넌트는 버튼이나 체크박스와 같은 일반적인 컴포넌트와 메뉴와 관련된 메뉴컴포넌트로 나누어진다. Component는 일반적인 컴포넌트의 최상위조상이며, MenuComponent는 메뉴관련 컴포넌트들의 최상위 조상이다.



[그림13-3] AWT가 제공하는 컴포넌트



[그림13-4] AWT 메뉴컴포넌트의 상속계층도



[그림13-5] GUI 어플리케이션의 메뉴구성에 사용되는 AWT컴포넌트

1.3 컴포넌트(Component)

Component는 MenuComponent를 제외한 AWT의 모든 컴포넌트의 조상이고 추상 클래스이다. Component에는 컴포넌트들이 가져야 할 공통적인 메서드들을 정의해놓고 있으며, 주로 사용되는 메서드는 다음과 같다.

[참고] 이 외에도 많은 메서드들이 있으며, 보다 자세한 것은 Java API문서를 참고하자.

메서드	설 명
Color getBackground()	컴포넌트의 배경색을 얻는다.
void setBackground(Color c)	컴포넌트의 배경색을 지정된 색으로 변경한다.
Cursor getCursor()	컴포넌트에 지정된 커서를 얻는다.
void setCursor(Cursor c)	컴포넌트의 커서(마우스포인터)를 지정한다.
Font getFont()	컴포넌트에 지정되어 있는 Font를 얻는다.
void setFont(Font f)	컴포넌트의 Font를 지정한다.
Color getForeground()	컴포넌트의 전경색을 얻는다.
void setForeground(Color c)	컴포넌트의 전경색을 지정한다.
int getHeigtht()	컴포넌트의 높이(Height)를 얻는다.
int getWidth()	컴포넌트의 폭(Width)를 얻는다.
void setBounds(int x, int y, int width, int height)	컴포넌트의 위치(x, y)와 크기(width, height)를 지정한다.
Rectangle getBounds()	컴포넌트의 위치와 크기(Rectangle객체)를 얻는다.

Point getLocation()	컴포넌트의 위치를 얻는다.
void setLocation(int x, int y)	컴포넌트의 위치를 지정한다.
Dimension getSize()	컴포넌트의 크기를 얻는다.
void setSize(int width, int height)	컴포넌트의 크기를 지정한다.
boolean hasFocus()	컴포넌트가 현재 focus를 갖고 있는지 알려준다.
void requestFocus()	컴포넌트가 focus를 갖도록 한다.
void paint(Graphics g)	컴포넌트를 화면에 그린다.
void repaint()	컴포넌트를 화면에 다시 그린다.
void setEnabled(boolean b)	컴포넌트를 사용가능(true)/불가능(false) 하게 한다.
Container getParent()	컴포넌트가 포함되어져 있는 컨테이너(parent)를 얻는다.
void setVisible(boolean b)	컴포넌트가 화면에 보이게(true)/보이지 않게(false) 한다.

[표13-2] Component클래스의 메서드

1.4 컨테이너(Container)

Component의 자손들 중에 Container와 그 자손들이 있는데, 이 들을 컨테이너라고 부른다. 컨테이너는 다른 컴포넌트들을 포함할 수 있어서 Button, Label과 같은 컴포넌트들을 포함할 수 있다. 또한 컨테이너가 컨테이너를 포함할 수도 있다.

1. **독립적인 컨테이너** - 독립적으로 사용될 수 있으며, 다른 컴포넌트나 종속적인 컨테이너를 포함할 수 있다.

컨테이너	설 명
Frame	가장 일반적인 컨테이너로 윈도우와 모양이 같다. titlebar와 크기조절버튼, 닫기버튼을 가지고 있다. 그리고 메뉴를 추가할 수 있다.
Window	Frame의 조상이며, 경계선, titlebar, 크기조절버튼, 닫기 버튼이 없으며, 메뉴도 추가할 수 없다. 단지 컴포넌트를 담을 수 있는 평면공간만을 갖는다.
Dialog	Frame처럼, titlebar와 닫기버튼을 갖고 있지만, 메뉴는 가질 수 없으며 기본적으로 크기를 변경할 수 없다. 주로 프로그램사용자에게 메시지를 보여 주거나, 응답을 받는데 사용한다.

2. **종속적인 컨테이너** - 독립적으로 사용될 수 없으며, 다른 컨테이너에 포함되어야만 한다.

컨테이너	설 명
Panel	평면공간으로 Frame과 같이 여러 컴포넌트를 담을 수 있으나 단독적으로 사용될 수는 없다.
ScrollPane	Panel과 같은 평면공간이지만, Panel과는 달리 단 하나의 컴포넌트만 포함할 수 있으며 자신보다 큰 컴포넌트가 포함되면 스크롤바가 자동적으로 나타난다.

컨테이너에는 여러 개의 오버로딩(overloading)된 add메서드들이 정의되어 있어서, 컨테이너에 단순히 add메서드를 사용하는 것만으로 다른 컴포넌트들을 포함시킬 수 있다. 컨테이너에 포함된 컴포넌트들은 기본적으로 컨테이너에 설정된 배경색(background color)과 전경색(foreground color), 글자체 등의 설정을 그대로 따르게 된다.

메서드	설 명
Component[] getComponents()	컨테이너에 포함되어 있는 모든 컴포넌트를 얻는다.
Component getComponent(int n)	컨테이너에 n번째로 추가된 컴포넌트를 얻는다.
Component getComponentAt(int x, int y)	컨테이너의 지정된 위치(x, y)에 있는 컴포넌트를 얻는다.
Component add(Component comp)	컨테이너에 컴포넌트를 추가한다.
void remove(Component comp)	컨테이너에서 지정된 컴포넌트를 제거한다.
Insets getInsets()	컨테이너의 경계의 크기를 알 수 있는 Inset객체를 얻는다.
LayoutManager getLayout()	컨테이너에 설정되어 있는 LayoutManager를 얻는다.
void setLayout(LayoutManager mgr)	컨테이너에 LayoutManager를 설정한다.

[표13-3] Container의 메서드

2. AWT의 주요 컴포넌트

이제 AWT를 이용한 GUI프로그래밍에 사용되는 컴포넌트들의 종류와 기본적인 기능에 대해서 자세히 알아보자.

2.1 Frame

GUI프로그래밍의 가장 대표적인 컴포넌트로 다른 컴포넌트들을 포함할 수 있는 컨테이너이다. titlebar와 최대화버튼, 최소화버튼, 닫기버튼이 있으며 크기를 조절할 수 있다.

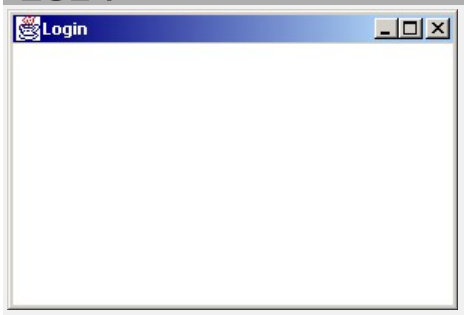
메서드 또는 생성자	설 명
Frame(String title)	Frame을 생성한다. title - Frame의 titlebar에 보여 질 text
String getTitle() void setTitle(String title)	titlebar에 있는 text를 얻는다.
	titlebar의 text를 변경한다.
void setState(int state)	Frame의 상태를 변경할 수 있으며, state에는 아래의 두 가지 값중 하나를 사용할 수 있다. Frame.ICONIFIED - Frame을 최소화 상태가 되게 한다. Frame.NORMAL - Frame을 정상적인 상태(최소화 이전상태)가 되게 한다.
int getState()	Frame의 현재 상태를 알 수 있다.
void setResizable(boolean resizable)	Frame의 크기를 변경가능 또는 불가능하게 한다. (resizable의 값이 false로 하면 사용자가 Frame의 크기를 변경할 수 없다.)

[표13-4] Frame의 메서드

[예제13-1]/ch13/**FrameTest**.java

```
import java.awt.*;

class FrameTest {
    public static void main(String args[]) {
          Frame f = new Frame("Login");     // Frame객체를 생성한다.
          f.setSize(300, 200);              // Frame의 크기를 설정한다.
          f.setVisible(true);               // 생성한 Frame을 화면에 보이도록 한다.
    }
}
```

[실행결과]



이 예제는 단순히 Frame 하나를 화면에 보여주는 것인데, 실행결과 그림은 윈도우즈에서 실행한 것이기 때문에 윈도우즈의 윈도우와 똑같은 모습을 하고 있다.(맥OS에서 위의 예제를 실행했다면, 맥OS의 윈도우와 같은 모양의 Frame이 화면에 나타났을 것이다.)

[참고] AWT는 OS의 컴포넌트들을 그대로 사용하는 것이기 때문에 AWT응용프로그램의 외향은 실행되는 OS에 따라 달라지게 된다.

크기 조절도 되고, titlebar를 드래그하면 이동할 수도 있는 등, 윈도우즈의 윈도우와 같은 기능을 가지고 있다. 단, 닫기 버튼은 눌러도 아무작동을 하지 않는데 이것을 가능하게 하려면 나중에 배울 이벤트처리를 이용해서 기능을 추가해주어야 한다.

이 예제의 실행을 멈추려면, 콘솔(도스창)에서 Ctrl키+c를 누르던가, 윈도우즈에서 Ctrl+Alt+Delete키를 누른 후 java.exe를 선택하도록 한다.

1. Frame객체를 하나 만들고, - 생성자에 사용된 String은 Frame의 titlebar에 나타난다.

```
Frame f = new Frame("Login");
```

2. 생성된 Frame의 크기를 설정한 다음, - 폭(width) : 300 픽셀(pixel), 높이 : 200 픽셀

```
f.setSize(300, 200);
```

3. Frame을 화면에 보이도록 한다. - Frame객체를 생성했다고 해서 화면에 보이는 것은 아니고 반드시 setVisible()을 사용해야 화면에 나타난다.

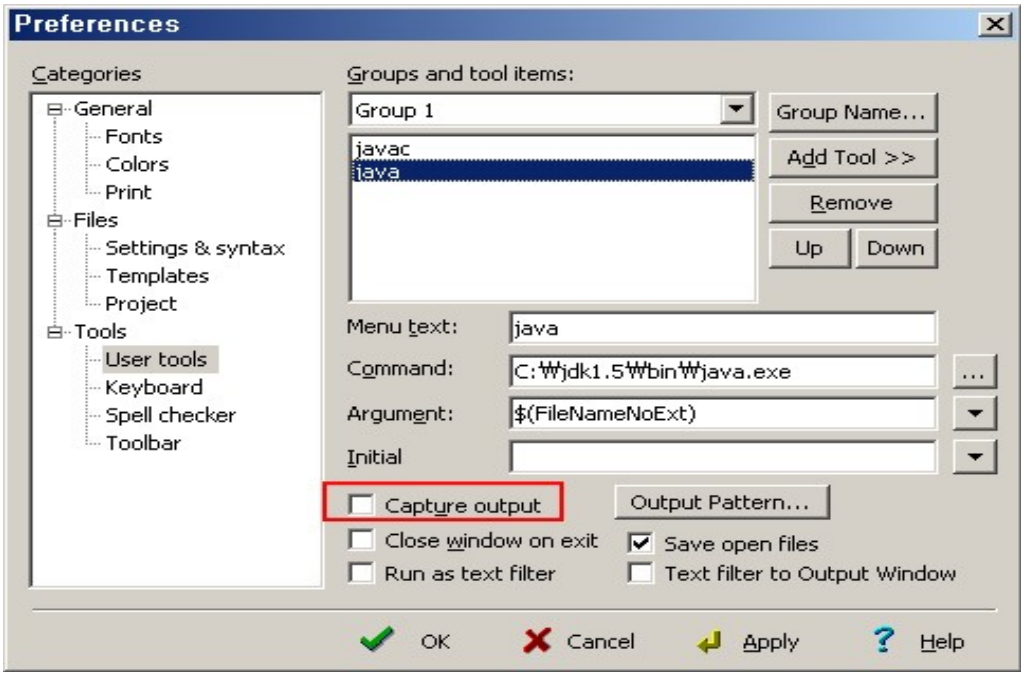
```
f.setVisible(true);
```

Frame객체의 설정작업(크기조절)이 모두 끝난 후에 마지막으로 setVisible()을 호출하도록 한다.

만일 setSize() 보다 setVisible(true)를 먼저 호출하면, Frame의 크기가 변경되는 과정이, 아주 짧은 시간동안이지만, 화면에 보일 것이다. 이렇게 되면, 보기에도 좋지 않을뿐더러 Frame이 변경될 때마다 화면을 갱신해야하므로 비효율적이다.

[참고] setSize()와 setVisible()은 Frame의 조상인 Component에 정의된 메서드이다.

[참고] Editplus에서 AWT프로그래밍을 할 때는 옵션을 아래의 그림과 같이 java.exe의 Capture Output옵션을 해제하는 것이 좋다. Frame에 이벤트처리를 하지 않았기 때문에 예제의 실행을 멈추기가 어렵기 때문이다.



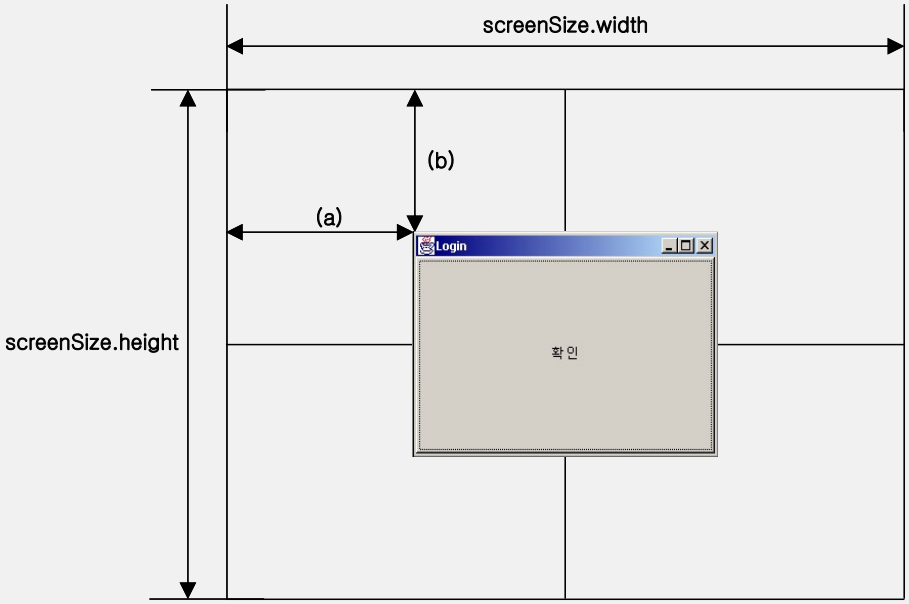
[그림13-6] Editplus에서 java.exe의 Capture output옵션을 해제하는 방법

예제13-1을 실행시키면, Frame이 모니터 화면의 좌측 상단에 나타나는데 다음 예제는 Toolkit클래스의 getScreenSize()를 이용해서 Frame이 화면의 중앙에 나타나게 한다.

[예제13-2]/ch13/FrameTest2.java

```
import java.awt.*;

class FrameTest2 {
    public static void main(String args[]) {
          Frame f = new Frame("Login");
          f.setSize(300, 200);

          Toolkit tk = Toolkit.getDefaultToolkit();   // 구현된 Toolkit객체를 얻는다.
          Dimension screenSize = tk.getScreenSize();  // 화면의 크기를 구한다.


          // 화면크기의 절반값에서 Frame크기의 절반값을 뺀 위치로 하면
          // Frame이 화면 가운데 위치하게 된다.
          f.setLocation(screenSize.width/2 - 150, screenSize.height/2 - 100);
          f.setVisible(true);                    // 생성한 Frame을 화면에 보이도록 한다.
    }
}
```

[실행결과]



(a) screenSize.width / 2 - Frame너비 / 2
(b) screenSize.height / 2 - Frame높이 / 2

2.2 Button

Button은 사용자가 클릭했을 때, 어떤 작업이 수행되도록 할 때 사용된다.

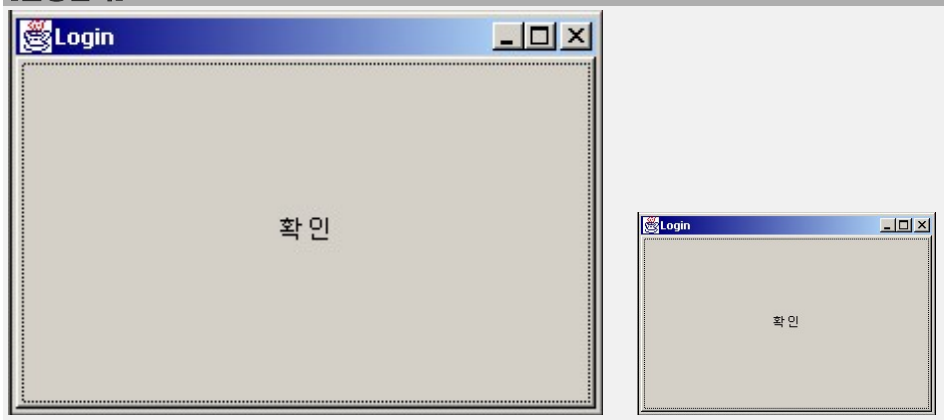
메서드 또는 생성자	설 명
Button(String label)	지정된 label을 가진 Button을 생성한다. label - Button위에 나타날 text
String getLabel()	Button에 나타나있는 text를 얻는다.
void setLabel(String label)	Button에 나타나있는 text를 변경한다.

[표13-5] Button의 메서드

[예제13-3]/ch13/**ButtonTest.java**

```
import java.awt.*;

class ButtonTest {
    public static void main(String args[]) {
        Frame f = new Frame("Login");
        f.setSize(300, 200);

        Button b = new Button("확 인");  // Button위에 "확 인"이라는 글자가 나타난다.
        b.setSize(100, 50);             // Button의 크기를 설정한다.

        f.add(b);                       // 생성된 Button을 Frame에 포함시킨다.
        f.setVisible(true);
    }
}
```

[실행결과]



Frame은 컨테이너이므로 다른 컴포넌트들을 포함할 수 있다. 이 예제에서는 Button을 하나 생성한 후 add메서드를 이용해서 Frame에 포함시켰다. 이와 같은 방식으로 Frame에 여러 가지 다양한 컴포넌트들을 추가함으로써 여러분들이 그동안 보아왔던 것과 같은 윈도우를 만들 수 있을 것이다.

위의 예제를 실행한 후, 나타난 화면을 보고 여러분들이 예상했던 화면과 좀 다르다고 생각했을 것이다.

Button의 크기를 폭 100픽셀, 높이 50픽셀로 설정했는데, 크기가 300픽셀, 200픽셀크기의 Frame에 가득 차 있다. 그리고 Frame의 모서리를 드래그 하여 Frame의 크기를 변경시키면, Button의 크기도 같이 변경되는 것을 볼 수 있을 것이다.

그 이유는 바로 Frame에 설정되어 있는 레이아웃 매니저(Layout Manager)때문이다. 컨테이너에는 레이아웃 매니저를 선택적으로 설정할 수 있는데, 레이아웃 매니저는 컨테이너의 크기가 변경될 때마다 컨테이너에 포함된 컴포넌트들의 크기와 위치를 자동적으로 관리해준다.

따라서 프로그래머가 수동적으로 일일이 컨테이너 내의 컴포넌트 배치에 대해 신경 쓰지 않아도 되는 장점이 있다. 기존의 Visual Basic과 같은 기존의 GUI프로그래밍에는 레이아웃 매니저개념이 없어서, 윈도우의 크기가 변경될 때 윈도우 내의 컴포넌트들을 재배치해주는 코드를 프로그래머가 별도로 작성해야만 한다.

[참고] Frame에는 기본적으로 레이아웃 매니저가 BorderLayout으로 설정되어 있으며, 레이아웃 매니저의 종류와 특성 등 자세한 내용은 앞으로 배우게 될 것이다.

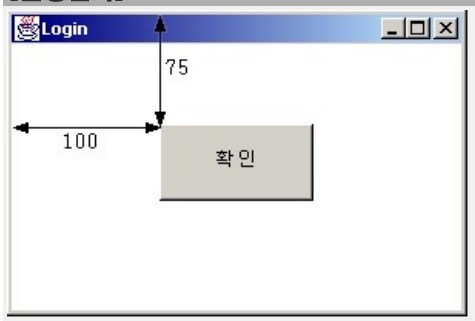
아래의 예제는 위의 예제에서 기본적으로 설정된 레이아웃 매니저를 없애고, Button의 크기와 위치를 수동적으로 설정해 준 것이다.

[예제13-4]/ch13/**ButtonTest2.java**

```
import java.awt.*;

class ButtonTest2 {
    public static void main(String args[]) {
        Frame f = new Frame("Login");
        f.setSize(300, 200);
        f.setLayout(null);          // 레이아웃 매니저의 설정을 해제한다.
```

```
            Button b = new Button("확 인");
            b.setSize(100, 50);         // Button의 크기를 설정한다.
            b.setLocation(100, 75);     // Frame내에서의 Button의 위치를 설정한다.

            f.add(b);
            f.setVisible(true);
        }
    }
```

[실행결과]



컨테이너의 레이아웃을 설정해주는 setLayout()에 null값을 넣어줌으로써, Frame에 아무런 레이아웃 매니저도 지정되어 있지 않도록 한다. 그렇기 때문에, 이제는 Frame의 크기를 변경해도 Button의 크기와 위치에는 전혀 변화가 없다.

대신 Button의 크기와 위치를 수동적으로 직접 지정해주어야 한다. 그래서 setLocation()과 setSize()를 통해 Button의 크기와 위치를 지정해주었다.

이 예제에 사용된 setLocation() 역시 Component클래스에 정의된 메서드로서 모든 컴포넌트에 사용할 수 있는 메서드이다.

지금 이 예제에서는 Button이 눌러졌을 때 수행해야할 코드를 적어주지 않았으므로 Button을 눌러도 아무런 일도 일어나지 않는다. Button이 눌러졌을 때 어떤 작업을 하도록 하기위해서는 Button에 이벤트 처리를 해주어야 하는데, 이벤트 처리에 대해서는 나중에 자세히 배우게 될 것이다.

2.3 Choice

여러 개의 item이 있는 목록을 보여주고, 그 중에서 한 가지를 선택하도록 할 때 Choice를 사용한다.

[참고] 기존의 GUI 프로그래밍에서는 Choice를 콤보박스(combo box) 또는 드랍다운 리스트박스(drop-down listbox)라고 부른다.

메서드 또는 생성자	설 명
void add(String item)	Choice에 item을 추가한다.
void remove(String item)	Choice에서 item을 제거한다.
void remove(int index)	지정된 순서에 있는 item을 제거한다.
void removeAll()	Choice의 모든 item을 제거한다.
void insert(String item, int index)	지정된 순서에 item을 추가한다.
String getItem(int index)	지정된 순서의 item을 얻는다.
int getItemCount()	현재 Choice에 추가되어 있는 item이 몇 개인지 알려준다.
int getSelectedIndex()	현재 선택되어져 있는 item의 index값을 얻는다.
String getSelectedItem()	현재 선택되어져 있는 item을 얻는다.

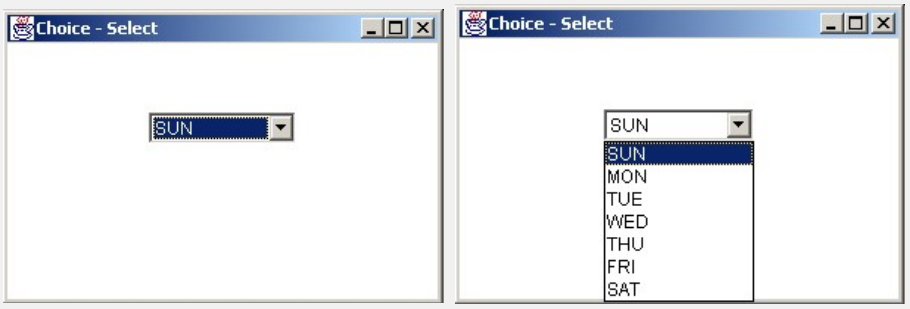
[표13-6] Choice의 메서드

[참고] index는 0부터 시작하므로, 첫번째 item의 index값은 0이다.

[예제13-5]/ch13/ChoiceTest.java

```
import java.awt.*;

class ChoiceTest {
    public static void main(String args[]) {
          Frame f = new Frame("Choice - Select");
          f.setSize(300, 200);
          f.setLayout(null);

          Choice day = new Choice();  // Choice객체를 생성한 다음
          day.add("SUN");             // Choice의 목록에 나타날 값들을 추가한다.
          day.add("MON");
          day.add("TUE");
          day.add("WED");
          day.add("THU");
          day.add("FRI");
          day.add("SAT");

          day.setSize(100, 50);
          day.setLocation(100, 70);

          f.add(day);
          f.setVisible(true);
    }
}
```

[실행결과]



add()를 이용하면 Choice에 나타날 항목을 추가할 수 있으며, 화살표버튼을 누르면 추가한 항목들이 순서대로 나타난다.

2.4 List

List 역시 Choice처럼 목록에서 중에서 원하는 항목(item)을 선택할 수 있도록 할 때 사용된다. 그러나 List는 Choice와는 달리, 처음부터 모든 item목록을 보여주며, 목록의 item 중에서 하나 또는 여러 개를 선택하도록 할 수 있다.

메서드 또는 생성자	설 명
List(int rows, boolean multipleMode)	rows - 몇 줄짜리 크기의 List를 보여줄 것인지 지정한다. multipleMode - List목록 다중선택이 가능하도록 할 것인가를 지정할 수 있다. true로 하면 여러 개의 item을 선택할 수 있다.
List(int rows)	List에 보여줄 item 수만 지정한다. multipleMode의 값은 false로 지정되어 하나의 목록만 선택가능하다.
List()	rows의 값은 기본값인 4로 지정되고, multipleMode의 값은 false가 되어 하나의 item만 선택가능하다.
void add(String item)	item을 List에 추가한다.
void add(String item, int index)	지정된 위치(index)에 item을 추가한다.
void replaceItem(String newValue, int index)	지정된 위치(index)의 item을 새로운 item(newValue)로 바꾼다.
void remove(String item) void remove(int index) void removeAll()	List에서 해당 item을 제거한다.
	index - 지정된 위치에 있는 item을 제거한다.
	List의 모든 item을 제거한다.
int getRows()	List에 scroll없이 볼 수 있는 item의 수를 얻는다.
String getItem(int index)	index - 지정된 위치에 있는 item을 얻는다.
String[] getItems()	List에 있는 모든 item을 얻는다.
int getItemCount()	List에 있는 item이 모두 몇 개인지 알려준다. (getRows()와 비교해볼 것)
void select(int index) void deselect(int index)	지정된 위치에 있는 item을 선택한다.
	지정된 위치에 있는 item을 선택해제한다.
int getSelectedIndex()	현재 선택되어 있는 item의 index값을 얻는다.
int[] getSelectedIndexes()	현재 선택되어 있는 item들의 index값을 얻는다. (List의 multipleMode가 true인 경우)
String getSelectedItem()	현재 선택되어 있는 item을 얻는다.
String[] getSelectedItems()	현재 선택되어 있는 item들을 얻는다. (multipleMode가 true인 경우)
boolean isIndexSelected(int index)	지정된 위치의 item이 선택되어있는지 알려준다.
void setMultipleMode(boolean b)	List를 multipleMode로 할 것인지 결정한다. b - true면, List를 multipleMode로 설정한다.

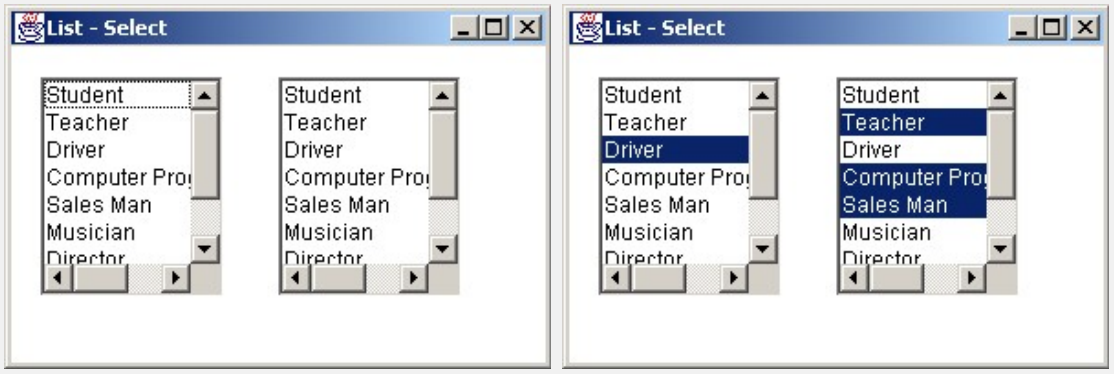
[표13-7] Choice의 메서드

[예제13-6]/ch13/ListTest.java

```
import java.awt.*;

class ListTest {
    public static void main(String args[]) {
```

```
        Frame f = new Frame("List - Select");
        f.setSize(300, 200);
        f.setLayout(null);

        List selectOne = new List(6);     // 6개 목록을 보여줄 수 있는 크기의 List를 만든다.
        selectOne.setLocation(20,40);
        selectOne.setSize(100, 120);
        selectOne.add("Student");
        selectOne.add("Teacher");
        selectOne.add("Driver");
        selectOne.add("Computer Programmer");
        selectOne.add("Sales Man");
        selectOne.add("Musician");
        selectOne.add("Director");

        // 생성자의 두번째 인자값을 true로 설정해서 List의 목록에서 여러 개를 선택할 수 있게 했다.
        List selectMany = new List(6, true);
        selectMany.setLocation(150, 40);
        selectMany.setSize(100,120);
        selectMany.add("Student");
        selectMany.add("Teacher");
        selectMany.add("Driver");
        selectMany.add("Computer Programmer");
        selectMany.add("Sales Man");
        selectMany.add("Musician");
        selectMany.add("Director");

        f.add(selectOne);
        f.add(selectMany);
        f.setVisible(true);
    }
}
```

[실행결과]



2.5 Label

Label를 사용하면 화면에 글자를 표시할 수 있으며, 설명이나 메시지를 화면에 나타내는데 주로 사용된다.

메서드 또는 생성자	설 명
Label(String text, int alignment)	text - 화면에 나타낼 글(text)을 String으로 넣는다. alignment -text의 정렬방식을 지정한다. Label.LEFT, Label.CENTER, Label.RIGHT 중 하나를 사용.
Label(String text)	text - 화면에 나타낼 text를 지정한다. alignment의 기본값인 Label.LEFT로 설정된다.
String getText()	Label의 text를 얻어온다.
void setText(String text)	Label의 text를 주어진 값으로 변경한다.
void setAlignment(int alignment)	Label의 text 정렬을 지정한다. Label.LEFT, Label.CENTER, Label.RIGHT 중 하나를 사용.

[표13-8] Label의 주요 메서드

[예제13-7]/ch13/**LabelTest**.java

```
import java.awt.*;

class LabelTest {
    public static void main(String args[]) {
          Frame f = new Frame("Login");
          f.setSize(300, 200);
          f.setLayout(null);

          Label id = new Label("ID :");      // Label을 생성하고 크기와 위치를 지정한다.
          id.setBounds(50, 50, 30, 10);      // 50, 50위치에 크기가 가로 30, 세로 10

          Label pwd = new Label("Password :");
          pwd.setBounds(50, 65, 100, 10);

          f.add(id);                         // 생성한 Label을 Frame에 포함시킨다.
          f.add(pwd);
          f.setVisible(true);
    }
}
```

[실행결과]

[참고] Frame에 Layout Manager가 설정되지 않았을 때, setSize() 또는 setBounds()를 이용해서 컴포넌트들의 크기를 지정해주지 않으면 컴포넌트가 화면에 나타나지 않는다.

2.6 Checkbox

Checkbox는 boolean과 같이 true/false 또는 on/off와 같이 둘 중의 한 값을 가질 수 있는 컴포넌트다. 또 CheckboxGroup을 이용하면, 여러 가지 값들 중에서 한 가지를 선택할 수 있는 radio button도 만들 수 있다.

메서드 또는 생성자	설 명
Checkbox(String text, boolean state)	text - Checkbox와 함께 보여질 text를 지정한다. state - true이면 Checkbox가 선택된 상태로 생성되고, false이면 Checkbox가 선택해제된 상태로 생성된다.
Checkbox(String text)	Checkbox와 함께 보여질 text를 지정한다. Checkbox는 선택해제된 상태로 생성된다.
Checkbox()	text없이 Checkbox만 나타나고, Checkbox는 선택해제된 상태로 생성된다.
Checkbox(String text, CheckboxGroup group, boolean state)	group - CheckboxGroup객체의 참조. CheckboxGroup을 이용해서 radio button으로 만든다.
String getLabel()	Checkbox의 label을 얻는다.
void setLabel(String label)	Checkbox의 label을 주어진 값으로 변경한다.
boolean getState()	Checkbox의 상태를 얻는다. 결과값이 true면 체크되어 있는 상태이다.
void setState(boolean state)	Checkbox의 상태를 설정한다. state값을 true로 하면 Checkbox가 체크되어 있는 상태가 되도록 한다.

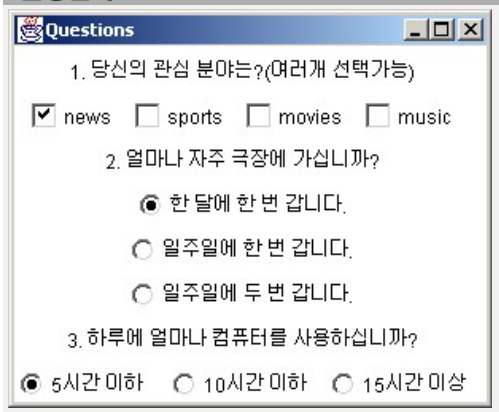
[표13-9] CheckBox의 메서드

[예제13-8]/ch13/CheckboxTest.java

```
import java.awt.*;

class CheckboxTest {
    public static void main(String args[]) {
          Frame f = new Frame("Questions");
          f.setSize(305, 250);
          // Frame의 LayoutManager를 FlowLayout으로 설정한다.
          f.setLayout(new FlowLayout());

          Label q1 = new Label("1. 당신의 관심 분야는?(여러개 선택가능)");
          Checkbox news = new Checkbox("news", true); // 선택된 상태로 생성
          Checkbox sports = new Checkbox("sports");
          Checkbox movies = new Checkbox("movies");
          Checkbox music = new Checkbox("music");

          f.add(q1); f.add(news); f.add(sports); f.add(movies); f.add(music);

          Label q2 = new Label("2. 얼마나 자주 극장에 가십니까?");
          CheckboxGroup group1 = new CheckboxGroup();
          Checkbox movie1 = new Checkbox("한 달에 한 번 갑니다.", group1, true);
          Checkbox movie2 = new Checkbox("일주일에 한 번 갑니다.", group1, false);
          Checkbox movie3 = new Checkbox("일주일에 두 번 갑니다.", group1, false);

          f.add(q2); f.add(movie1); f.add(movie2); f.add(movie3);
```

```
            Label q3 = new Label("3. 하루에 얼마나 컴퓨터를 사용하십니까?");
            CheckboxGroup group2 = new CheckboxGroup();
            Checkbox com1 = new Checkbox("5시간 이하 ", group2, true);
            Checkbox com2 = new Checkbox("10시간 이하", group2, false);
            Checkbox com3 = new Checkbox("15시간 이상", group2, false);

            f.add(q3); f.add(com1); f.add(com2); f.add(com3);
            f.setVisible(true);
        }
    }
```

[실행결과]



위의 예제는 Checkbox와 Label을 이용해서 몇가지 질문을 만든 것이다. Frame의 레이아웃 매니저를 FlowLayout으로 설정하였기 때문에 사용된 컴포넌트들의 크기와 위치는 따로 설정하지 않았다.

첫 번째 질문의 Checkbox들은 그룹화하지 않았기 때문에 각각 독립적으로 선택 가능하다. 4가지 모두를 선택할 수도 있고, 4가지 모두 선택하지 않을 수도 있다.

실행했을 때, news가 선택되어진 이유는 생성자의 두 번째 매개변수에 true를 설정했기 때문이다. 기본적으로는 false기 때문에 이처럼 따로 값을 지정하지 않으면 선택되지 않은 상태로 나타난다.

```
Checkbox news = new Checkbox("news", true);        // 선택된 상태로 생성
```

두 번째와 세 번째 질문은 Checkbox들을 CheckboxGroup(group1과 group2)을 이용해서 그룹화하였기 때문에 라디오 버튼(radio button)이 되었다. 이 그룹화된 Checkbox들은, 같은 그룹 내의 Checkbox들 중에서 한 가지만 선택할 수 있다.

2.7 TextField

사용자로부터 값을 입력을 받을 수 있는 컴포넌트이다. 편집이 가능하며 한 줄만 입력할 수 있어서 이름, id, 비밀번호 등 비교적 짧은 값의 입력에 사용된다.

메서드 또는 생성자	설 명
TextField(String text, int col)	text - TextField에 보여질 text를 지정한다. col - 입력 받을 글자의 수를 적는다. col의 값에 따라서 TextField의 크기가 결정된다.
TextField(int col)	col - 입력 받을 글자의 수를 적는다.
TextField(String text)	text - TextField에 보여질 text를 지정한다.
void setEchoChar(char c)	지정된 문자를 EchoChar로 한다. (비밀번호 입력에 주로 사용됨)
int getColumns()	TextField의 칼럼 수를 얻는다.
void setText(String t)*	지정된 문자열을 TextField의 text로 한다.
String getText()*	TextField의 text를 얻는다.
void select(int selectionStart, int selectionEnd)*	selectionStart부터 selectionEnd까지의 text를 선택(하이라이트)한다.
void selectAll()*	TextField의 모든 text를 선택된 상태가 되도록 한다.
String getSelectedText()*	TextField의 text중 선택되어진 부분을 얻는다.
void setEditable(boolean b)*	TextField의 text를 편집가능(true)/불가능(false) 하도록 한다.

[표13-10] TextField의 메서드

[참고] '*' 표시가 되어있는 것은 TextField의 조상인 TextComponent로부터 상속받은 것이다.

[예제13-9]/ch13/TextFieldTest.java

```
import java.awt.*;

class TextFieldTest {
    public static void main(String args[]) {
          Frame f = new Frame("Login");
          f.setSize(400, 65);
          f.setLayout(new FlowLayout()); // LayoutManager를 FlowLayout으로 한다.

          Label lid = new Label("ID :", Label.RIGHT); // 정렬을 오른쪽으로.
          Label lpwd = new Label("Password :", Label.RIGHT);

          TextField id = new TextField(10);//약 10개의 글자를 입력할 수 있는 TextField 생성
          TextField pwd = new TextField(10);
          pwd.setEchoChar('*');      // 입력한 값 대신 '*'가 보이도록 한다.

          f.add(lid);      // 생성한 컴포넌트들을 Frame에 포함시킨다.
          f.add(id);
          f.add(lpwd);
          f.add(pwd);
          f.setVisible(true);
    }
}
```

[실행결과]

TextField는 예제를 실행시켜보는 것만으로도 다른 설명이 필요없을 정도로 여러분들에게 익숙한 컴포넌트일 것이다.

패스워드를 입력하는 TextField에는 void setEchoChar(char c)를 사용하면, 입력하는 값에 관계없이 화면에는 지정된 문자 '*'가 보이게 된다. 그렇다 할지라도, TextField는 실제로 입력한 값을 갖게 된다.

여기서도 Frame의 레이아웃 매니저를 FlowLayout으로 하였기 때문에, Frame내의 컴포넌트들의 크기와 위치를 따로 지정하지 않았다.

[참고] TextField에는 이 외에도 많은 기능들이 있는데, AWT의 이벤트처리에서 더 자세히 배우게 될 것이다.

2.8 TextArea

TextArea는 여러 줄의 text를 입력하거나 보여줄 수 있는 편집가능한 컴포넌트이다. 그리고 스크롤바를 이용해서 실제화면에 보이는 것보다 많은 양의 text를 담을 수 있다.

메서드 또는 생성자	설 명
TextArea(String text, int row, int col, int scrollbar)	text - TextArea에 보여질 text를 지정한다. row - TextArea의 줄(row) 수를 지정한다. col - TextArea의 열(column) 수를 적는다. scrollbar - TextArea에 사용할 scrollbar의 종류와 사용여부 지정. 아래의 4가지 값 중에서 하나를 선택 TextArea.SCROLLBARS_BOTH TextArea.SCROLLBARS_NONE TextArea.SCROLLBARS_HORIZONTAL_ONLY TextArea.SCROLLBARS_VERTICAL_ONLY
TextArea(int row, int col)	row - TextArea의 줄(row) 수를 지정한다. col - TextArea의 열(column) 수를 적는다. 아무런 text도 없는 빈 TextArea를 생성한다. scrollbar는 수평(HORIZONTAL), 수직(VERTICAL) 모두 갖는다.
int getRows()	TextArea의 행(row)의 개수를 얻는다.
int getColumns()	TextArea의 열(column)의 개수를 얻는다.
void setRows(int rows)	지정된 값으로 TextArea의 행(row)의 개수를 설정한다.
void setColumns(int columns)	지정된 값으로 TextArea의 열(column)의 개수를 설정한다.
void append(String str)	TextArea에 있는 text의 맨 마지막에 문자열을 덧붙인다.
void insert(String str, int pos)	TextArea에 있는 text의 지정된 위치에 문자열을 넣는다.
void replaceRange(String str, int start, int end)	TextArea에 있는 text의 start부터 end범위에 있는 문자열을 str에 지정된 값으로 변경한다.
void setText(String t)*	지정된 문자열을 TextArea의 text로 한다.
String getText()*	TextArea의 text를 얻는다.

void select(int selectionStart, int selectionEnd)*	selectionStart부터 selectionEnd까지의 text가 선택되게 한다.
void selectAll()*	TextArea의 모든 text를 선택되게 한다.
String getSelectedText()*	TextArea의 text중 선택된 부분을 얻는다.
void setEditable(boolean b)*	TextArea의 text를 편집가능(true)/불가능(false) 하도록 한다.

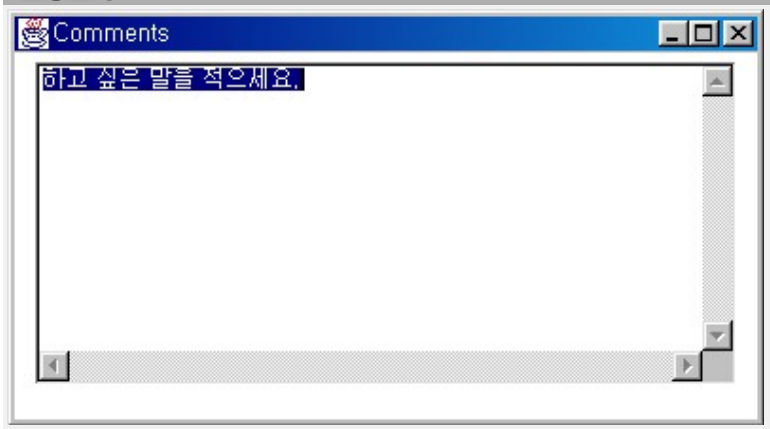
[표13-11] TextArea의 메서드

[참고] '*' 표시가 되어있는 것은 TextArea의 조상인 TextComponent로부터 상속받은 것이다.

[예제13-10]/ch13/TextAreaTest.java

```
import java.awt.*;

class TextAreaTest {
    public static void main(String args[]) {
          Frame f = new Frame("Comments");
          f.setSize(400, 220);
          f.setLayout(new FlowLayout());

          TextArea comments = new TextArea("하고 싶은 말을 적으세요.", 10, 50);

          f.add(comments);
          comments.selectAll();  // TextArea의 text 전체가 선택 되도록 한다.
          f.setVisible(true);
    }
}
```

[실행결과]



2.9 Scrollbar

Scrollbar는 사용자가 정해진 범위 내에 있는 값을 쉽게 선택할 수 있도록 해주는 컴포넌트이다. 주로 볼륨설정이나, 속도조절, 색상 선택과 같은 곳에 사용된다.

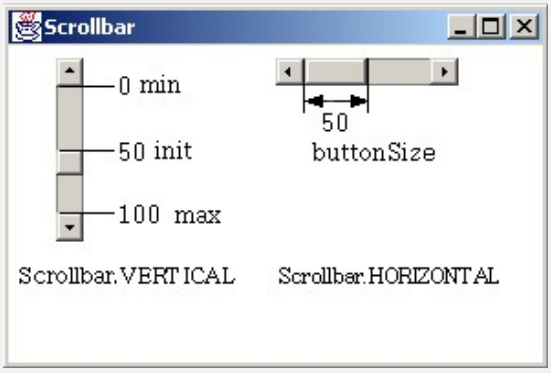
메서드 또는 생성자	설 명
Scrollbar() Scrollbar(int orientation) Scrollbar(int orientation, int value, int visible, int min, int max)	orientation - Scrollbar의 종류를 지정해준다. Scrollbar.VERTICAL, Scrollbar.HORIZONTAL 중의 하나 value - Scrollbar의 초기값 visible - Scroll버튼(bubble)의 크기 min - Scrollbar가 가질 수 있는 최소값 max - Scrollbar가 가질 수 있는 최대값
int getValue()	현재 설정된 Scrollbar의 값을 얻어온다.
void setValue(int newValue)	Scrollbar의 값을 지정된 값(newValue)으로 설정한다.

[표13-12] Scrollbar의 메서드

[예제13-11]/ch13/ScrollbarTest.java

```
import java.awt.*;

class ScrollbarTest {
    public static void main(String args[]) {
        Frame f = new Frame("Scrollbar");
        f.setSize(300, 200);
        f.setLayout(null);

        Scrollbar hor = new Scrollbar(Scrollbar.HORIZONTAL, 0, 50, 0, 100);
        hor.setSize(100, 15);
        hor.setLocation(60, 30);
        Scrollbar ver = new Scrollbar(Scrollbar.VERTICAL, 50, 20, 0, 100);
        ver.setSize(15, 100);
        ver.setLocation(30, 30);

        f.add(hor);
        f.add(ver);
        f.setVisible(true);
    }
}
```

[실행결과]



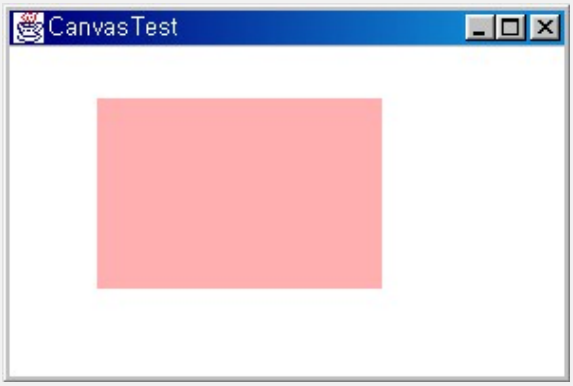
2.10 Canvas

Canvas는 이름에서 알 수 있듯이 빈 평면 공간을 제공하는 컴포넌트이다. 여기에 그림을 그릴수도 있고, 글자를 적을 수도 있다. 주로 그림을 그리거나 이미지를 위한 공간으로 사용되며, 사용자정의 컴포넌트를 만들 때도 사용된다.

[예제13-12]/ch13/CanvasTest.java

```
import java.awt.*;

class CanvasTest {
    public static void main(String args[]) {
          Frame f = new Frame("CanvasTest");
          f.setSize(300, 200);
          f.setLayout(null);         // Frame의 Layout Manager설정을 해제한다.

          Canvas c = new Canvas();
          c.setBackground(Color.pink);    // Canvas의 배경을 분홍색(pink)으로 한다.
          c.setBounds(50, 50, 150, 100);

          f.add(c);        // Canvas을 Frame에 포함시킨다.
          f.setVisible(true);
    }
}
```

[실행결과]



2.11 Panel

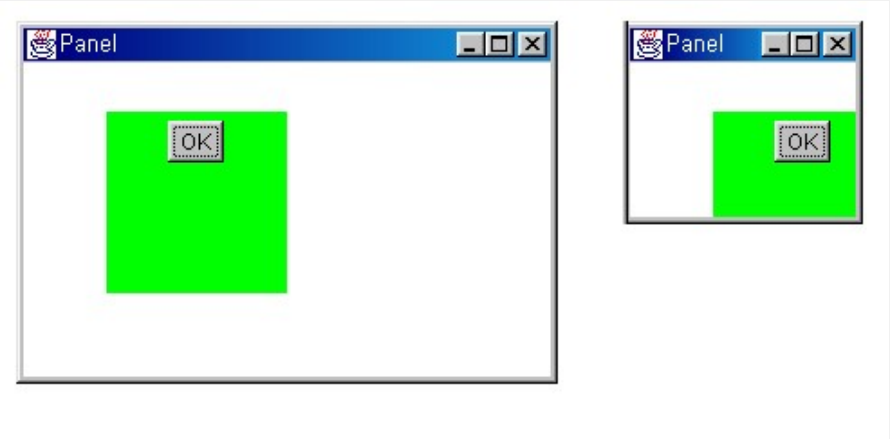
Panel은 Frame과 같이 다른 컴포넌트를 자신의 영역 내에 포함시킬 수 있는 컨테이너다. 동시에 Panel 자신이 다른 컨테이너에 포함될 수 있기도 하다. 심지어는 Panel이 Panel에 포함되는 것도 가능하다.

Panel은 Frame과는 달리 titlebar나 버튼도 없고, 단지 비어있는 평면공간만을 갖는다. Panel도 컨테이너라 자신만의 레이아웃(Layout)을 유지할 수 있어서, Panel을 이용하면 Frame내에서 컴포넌트들의 배치를 다양하게 할 수 있다. 앞으로 레이아웃 관리자에 대해서 배우게 될 텐데, 이 때 Panel이 컴포넌트의 배치에 얼마나 유용하게 사용될 수 있는지 알 수 있을 것이다.

[예제13-13]/ch13/**PanelTest.java**

```
import java.awt.*;

class PanelTest {
    public static void main(String args[]) {
        Frame f = new Frame("Panel");
        f.setSize(300, 200);
        f.setLayout(null);          // Frame이 Layout Manager를 사용하지 않도록 한다.

        Panel p = new Panel();
        p.setBackground(Color.green);   // Panel의 배경을 녹색으로 한다.
        p.setSize(100, 100);
        p.setLocation(50, 50);

        Button ok = new Button("OK");

        p.add(ok);      // Button을 Panel에 포함시킨다.
        f.add(p);       // Panel을 Frame에 포함시킨다.
        f.setVisible(true);
    }
}
```

[실행결과]



위의 결과에서 녹색으로 표시된 부분이 Panel이다. Frame에는 레이아웃 매니저를 사용하지 않았으므로 Panel의 위치와 크기를 지정해 주어야 했지만, Panel은 기본적으로 FlowLayout을 레이아웃 매니저로 사용하므로 Panel안에 포함된 버튼의 위치와 크기는

따로 지정해 주지 않아도 되었다.

Button은 Panel에 포함되어 있는 것이기 때문에 Panel에 지정된 레이아웃 매니저의 영향을 받으며, Frame의 레이아웃 매니저의 영향을 받지는 않는다.

[참고] 오른쪽 그림은 다음에 배우게 될 ScrollPane과 비교하기 위해서 Frame의 크기를 줄여 본 것이다.

2.12 ScrollPane

ScrollPane은 컨테이너이므로, 다른 컴포넌트를 포함시킬 수 있으나 다른 컨테이너들과는 달리 단 하나의 컴포넌트만을 포함시킬 수 있다.

제한된 공간에서 크기가 큰 컴포넌트를 화면에 보여줄 수 있도록 하는데 사용되며, 포함된 컴포넌트의 크기가 ScrollPane 자신보다 큰 경우 스크롤바를 이용해서 볼 수 있게 해준다.

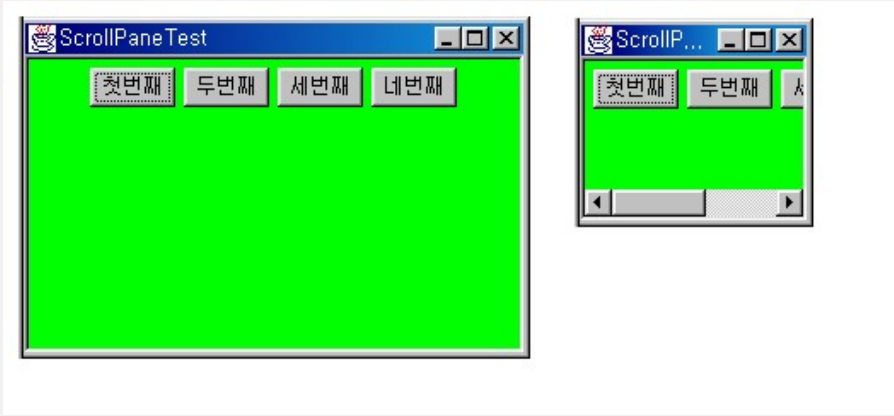
메서드 또는 생성자	설 명
ScrollPane(int scrollbarDisplayPolicy)	srollbarDisplayPolicy - 아래 값 중의 하나를 지정한다. SCROLLBARS_ALWAYS - 스크롤바가 항상 보이게 한다. SCROLLBARS_AS_NEEDED - 필요할 때만 보이게 한다. SCROLLBARS_NEVER - 항상 보이지 않도록 한다.
ScrollPane()	ScrollPane의 객체를 생성한다.

[표13-13] ScrollPane의 메서드

[예제13-14]/ch13/ScrollPaneTest.java

```
import java.awt.*;

class ScrollPaneTest {
    public static void main(String args[]) {
          Frame f = new Frame("ScrollPaneTest");
          f.setSize(300, 200);

          ScrollPane sp = new ScrollPane();
          Panel p = new Panel();
          p.setBackground(Color.green);   // Panel의 배경을 green으로 한다.
          p.add(new Button("첫번째"));      // Button을 Panel에 포함시킨다.
          p.add(new Button("두번째"));
          p.add(new Button("세번째"));
          p.add(new Button("네번째"));

          sp.add(p);       // Panel을 ScrollPane에 포함시킨다.
          f.add(sp);       // ScrollPane을 Frame에 포함시킨다.
          f.setVisible(true);
    }
}
```

[실행결과]



이번 예제에서는 4개의 Button을 가진 Panel을 ScrollPane에 넣고, ScrollPane을 Frame에 넣었다.

위의 결과를 이전의 예제13-13의 실행결과와 비교해보자. Frame의 크기를 줄였을 때, 이전 예제에서는 컴포넌트들이 가려져 Frame의 크기를 늘리기 전까지는 다시 볼 수 없었으나, 이번 예제에서는 Frame의 크기를 줄였을 때, 자동적으로 스크롤바가 나타나서 Frame의 크기를 변경하지 않고도 스크롤바를 통해서 가려진 컴포넌트들을 볼 수 있다.

2.13 Dialog

Dialog는 주로 화면에 메시지창을 보여주는데 사용된다. 프로그램의 실행 중에 사용자에게 에러가 발생했음을 알린다던가, 파일을 삭제하기 전에 사용자로부터 응답을 받아야 한다던가 하는데 사용된다.

Dialog역시 다른 컴포넌트들을 포함할 수 있는 컨테이너이며, Frame과 유사한 모양을 하고 있다.

메서드 또는 생성자	설 명
Dialog(Frame parent, String title, boolean modal)	parent - 어떤 Frame에 속한 것인지 지정. title - Dialog의 titlebar에 나타날 text지정 modal - Dialog를 modal(필수응답)로 할 것 인지를 결정.
Dialog(Frame parent, String title)	modal을 지정하지 않으면, false가 된다.
void show()	Dialog가 화면에 나타나도록 한다.
void hide()	Dialog가 화면에 보이지 않도록 한다.(메모리에는 남아있다.)
void dispose()	Dialog를 닫는다.(화면에 안 보이게 한 후, 메모리에서 제거)
String getTitle()	Dialog의 titlebar에 나타난 text를 얻는다.
void setModal(boolean b)	Dialog를 modal(true) 또는 modaless(false)로 한다.
void setResizable(boolean resizable)	사용자에 의해서 Dialog의 크기가 변경가능/불가능 하도록 한다.(기본적으로 Dialog는 크기변경이 불가능하다.)

[표13-14] Dialog의 메서드

[참고] hide()는 Dialog가 화면에 보이지 않게 할 뿐이지 메모리에서 제거되는 것은 아니다. Dialog를 다시 사용하지 않을 것이라면 dispose()를, 잠시 안보이게만 할 것이라면 hide()를 사용하도록 한다.

[참고] show()와 hide()는 JDK1.5부터 deprecated되었다. 대신 void setVisible(boolean b)를 사용하자.

Dialog의 생성자를 보면, Dialog을 사용하기 위해서는 하나의 Frame이 Dialog의 부모(parent)로 지정되어야 한다. 상속관계의 부모자식 관계는 아니고, 단순히 Dialog가 어느 Frame과 관련된 것인지를 지정해 주기 위한 것이다.

Dialog가 modal(필수응답)인 경우에는 부모로 지정된 Frame은 Dialog가 사라지기 전까지는 사용할 수 없게 된다. 주로 사용자로부터 중요한 확인을 받아야 할 경우에 사용되며, Dialog의 modal값을 false로 설정한 경우에는 Dialog가 열려있더라도 부모 Frame을 사용할 수 있다.

[예제13-15]/ch13/DialogTest.java

```
import java.awt.*;

class DialogTest {
    public static void main(String args[]) {
          Frame f = new Frame("Parent");
          f.setSize(300, 200);

          // parent Frame을 f로 하고, modal을 true로 해서 필수응답 Dialog로 함.
          Dialog info = new Dialog(f, "Information", true);
          info.setSize(140, 90);
          info.setLocation(50, 50);    // parent Frame이 아닌, 화면이 위치의 기준이 된다.
          info.setLayout(new FlowLayout());

          Label msg = new Label("This is modal Dialog", Label.CENTER);
          Button ok = new Button("OK");
          info.add(msg);
          info.add(ok);

          f.setVisible(true);
          info.setVisible(true);     // Dialog를 화면에 보이게 한다.
    }
}
```

[실행결과]

Dialog도 Frame과 같이 기본 레이아웃 매니저가 BorderLayout이기 때문에, FlowLayout으로 변경하였다. Dialog를 modal로 생성했기 때문에, Dialog가 열려있는 동안은 Dialog의 부모(parent)인 Frame을 사용할 수 없다. 위의 예제에서 Dialog를 modal값을 false로 바꾼 다음에 Frame을 사용할 수 있는지 확인해 보도록 한다.

Dialog에 있는 OK버튼을 누르면, Dialog가 닫히도록 해야 하는데, OK버튼을 눌렀을 때의 이벤트처리가 되어 있지 않기 때문에 OK버튼을 눌러도 아무 일도 일어나지 않는다.

다음 예제는 OK버튼에 이벤트처리를 해서, OK버튼을 누르면 Dialog가 닫히도록 했다. 예제를 실행하여, Dialog가 닫힌 후에 parent Frame이 사용가능한지 테스트 해보자.

[예제13-16]/ch13/DialogTest2.java

```
import java.awt.*;
import java.awt.event.*; // 이벤트처리를 위해서 추가해야한다.

class DialogTest2 {
    public static void main(String args[]) {
          Frame f = new Frame("Parent");
          f.setSize(300, 200);

          // parent Frame을 f로 하고, modal을 true로 해서 필수응답 Dialog로 함.
          final Dialog info = new Dialog(f, "Information", true);
          info.setSize(140, 90);
          info.setLocation(50, 50);  // parent Frame이 아닌, 화면기준의 위치
          info.setLayout(new FlowLayout());

          Label msg = new Label("This is modal Dialog", Label.CENTER);
          Button ok = new Button("OK");

          ok.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) { //OK버튼을 누르면 수행됨.
//                    info.setVisible(false);    // Dialog를 안보이게 한다.
                      info.dispose();       // Dialog를 메모리에서 없앤다.
                }
          });
          info.add(msg);
          info.add(ok);
          f.setVisible(true);
          info.setVisible(true);     // Dialog를 화면에 보이게 한다.
    }
}
```

이 예제는 OK버튼에 익명 클래스를 이용해서 이벤트를 처리하는 코드를 추가한 것이다. 앞으로 이벤트 처리에 대해서 자세히 배울 것이므로 지금은 예제를 실행해보고 그 결과를 확인하는 정도면 충분하다.

Dialog의 OK버튼을 눌렀을 때, dispose()를 이용해서 Dialog를 메모리에서 제거할 수도 있고, setVisible(false)를 이용해서 일시적으로 보이지 않게 할 수도 있다.

두 가지 방법 중 어느 것을 사용하던 간에 Dialog가 화면에서 보이지 않게 되며, Frame은 사용가능하게 된다.

2.14 FileDialog

FileDialog는 파일을 열거나 저장할 때 사용되는 Dialog이다. 대부분의 기능들이 이미 구현되어 있기 때문에 따로 코드를 추가하지 않아도 된다.

메서드 또는 생성자	설 명
FileDialog(Frame parent, String title, int mode)	parent - 어떤 Frame에 속한 것인지 지정. title - FileDialog의 titlebar에 나타날 text지정 mode - FileDialog.LOAD, FileDialog.SAVE 중 하나 선택

FileDialog(Frame parent, String title)	mode를 생략하면, 디폴드로 FileDialog.LOAD가 사용된다.
String getFile()	FileDialog에 의해 선택된 파일의 이름을 얻는다.
String getDirectory()	FileDialog에 의해 선택된 파일의 경로(path)를 얻는다.
void setFile(String file)	FileDialog에 지정된 파일을 설정한다.
void setDirectory(String dir)	FileDialog에 지정된 디렉토리를 설정한다.

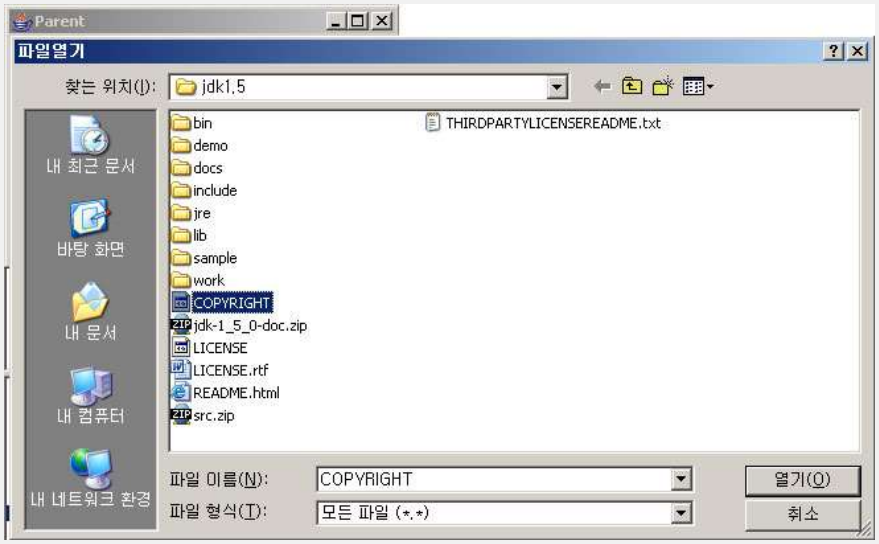
[표13-15] FileDialog의 메서드

[참고] getDirectory()과 getFile()로 선택한 파일의 전체경로를 얻을 수 있다

[예제13-17]/ch13/FileDialogTest.java

```
import java.awt.*;
import java.awt.event.*;

class FileDialogTest {
    public static void main(String args[]) {
        Frame f = new Frame("Parent");
        f.setSize(300, 200);

        FileDialog fileOpen = new FileDialog(f, "파일열기", FileDialog.LOAD);

        f.setVisible(true);
        fileOpen.setDirectory("c:\\jdk1.5");
        fileOpen.setVisible(true);

        //파일을 선택한 다음, FileDialog의 열기버튼을 누르면,
        // getFile()과 getDirectory()를 이용해서 파일이름과 위치한 디렉토리를 얻을 수 있다.
        System.out.println(fileOpen.getDirectory() + fileOpen.getFile());
    }
}
```

[실행결과]



setDirectory("c:₩₩jdk1.5")를 호출해서 FileDialog가 열렸을 때, c:₩jdk1.5디렉토리 내의 파일들이 보이게 된다. 만일 파일을 선택하지 않고 취소버튼을 누르면, getFile()은 null을 반환한다.

[참고] FileDialog.SAVE를 이용해서 파일저장을 위한 프로그램을 만들어 보자.

3. 그 외의 AWT 클래스

지금까지 AWT의 컴포넌트들에 대해서 알아봤는데 이 외에도 AWT에서 사용되는 클래스들을 몇 가지 더 소개하겠다.

3.1 Font

Component의 setFont(Font f)를 이용하면, 컴포넌트에 사용되는 text의 글자체(font)를 원하는 것으로 지정할 수 있다.
그렇게 하기 위해서는 먼저 Font의 인스턴스를 생성해야하는데, Font의 생성자는 아래와 같다.

메서드 또는 생성자	설 명
Font(String name, int style, int size)	name - 사용할 폰트의 이름. (예: "Serif") style - 폰트의 스타일을 지정한다. 보통체(Font.PLAIN), 굵은체(Font.BOLD), 기울임체(Font.ITALIC), 굵은 기울임체(Font.BOLD+Font.ITALIC) 이들 중 하나를 선택. size - 폰트의 크기

[표13-16] Font의 생성자

위 생성자의 매개변수 style에 사용되는 Font.BOLD, Font.PLAIN, Font.ITALIC은 Font의 static멤버인 정수형 상수이다. 그렇기 때문에 Font.BOLD+Font.ITALIC처럼 덧셈 연산이 가능한 것이다. BOLD를 예로 들면, 실제 소스인 Font.java에 다음과 같이 선언되어 있다.

```
public static final int BOLD   = 1;
```

JDK에서는 아래의 5가지 폰트를 기본적으로 제공한다. 그 외의 다른 종류의 폰트를 사용했을 때는 사용된 폰트가 실행될 컴퓨터에 설치되어 있어야만, 바른 실행결과를 얻을 수 있다.

```
▶ Serif
▶ SansSerif
▶ Dialog
▶ DialogInput
▶ Monospaced
```

현재 사용 중인 시스템에 설치된 Font의 리스트를 얻기 위해서는 GraphicsEnvironment 클래스를 사용하면 된다.

[참고] Monospaced는 모든 문자의 폭이 일정한 고정길이 폭(Fixed width) 폰트이다. 즉, i와 l과 같이 폭이 좁은 문자들도 M, W와 같이 폭이 넓은 문자들과 같은 폭으로 표현된다.

[예제13-18]/ch13/FontList.java

```
import java.awt.*;

class FontList
{
    public static void main(String[] args)
    {
          GraphicsEnvironment ge = null;
          ge = GraphicsEnvironment.getLocalGraphicsEnvironment();
          Font[] fonts = ge.getAllFonts();

          for(int i=0; i < fonts.length; i++) {
                System.out.println(fonts[i].getFontName());
          }
    }
}
```

[실행결과]

```
Abadi MT Condensed Light
Albertus Extra Bold
Albertus Medium
Albertus Medium Cursiva
... 중간생략
```

[예제13-19]/ch13/FontTest.java

```
import java.awt.*;

class FontTest {
    public static void main(String args[]) {
          Frame f = new Frame("Font Test");
          String abc = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

          // A 부터 Z를 내용으로 갖는 Label들을 생성한다.
          Label abc1 = new Label(abc);
          Label abc2 = new Label(abc);
          Label abc3 = new Label(abc);
          Label abc4 = new Label(abc);
          Label abc5 = new Label(abc);

          // Serif체이며, 크기가 20인 Font
          Font f1 = new Font("Serif", Font.PLAIN, 20);      // 보통체
          Font f2 = new Font("Serif", Font.ITALIC, 20);     // 기울임체
          Font f3 = new Font("Serif", Font.BOLD, 20);       // 굵은체
          Font f4 = new Font("Serif", Font.BOLD+Font.ITALIC, 20); // 굵은 기울임체

          abc1.setFont(f1);          // Label에 새로운 Font를 적용한다.
          abc2.setFont(f2);
          abc3.setFont(f3);
          abc4.setFont(f4);

          f.setLayout(new FlowLayout());
          f.add(abc1);
          f.add(abc2);
          f.add(abc3);
```

```
            f.add(abc4);
            f.add(abc5);

            f.setSize(400, 200);
            f.setVisible(true);
        }
    }
```

[실행결과]

Font와 setFont(Font f)를 이용해서 Label에 폰트를 Serif로 설정하고 style을 각각 보통체, 굵은체, 기울임체, 굵은 기울임체로 변화를 주어 비교할 수 있도록 하였다.

마지막 Label은 setFont(Font f)로 폰트를 지정해 주지 않았기 때문에 자신이 포함된 컨테이너(Frame)에 기본적으로 설정되어 있는 폰트 설정의 영향을 받았다.

3.2 Color

Color는 색을 표현하기 위해 사용되는 클래스로써, 원하는 색의 RGB값만 알고 있으면 그 색을 표현할 수 있는 객체를 생성하여 사용할 수 있다.

Color클래스 내에는 blue, red, green, yellow 등 자주 쓰이는 색 13가지가 static 멤버변수로 정의되어 있어서 색의 RGB값을 몰라도 쉽게 사용할 수 있다.

주로 컴포넌트의 배경색과 전경색을 설정하는 setForeground(Color c), setBackground(Color c)와 같은 메서드의 매개변수로 사용된다.

메서드 또는 생성자	설 명
Color(int r, int g, int b)	r - red, g - green, b - blue r, g, b 모두 0~255사이의 정수값을 갖는다.
Color(float r, float g, float b)	r - red, g - green, b - blue r, g, b 모두 0.0~1.0사이의 실수값을 갖는다.
Color(int r, int g, int b, int a)	a - alpha값으로 0~255사이의 정수값
Color(float r, float g, float b, float a)	a - alpha값으로 0.0~1.0사이의 실수값

[표13-17] Color의 생성자

[참고] alpha는 색의 불투명도를 뜻한다. 값이 클수록 색이 불투명해진다.

[예제13-20]/ch13/ColorTest.java

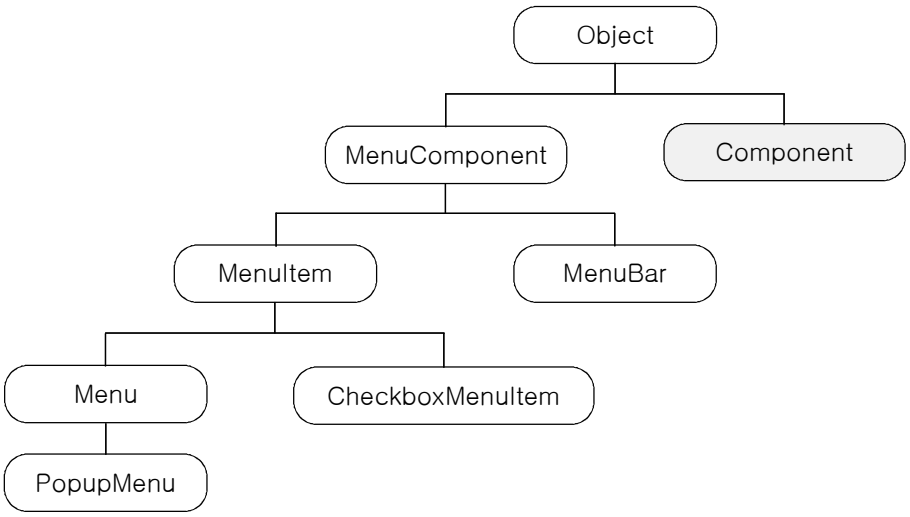
```
import java.awt.*;

class ColorTest {
    public static void main(String args[]) {
          Frame f = new Frame("Color Test");
          f.setLayout(new GridLayout(14, 2));
          Panel p1 = new Panel();     p1.setBackground(Color.black);
          Panel p2 = new Panel();     p2.setBackground(Color.blue);
          Panel p3 = new Panel();     p3.setBackground(Color.cyan);
          Panel p4 = new Panel();     p4.setBackground(Color.darkGray);
          Panel p5 = new Panel();     p5.setBackground(Color.gray);
          Panel p6 = new Panel();     p6.setBackground(Color.green);
          Panel p7 = new Panel();     p7.setBackground(Color.lightGray);
          Panel p8 = new Panel();     p8.setBackground(Color.magenta);
          Panel p9 = new Panel();     p9.setBackground(Color.orange);
          Panel p10 = new Panel();    p10.setBackground(Color.pink);
          Panel p11 = new Panel();    p11.setBackground(Color.red);
          Panel p12 = new Panel();    p12.setBackground(Color.white);
          Panel p13 = new Panel();    p13.setBackground(Color.yellow);
          Panel p14 = new Panel();    p14.setBackground(new Color(50,100,100));
          f.add(new Label("Color.black"));           f.add(p1);
          f.add(new Label("Color.blue"));            f.add(p2);
          f.add(new Label("Color.cyan"));            f.add(p3);
          f.add(new Label("Color.darkGray"));        f.add(p4);
          f.add(new Label("Color.gray"));            f.add(p5);
          f.add(new Label("Color.green"));           f.add(p6);
          f.add(new Label("Color.lightGray"));       f.add(p7);
          f.add(new Label("Color.magenta"));         f.add(p8);
          f.add(new Label("Color.orange"));          f.add(p9);
          f.add(new Label("Color.pink"));            f.add(p10);
          f.add(new Label("Color.red"));             f.add(p11);
          f.add(new Label("Color.white"));           f.add(p12);
          f.add(new Label("Color.yellow"));          f.add(p13);
          f.add(new Label("Color(50, 100, 100)"));   f.add(p14);
          f.setSize(250, 300);
          f.setVisible(true);
    }
}
```

[실행결과]

4. 메뉴만들기

4.1 메뉴를 구성하는 컴포넌트

Frame에 메뉴를 구성하기 위해서는 MenuBar, Menu, MenuItem과 같은 컴포넌트들을 사용한다. 이 들의 상속관계는 다음과 같다.



[그림13-7] 메뉴관련 컴포넌트의 상속계층도

모든 AWT컴포넌트의 조상이 Component인 것처럼 메뉴를 구성하는 컴포넌트들의 조상은 MenuComponent이다.

폴더에 폴더를 담듯이 Menu에 다시 Menu를 담아서 계층형으로 메뉴를 구성할 수 있으며, 항상 화면에 나타나는 최상위 메뉴는 MenuBar에 담고 MenuBar는 다시 Frame에 추가함으로써 메뉴 작성을 마치게 된다.

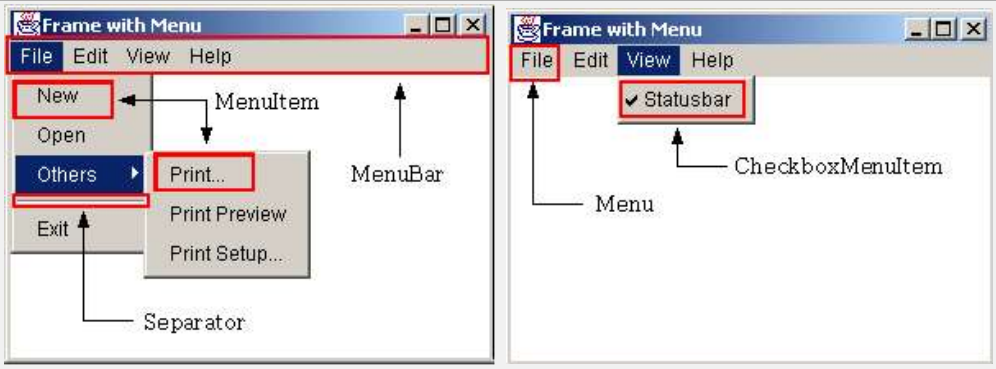
메뉴를 작성하는 것은 간단해서 예제를 보고나면 쉽게 이해가 될 것이다.

[예제13-21]/ch13/**MenuTest**.java

```
import java.awt.*;

class MenuTest {
    public static void main(String args[]) {
          Frame f = new Frame("Frame with Menu");
          f.setSize(300, 200);

          MenuBar mb = new MenuBar();
          Menu mFile = new Menu("File");

          MenuItem miNew = new MenuItem("New");
          MenuItem miOpen = new MenuItem("Open");
          Menu  mOthers = new Menu("Others");       // MenuItem이 아니라 Menu임에 주의
          MenuItem miExit = new MenuItem("Exit");

          mFile.add(miNew);           // Menu에 MenuItem들을 추가한다.
          mFile.add(miOpen);
          mFile.add(mOthers);         // Menu에 Menu를 추가한다.
          mFile.addSeparator();       // 메뉴 분리선을 넣는다.
```

```
        mFile.add(miExit);

        MenuItem miPrint = new MenuItem("Print ...");
        MenuItem miPreview = new MenuItem("Print Preview");
        MenuItem miSetup = new MenuItem("Print Setup ...");
        mOthers.add(miPrint);
        mOthers.add(miPreview);
        mOthers.add(miSetup);

        Menu mEdit = new Menu("Edit");
        Menu mView = new Menu("View");
        Menu mHelp = new Menu("Help");
        CheckboxMenuItem miStatusbar = new CheckboxMenuItem("Statusbar");
        mView.add(miStatusbar);

        mb.add(mFile);              // MenuBar에 Menu를 추가한다.
        mb.add(mEdit);
        mb.add(mView);
        mb.setHelpMenu(mHelp);      // mHelp를 HelpMenu로 지정한다.

        f.setMenuBar(mb);           // Frame에 MenuBar를 포함시킨다.
        f.setVisible(true);
    }
}
```

[실행결과]



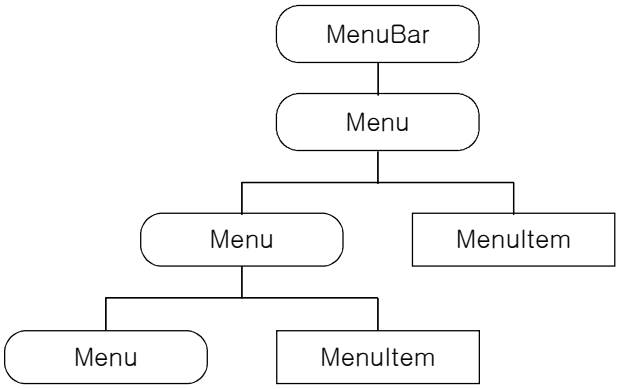
메서드 또는 생성자	설 명
setHelpMenu(Menu menu)	인자로 넘겨받은 menu를 Help menu로 지정한다.
addSeparator()	Menu에 분리선(Separator)을 추가한다.
CheckboxMenuItem(String name, boolean status)	name - CheckboxMenuItem에 보여질 menu이름. status - 값이 true면, 체크된 상태로 생성된다.
CheckboxMenuItem(String name)	name - CheckboxMenuItem에 보여질 menu이름. status값을 지정하지 않으면 기본적으로 false값을 갖는다.

[표13-18] 예제13-21에 사용된 주요 메서드

setHelpMenu()에 의해서 Help메뉴로 지정된 메뉴는 OS마다 다르게 취급된다. 예를 들어, SUN사의 Solaris OS에서는 Help메뉴로 지정된 메뉴는 MenuBar의 맨 오른쪽에 따로 위치하게 되지만, MS의 윈도우OS에서는 메뉴를 Help메뉴로 지정해도 특별히 다르게 다뤄지지 않는다.

이 예제에 사용된 컴포넌트들을 종류에 따라 분류하면 아래와 같다.

Menu - File, Edit, View, Help, Others
MenuItem - New, Open, Print..., Print Preview, Print Setup..., Exit,
CheckboxMenuItem - Statusbar

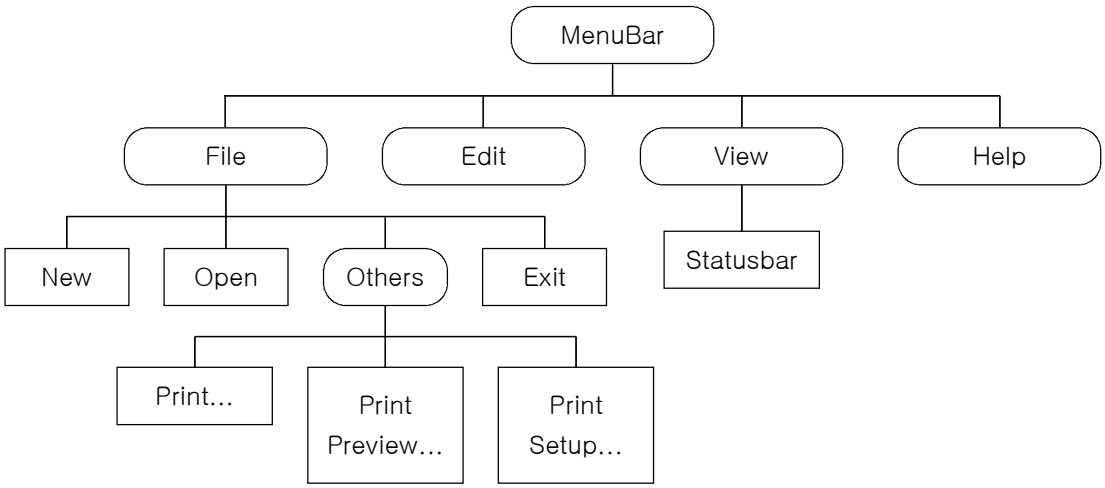


[그림13-8] 메뉴 컴포넌트간의 포함관계

위의 그림은 menu관련 컴포넌트들의 기본 구조를 표현한 것이다. 각 클래스간의 상속관계를 표현한 것이 아니라는 것에 주의하자.

- MenuBar에는 Menu만 포함될 수 있다.(MenuItem은 포함시킬 수 없다.)
- Menu에는 Menu와 MenuItem이 포함될 수 있다.
- 계층형 menu를 만들고자 할 때는 Menu에 Menu를 포함시키면 된다.

아래의 그림은 예제에 사용된 메뉴의 구조를 표현한 것이다.



[그림13-9] 예제13-21의 메뉴구성도

Menu(File)아래에 Menu(Others)를 넣음으로써 계층형 메뉴가 가능하도록 했다. View Menu아래 있는 Statusbar는 CheckboxMenuItem으로써 Checkbox처럼 메뉴를 클릭할 때마다 메뉴 앞에 체크표시가 설정되거나 해제된다.

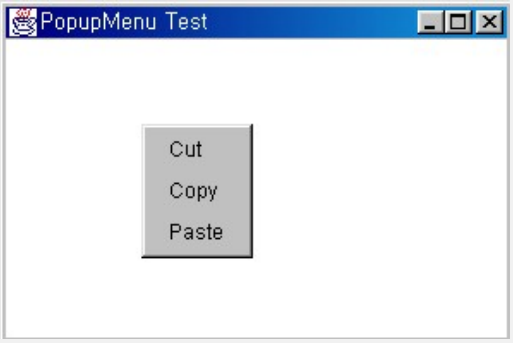
4.2 PopupMenu

PopupMenu는 Frame에 붙어있는 고정적인 Menubar와는 달리, Frame내의 어디서나 마우스의 오른쪽 버튼을 누르면 나타나는, 위치가 고정되어 있지 않은 메뉴이다. 여러분들은 이미 PopupMenu에 익숙해져있으며, 그 편리함도 잘 알고 있을 것이다.

[예제13-22]/ch13/**PopupMenuTest.java**

```
import java.awt.*;
import java.awt.event.*;       // 이벤트 처리를 위해서 추가했다.

class PopupMenuTest {
    public static void main(String args[]) {
          final Frame f = new Frame("PopupMenu Test");
          f.setSize(300, 200);

          final PopupMenu pMenu = new PopupMenu("Edit");
          MenuItem miCut = new MenuItem("Cut");
          MenuItem miCopy = new MenuItem("Copy");
          MenuItem miPaste = new MenuItem("Paste");
          pMenu.add(miCut);     // PopupMenu에 MenuItem들을 추가한다.
          pMenu.add(miCopy);
          pMenu.add(miPaste);

          f.add(pMenu);    // PopupMenu를 Frame에 추가한다.
          f.addMouseListener( new MouseAdapter() { // 익명클래스
               public void mousePressed(MouseEvent me) {
                    // 오른쪽 마우스버튼을 누르면 PopupMenu를 화면에 보여준다.
                    if(me.getModifiers() == me.BUTTON3_MASK)
                         pMenu.show(f, me.getX(), me.getY());
               }
          });
          f.setVisible(true);
    }
}
```

[실행결과]



예제를 실행시킨 후 Frame위에서 마우스 오른쪽 버튼을 누르면 위의 그림과 같이 PopupMenu가 나타난다.
익명 클래스를 main메서드 내에 정의했기 때문에, 익명 클래스에서 Frame타입 참조변수 f와 PopupMenu타입 참조변수 pMenu를 참조하기 위해서 final 키워드를 사용했다.

[참고] 메서드 내에 정의된 클래스에서는, 같은 메서드 내에 선언된 변수들(Local Variable) 중 final 키워드가 붙은 것들만 참조가 가능하다는 것을 기억하자.

Frame에 이벤트처리를 해서 Frame위에서 마우스 오른쪽 버튼을 누르면, PopupMenu가 나타나게 했다. 그리고 getX(), getY()는 Frame내에서의 마우스 포인터의 위치 즉, Frame내에서의 x, y좌표를 얻는데 사용되었다.

[참고] PopupMenu에도 Menu를 포함시킴으로써 계층형 메뉴를 구현할 수 있다.

5. 레이아웃 매니저

AWT프로그래밍에 있어서 화면 구성을 위해 컨테이너 내에 컴포넌트를 적절하게 배치한다는 것은 그리 쉬운 일이 아니다. 하지만 레이아웃 매니저를 이용하면 컨테이너에 컴포넌트 배치를 손쉽게 처리할 수 있다.
AWT에서 제공하는 레이아웃 매니저의 종류와 사용방법에 대해서 알아보도록 하자.

5.1 레이아웃 매니저를 이용한 컴포넌트 배치

레이아웃 매니저는 컨테이너에 포함된 컴포넌트들의 배치(Layout)를 자동적으로 관리해주는 일을 한다. 컨테이너에 새로운 컴포넌트가 추가되거나 컨테이너의 크기가 변경되었을 경우, 컨테이너에 포함된 컴포넌트들의 재배치를 레이아웃 매니저가 자동적으로 처리해주기 때문에 프로그래머가 컴포넌트의 재배치를 위한 코드를 작성하지 않아도 된다.

[참고] 기존의 GUI프로그래밍에서는 컨테이너의 크기가 변경될 때마다 컨테이너에 포함된 컴포넌트들을 재배치하는 코드를 프로그래머가 직접 작성해야 한다.

이처럼 컨테이너에 컴포넌트들을 효율적으로 배치하게 위해서는 레이아웃 매니저의 사용이 필수적이라고 할 수 있다.
AWT에서는 5가지 종류의 레이아웃 매니저를 제공하며, 그 종류는 다음과 같다.

```
- BorderLayout
- FlowLayout
- GridLayout
- CardLayout
- GridbagLayout
```

[참고] 여기서는 GridbagLayout을 제외한 나머지 4가지 레이아웃 매니저에 대해서 알아볼 것이다.

컨테이너에는 단 하나의 레이아웃 매니저만을 설정할 수 있으며 모든 컨테이너는 따로 설정하지 않아도 기본적으로 레이아웃 매니저가 지정되어 있다.
각 컨테이너별 기본 레이아웃 매니저는 다음과 같다.

▶ 컨테이너별 기본 레이아웃 매니저
FlowLayout - Panel, Applet
BorderLayout - Window, Dialog, Frame

자, 이제 레이아웃 매니저를 하나씩 살펴보도록 하자.

5.2 BorderLayout

BorderLayout은 컨테이너를 North, South, East, West, Center 모두 5개의 영역으로 나누고, 각 영역에 하나의 컴포넌트만을 배치할 수 있도록 한다. 그래서 한 영역에 여러 개의 컴포넌트를 배치하면, 마지막에 추가한 컴포넌트만 보이게 된다.
한 영역에 하나 이상의 컴포넌트를 넣기 위해서는 Panel을 이용해야한다. Panel에 원하는 컴포넌트들을 넣은 다음, BorderLayout의 한 영역에 다시 이 Panel을 넣으면 된다.
BorderLayout은 컨테이너인 Window, Frame, Dialog의 기본 Layout이다. 즉, 이 컨테이너들의 레이아웃 매니저는 따로 설정하지 않아도 생성되어질 때부터 이미 BorderLayout으로 설정되어있다는 것이다.

메서드 또는 생성자	설 명
BorderLayout(int hgap, int vgap)	각 영역 사이에 간격이 있는 BorderLayout을 생성한다. hgap - 각 영역의 사이에 간격을 준다.(좌우) vgap - 각 영역의 사이에 간격을 준다.(위아래)
BorderLayout()	영역 사이에 간격이 없는 BorderLayout을 생성한다.
add(String name, Component c) 또는 add(Component c, String name)	c - 추가하려는 컴포넌트 name- "North", "South", "East", "West", "Center"중의 하나 또는 BorderLayout.NORTH, BorderLayout.SOUTH, BorderLayout.EAST,BorderLayout.WEST, BorderLayout .CENTER 중의 하나

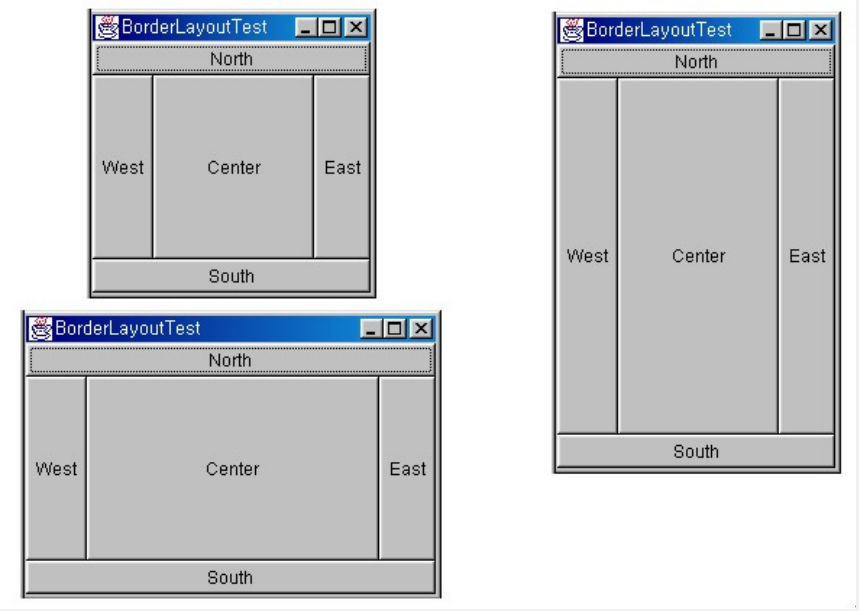
[표13-19] BorderLayout의 메서드

[예제13-23]/ch13/**BorderLayoutTest.java**

```
import java.awt.*;

class BorderLayoutTest {
    public static void main(String args[]) {
        Frame f = new Frame("BorderLayoutTest");
        f.setSize(200, 200);
        // Frame은 기본적으로 BorderLayout로 설정되어 있으므로 따로 설정하지 않아도 됨
        f.setLayout(new BorderLayout());
        Button north = new Button("North");
        Button south = new Button("South");
```

```
        Button east = new Button("East");
        Button west = new Button("West");
        Button center = new Button("Center");

        // Frame의 5개의 각 영역에 Button을 하나씩 추가한다.
        f.add(north, "North");        // f.add("North",north);와 같이 쓸 수도 있다.
        f.add(south, "South");        // South의 대소문자 정확히
        f.add(east, "East");          // East대신, BorderLayout.EAST 사용가능
        f.add(west, "West");
        f.add(center, "Center");

        f.setVisible(true);
    }
}
```

[실행결과]



예제를 실행시킨 후 Frame의 크기를 좌우로 그리고 상하로 변경해 본 것이다. 이때의 변화를 잘 살펴보자.

▶ Frame의 크기를 상하로 늘렸을 때
- West, Center, East영역의 길이만 상하로 늘어나고 North와 South영역에는 아무런 변화가 없다.

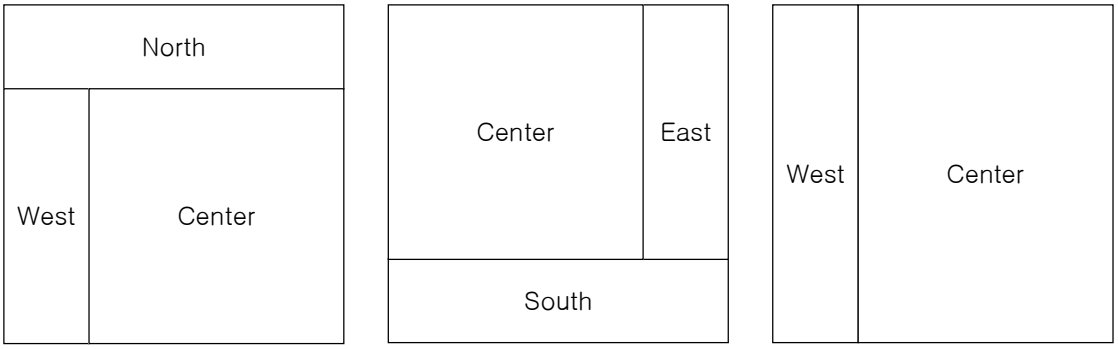
▶ Frame의 크기를 좌우로 늘렸을 때
- North, Center, South영역의 길이만 좌우로 늘어나고 West와 East영역에는 아무런 변화가 없다.

[참고] 컨테이너의 크기가 변경되어도 North와 South영역에 배치된 컴포넌트의 높이(height)와 East와 West영역에 배치된 컴포넌트의 너비(width)는 적절한 크기(preferred size)를 그대로 유지한다.

[참고] preferred size - 컴포넌트들이 생성될 때 지정된 값과 설정된 폰트의 크기 등의 정보를 바탕으로 계산된 크기. BoderLayout과 FlowLayout은 컴포넌트 배치에 컴포넌트의 preferred size를 사용한다.

BorderLayout의 각 영역에 컴포넌트를 추가 할 때는, 컨테이너에 add(Component c, String area) 또는 add(String area, Component c)를 사용한다. 5개의 영역을 모두 채워줘야 하는 것은 아니고 필요한 영역에만 컴포넌트를 추가할 수 있다.

아래의 그림들은 BorderLayout으로 가능한 컴포넌트 배치의 몇 가지 예를 보여준다.



[그림13-10] BorderLayout으로 구성가능한 화면의 예

5.3 FlowLayout

FlowLayout은 컴포넌트들이 추가되는 순서에 따라 왼쪽에서 오른쪽으로 이어져나가며, 공간이 부족하면, 아랫줄에 추가된다.

이것은 마치 글자를 입력하면 왼쪽에서 오른쪽으로 진행해 나가며, 한 라인의 끝에 다다르면, 다음 라인의 처음부터 글자가 입력되는 워드프로세서에서의 문자입력과 매우 유사하다.

또한 컴포넌트의 정렬에 있어서도 워드프로세서에서와 같이 왼쪽 정렬, 오른쪽 정렬, 그리고 가운데 정렬이 가능하다.

메서드 또는 생성자	설 명
FlowLayout(int align, int hgap, int vgap)	align - 컴포넌트들의 정렬 방법을 지정한다. FlowLayout.LEFT(왼쪽 정렬), FlowLayout.CENTER(가운데 정렬) FlowLayout.RIGHT(오른쪽 정렬) 중 하나를 지정한다. hgap - 각 컴포넌트간의 사이에 간격을 준다.(좌우) vgap - 각 컴포넌트간의 사이에 간격을 준다.(위아래)
FlowLayout(int align)	hgap과 vgap이 5픽셀인 FlowLayout을 생성한다. align - 컴포넌트들의 정렬 방법을 지정한다.
FlowLayout()	가운데 정렬이면서 hgap과 vgap이 5픽셀인 FlowLayout을 생성한다.

[표13-20] FlowLayout의 생성자

[예제13-24]/ch13/FlowLayoutTest.java

```
import java.awt.*;

class FlowLayoutTest {
    public static void main(String args[]) {
          Frame f = new Frame("FlowLayoutTest");
          f.setSize(250, 100);
          f.setLayout(new FlowLayout(FlowLayout.LEFT)); // 왼쪽정렬의 FlowLayout생성
```

```
            f.add(new Button("첫 번째"));
            f.add(new Button("두 번째"));
            f.add(new Button("세 번째"));
            f.add(new Button("네 번째"));
            f.add(new Button("다섯 번째"));
            f.setVisible(true);
        }
    }
```

[실행결과]

위의 결과에서도 볼 수 있듯이, Frame의 크기를 변경함에 따라 컴포넌트들의 배치가 달라지는 것을 볼 수 있다. 컴포넌트들이 추가된 순서와 Frame의 크기를 늘렸을 때와 줄였을 때의 컴포넌트들의 달라진 배치를 유심히 살펴보기 바란다.

컨테이너인 Frame의 크기가 변해도, 포함된 컴포넌트들의 크기는 변하지 않는데, 레이아웃 매니저가 FlowLayout으로 설정된 컨테이너에 추가된 컴포넌트들은 컨테이너의 크기가 변해도 컴포넌트의 크기는 영향을 받지 않고 배치에만 영향을 받는다. 이 때 컴포넌트들은 적절한 크기(preferred size)를 유지하게 된다.

5.4 GridLayout

GridLayout은 컨테이너를 테이블처럼 행과 열로 나누어 컴포넌트를 배치한다. 여기에 추가되는 컴포넌트들은 모두 같은 크기로 나누어지며, 컨테이너의 크기를 변경하면 각 영역이 모두 같은 비율로 커지거나 작아지는 성질을 갖고 있다.

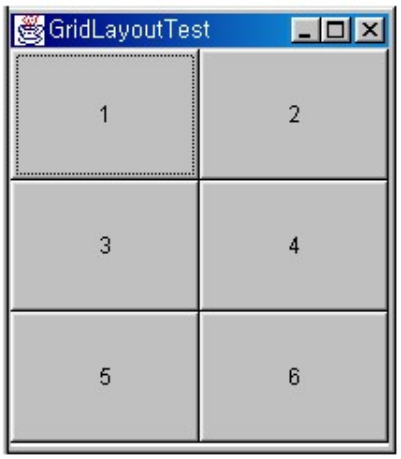
메서드 또는 생성자	설 명
GridLayout(int row, int col, int hgap, int vgap)	영역들 간의 사이에 간격이 있는 GridLayout을 생성한다. row - 컨테이너를 몇 개의 행(row)으로 나눌 것인지 적는다. col - 컨테이너를 몇 개의 열(column)로 나눌 것인지 적는다. hgap - 각 영역간의 사이에 간격을 준다.(좌우) vgap - 각 영역간의 사이에 간격을 준다.(위아래)
GridLayout(int row, int col)	영역들 간의 사이에 간격이 없는 GridLayout을 생성한다. row - 컨테이너를 몇 개의 행(row)으로 나눌 것인지 적는다. col - 컨테이너를 몇 개의 열(column)로 나눌 것인지 적는다.

[표13-21] GridLayout의 생성자.

[예제13-25]/ch13/GridLayoutTest.java

```
import java.awt.*;

class GridLayoutTest {
    public static void main(String args[]) {
          Frame f = new Frame("GridLayoutTest");
          f.setSize(150, 150);
          f.setLayout(new GridLayout(3, 2)); // 3행 2열의 테이블을 만든다.
          f.add(new Button("1"));            // 추가되는 순서대로 Button에 번호를 붙였다.
          f.add(new Button("2"));
          f.add(new Button("3"));
          f.add(new Button("4"));
          f.add(new Button("5"));
          f.add(new Button("6"));

          f.setVisible(true);
    }
}
```

[실행결과]



위의 결과를 보면, Frame에 Button이 추가될 때 어떤 순서로 GridLayout의 영역에 추가되는지 쉽게 알 수 있을 것이다. 첫 번째 행의 열들을 모두 채운 후에 그 아래 행의 열들을 채운다. 이와 같은 방식으로 컴포넌트들이 추가되게 된다.

그리고, 오른쪽의 그림은 실행 후 Frame의 크기를 늘린 것인데, 늘어난 후에도 각 영역의 Button들이 모두 같은 크기를 유지하는 것을 알 수 있다.

[참고] GridbagLayout은 GridLayout과 같이 컨테이너를 열과 행으로 나누어 컴포넌트들을 배치할 수 있다. 각 영역은 서로 다른 크기로 지정될 수 있으며, 인접한 열 또는 행으로의 확장이 가능하다. 마치 HTML문서에서의 Table의 레이아웃을 설정하는 것과 비슷하다.

5.5 CardLayout

CardLayout은 여러 화면을 슬라이드처럼 바꿔가며 보여줄 수 있다. 즉, 여러 개의 컨테이너를 CardLayout에 추가한 다음, 순서대로 또는 임의의 컨테이너를 선택해서 보여줄 수 있다. 설치프로그램과 같이 단계별로 다른 화면으로 이동하는 경우에 사용하면 유용할 것이다.

메서드 또는 생성자	설 명
CardLayout(int hgap, int vgap)	hgap - 컨테이너와 CardLayout 사이에 간격을 준다.(수평) vgap - 컨테이너와 CardLayout 사이에 간격을 준다.(수직)
CardLayout()	컨테이너와 간격이 없는 CardLayout을 생성한다.
add(Container parent, String name)	name - 주어진 이름으로, parent - 지정된 컨테이너에 추가한다.
show(Container parent, String name)	주어진 이름의 컴포넌트를 컨테이너에 보여준다. name - 나타낼 컨테이너의 이름(추가할 때 사용한 이름) parent - 컴포넌트가 보여질 컨테이너
first(Container parent)	parent - 지정된 컨테이너에 첫 번째로 추가된 컴포넌트를 보여준다.
last(Container parent)	parent - 지정된 컨테이너에 마지막으로 추가된 컴포넌트를 보여준다.
previous(Container parent)	지정된 컨테이너에 현재 보이는 컴포넌트보다 이전에 추가된 것을 보여준다.(현재 보이는 것이 첫 번째 것이면, 제일 마지막 것이 나타나게 된다.)
next(Container parent)	지정된 컨테이너에 현재 보이는 컴포넌트보다 다음에 추가된 것을 보여준다.(현재 보이는 것이 마지막 것이면, 제일 첫 번째 것이 나타나게 된다.)

[표13-22] CardLayout의 메서드

[예제13-26]/ch13/CardLayoutTest.java

```
import java.awt.*;
import java.awt.event.*;

class CardLayoutTest {
    public static void main(String args[]) {
          final Frame f = new Frame("CardLayoutTest");
          final CardLayout card = new CardLayout(10, 10);
          f.setLayout(card);

          Panel card1= new Panel();
          card1.setBackground(Color.lightGray);
          card1.add(new Label("Card 1"));
          Panel card2= new Panel();
          card2.add(new Label("Card 2"));
          card2.setBackground(Color.orange);
          Panel card3= new Panel();
          card3.add(new Label("Card 3"));
          card3.setBackground(Color.cyan);

          f.add(card1, "1");          // Frame에 card1을 "1"이라고 이름 붙여 추가한다.
          f.add(card2, "2");
```

```
        f.add(card3, "3");

        class Handler extends MouseAdapter {
            public void mouseClicked(MouseEvent e) {
                // 마우스 오른쪽 버튼을 눌렀을 때
                if(e.getModifiers() == e.BUTTON3_MASK)     {
                    card.previous(f);  //  CardLayout의 이전 Panel을 보여준다.
                } else {
                    card.next(f);     //  CardLayout의 다음 Panel을 보여준다.
                }
            }
        } // class Handler

        card1.addMouseListener(new Handler());
        card2.addMouseListener(new Handler());
        card3.addMouseListener(new Handler());

        f.setSize(200, 200);
        f.setLocation(200, 200);
        f.setVisible(true);

        card.show(f,"1");     // Frame에 추가된 Component중 이름이 "1"인 것을 보여준다.
    }
}
```

[실행결과]

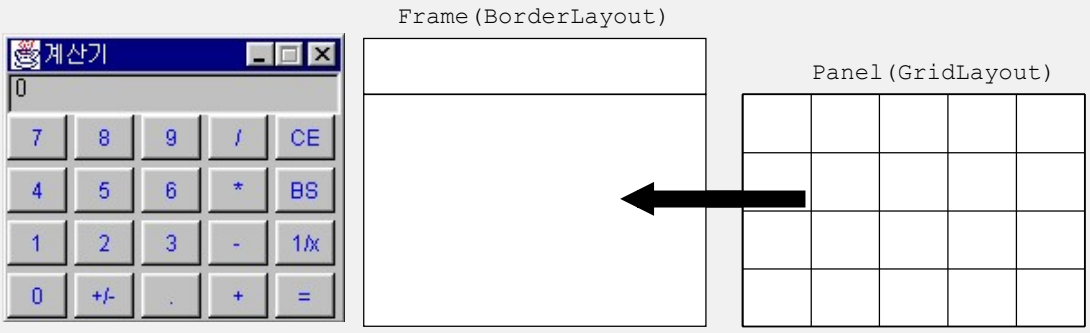
위의 예제는 Frame의 레이아웃 매니저를 CardLayout으로 하고, CardLayout에 3개의 Panel을 추가하였다. 실행하면 첫 번째 Panel(Card 1)이 보이도록 했으며, Panel위를 마우스 왼쪽 버튼으로 클릭하면, 두 번째 Panel(Card 2)이 보이도록 했다. 또 마우스 오른쪽 버튼을 누르면, 클릭한 Panel 보다 이전에 있는 Panel이 보이도록 했다.

[예제13-27]/ch13/Calc.java

```
import java.awt.*;
import java.awt.event.*;

class Calc {
    public static void main(String args[]) {
        Frame f = new Frame("계산기");
        TextField tf = new TextField("0");
        tf.setEditable(false);
        f.setSize(190, 160);
```

```
        f.setLocation(300,300);

        f.add("North", tf);
        Panel numPanel = new Panel();
        Button[] numButtons = null;
        numPanel.setLayout(new GridLayout(4, 5, 4, 4));
        numPanel.setBackground(Color.lightGray);
        f.add("Center", numPanel);

        String numStr[] = { "7", "8", "9", "/", " CE ",
                            "4", "5", "6", "*", "BS",
                            "1", "2", "3", "-", "1/x",
                            "0", "+/-", ".", "+", "=" };
        numButtons = new Button[numStr.length];
        for(int i=0;i<numStr.length;i++) {
              numButtons[i] = new Button(numStr[i]);
              numButtons[i].setForeground(Color.blue);
              numPanel.add(numButtons[i]);
        }

        f.setResizable(false);
        f.setVisible(true);
    }
}
```

[실행결과]



이번 예제에서는 BorderLayout과 GridLayout, 이 두 개의 레이아웃 매니저를 이용해서 간단한 계산기를 만들어 보았다. 이벤트 처리를 하지 않았기 때문에 아무런 기능도 못하지만, Panel을 이용한 조금 복잡한 레이아웃으로 구성되어 있다.

Frame은 North와 Center 두 영역으로만 있는 BorderLayout으로 설정하고, Frame의 Center영역에 4행 5열의 GridLayout으로 구성된 Panel을 추가한다.

심지어는 Panel을 Panel에 넣는 방식으로 더 다양하고 복잡한 Layout의 화면을 만들 수 있다. 계산결과를 보여주는 창을 TextField로 하였는데, TextField는 text의 정렬이 안되기 때문에 오른쪽이 아닌 왼쪽에 '0'이 있다. 이것을 개선하기 위해서는 Label을 사용하던가 Swing의 JTextField를 사용하면 된다. Swing과 AWT의 컴포넌트 간에는 서로 섞어쓰지 않는 것이 좋지만, 이런 경우에는 별문제 없다.

[참고] 많은 수의 Button을 가지고 작업을 해야 하기 때문에 배열과 반복문을 이용하여 코드를 간단히 했다. 이렇게 하는 것이 Button의 이름 또는 순서를 바꾸는데 더 용이하다.

6. 이벤트 처리(event handling)

6.1 이벤트(event)란?

이벤트(Event)란, 사용자 또는 프로그램 코드에 의해서 발생할 수 있는 사건을 말한다. 예를 들면, 사용자가 마우스를 움직이거나 클릭할 때, 키보드를 눌렀을 때, 또는 Frame의 크기를 변경할 때 이벤트가 발생한다.

종류	설명
이벤트 소스 (Event Source, 이벤트 발생지)	이벤트가 발생한 컴포넌트. 사용자가 Button을 눌렀을 때 이벤트가 발생하고, Button은 이 이벤트의 이벤트 소스가 된다.
이벤트 핸들러 (Event Handler, 이벤트 처리기)	이벤트가 발생했을 때 실행될 코드를 구현해 놓은 클래스
이벤트 리스너 (Event Listener, 이벤트 감지기)	이벤트를 감지하고 처리한다. 이벤트 핸들러를 이벤트 리스너로 이벤트 소스에 연결해야 이벤트가 발생했을 때 이벤트가 처리된다.

[표13-23] 이벤트 관련 용어설명

이벤트에 대한 수행코드를 작성하여 이벤트 소스에 이벤트 리스너(Event Listener)로 등록하는 것을 이벤트 처리(Event handling)라고 한다. 이벤트가 발생하더라도 그에 대한 이벤트 처리를 하지 않으면, 아무 일도 일어나지 않는다.
사실 사용자가 AWT프로그램을 사용하면서 행하는 모든 동작은 이벤트를 발생시킨다. 단지 그에 대한 이벤트 처리가 되어있지 않아서 이벤트가 발생한 사실을 사용자가 알지 못하는 것이다.

6.2 이벤트의 발생과 처리

이벤트가 발생하면 해당 이벤트 클래스의 인스턴스가 생성된다.(ActionEvent가 발생하면 ActionEvent인스턴스가 생성된다). 그 다음에 이벤트를 처리할 Listener(이벤트 리스너)가 이벤트 소스에 등록되어 있는지 확인해서, 등록된 Listener중에서 발생한 이벤트를 처리할 Listener가 있으면, 이 Listener가 이벤트를 처리하게 된다. 반대로 발생한 이벤트를 처리할 Listener가 등록되어 있지 않으면, 아무 일도 일어나지 않는다.

이벤트의 발생에서부터 처리까지의 과정에 대해서 알아봤는데, 예외처리와 유사하다는 느낌을 많이 받았을 것이다. 예외 발생 시에 해당하는 종류의 예외클래스의 인스턴스가 생성되는 점 그리고, 예외가 발생한 곳에서 발생한 예외의 종류와 일치하는 catch블럭(이벤트에서는 Listener)이 처리하는 점, 예외 발생 시 생성된 객체를 catch블럭 내에서 사용 가능하다는 점 등 처리방법이 서로 많이 닮아있으므로 이벤트 처리를 예외처리와 비교해 보면 보다 쉽게 이해 할 수 있을 것이다.

[참고] 예외처리와 이벤트처리의 차이점은, 이벤트는 처리해주지 않아도 아무런 문제가 되지 않는다는 것이다.

6.3 이벤트 처리방법

AWT프로그램 실행 중에 사용자의 어떠한 동작에 의해서 이벤트가 발생했을 때, 이에 대해 적절한 작업을 수행되도록 하는 것을 이벤트 처리(Event handling)이라고 한다.

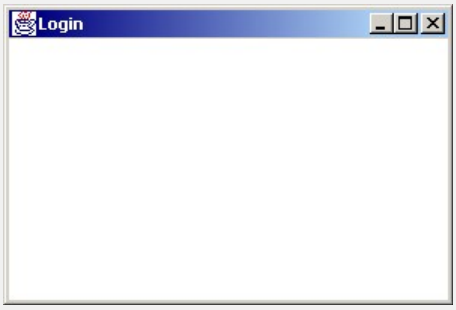
예제13-1에서 Frame의 닫기 버튼을 눌러도 아무런 일도 일어나지 않는 이유는, 이 이벤트가 발생했을 때 작업을 수행하는 코드를 작성하지 않았기 때문이다.

이 예제에 이벤트 처리를 추가하여 닫기 버튼 을 눌렀을 때 Frame이 닫히도록 하겠다.

[예제13-28]/ch13/**FrameTest3**.java

```
import java.awt.*;
import java.awt.event.*;

class FrameTest3 {
    public static void main(String args[]) {
          Frame f = new Frame("Login");     // Frame객체를 생성한다.
          f.setSize(300, 200);              // Frame의 크기를 설정한다.

          // EventHandler클래스의 객체를 생성해서 Frame의 WindowListener로 등록한다.
          f.addWindowListener(new EventHandler());
          f.setVisible(true);                   // 생성한 Frame을 화면에 보이도록 한다.
    }
}

class EventHandler implements WindowListener
{
    public void windowOpened(WindowEvent e) {}
    public void windowClosing(WindowEvent e) { // Frame의 닫기 버튼을 눌렀을 때 호출된다.
          e.getWindow().setVisible(false);      // Frame을 화면에서 보이지 않도록 하고
          e.getWindow().dispose();              // 메모리에서 제거한다.
          System.exit(0);                       // 프로그램을 종료한다.
    }
    public void windowClosed(WindowEvent e) {} // 아무내용도 없는 메서드 구현
    public void windowIconified(WindowEvent e) {}
    public void windowDeiconified(WindowEvent e) {}
    public void windowActivated(WindowEvent e) {}
    public void windowDeactivated(WindowEvent e) {}
}
```

[실행결과]



이 예제를 실행시킨 후, Frame의 오른쪽 맨 위의 닫기 버튼을 누르면, 예제13-1과는 달리 Frame이 닫히게 될 것이다. 그 이유는, 닫기 버튼을 누르는 이벤트가 발생했을 때 Frame을 닫는 작업을 하는 코드를 추가했기 때문이다.

▶ 이벤트 처리 방법

1. 표13-25에 있는 메서드 중에서 필요한 것을 찾는다.

```
windowClosing(WindowEvent we)
```

2. 선택한 메서드가 속해있는 인터페이스를 구현하는 클래스를 작성한다.(표13-24 참고)

```
class EventHandler implements WindowListener {
     public void windowClosing(WindowEvent e) { /* 코드 작성 */  }
      ...
}
```

3. 위에서 구현한 클래스의 인스턴스를 생성해서 이벤트 소스에 Listener로 등록한다.

```
f.addWindowListener(new EventHandler());
```

[참고] Listener등록에 필요한 메서드는 표13-27을 참고하자.

EventHandler클래스는 WindowListener인터페이스를 구현해야하므로 WindowListener인터페이스에 정의되어 있는 모든 추상 메서드의 몸통(body) 부분을 만들어 주어야만 한다. 그래서, 내용은 없지만 메서드의 몸통을 만들어 주었다. 이런 메서드들은 이벤트가 발생하여 호출되어도 실행될 문장이 하나도 없기 때문에 아무 일도 하지 않고 바로 종료된다.

```
class EventHandler implements WindowListener
{
     public void windowOpened(WindowEvent e) {}
     public void windowClosing(WindowEvent e) { //Frame의 닫기 버튼을 누르면 호출된다.
          e.getWindow().setVisible(false);     //Frame을 화면에서 보이지 않도록 하고
          e.getWindow().dispose();             //메모리에서 제거한다.
          System.exit(0);                      //프로그램을 종료한다.
     }
     public void windowClosed(WindowEvent e) {} //아무내용도 없는 메서드 구현
     public void windowIconified(WindowEvent e) {}
     public void windowDeiconified(WindowEvent e) {}
     public void windowActivated(WindowEvent e) {}
     public void windowDeactivated(WindowEvent e) {}
}
```

windowClosing메서드는 Frame의 닫기 버튼을 눌렀을 때 호출되므로 어플리케이션을 종료하는 코드를 추가했다.

Frame의 닫기 버튼을 눌렀을 때, 그 이후의 진행과정은 다음과 같다.

```
사용자가 Frame의 닫기 버튼을 누르면,
1. WindowEvent가 발생하고(WindowEvent의 인스턴스가 생성됨),
2. Frame에 WindowListener로 등록되어 있는 이벤트 핸들러의 windowClosing메서드를
호출한다. 이 메서드 내에서는 이벤트 발생시 생성된 WindowEvent인스턴스의 참조를
사용할 수 있어서 WindowsEvent인스턴스의 메서드들을 사용할 수 있다.
```

이벤트	인터페이스	메서드
ActionEvent	ActionListener	actionPerformed(ActionEvent ae)
ComponentEvent	ComponentListener	componentMoved(ActionEvent ae)
		componentHidden(ActionEvent ae)
		componentResized(ActionEvent ae)
		componentShown(ActionEvent ae)
MouseEvent	MouseMotionListener	mouseDragged(MouseEvent me)
		mouseMoved(MouseEvent me)
	MouseListener	mousePressed(MouseEvent me)
		mouseReleased(MouseEvent me)
		mouseEntered(MouseEvent me)
		mouseExited(MouseEvent me)
		mouseClicked(MouseEvent me)
MouseWheelEvent	MouseWheelListener	mouseWheelMoved(MouseWheelEvent e)
KeyEvent	KeyListener	keyPressed(KeyEvent ke)
		keyReleased(KeyEvent ke)
		keyTyped(KeyEvent ke)
TextEvent	TextListener	textValueChanged(TextEvent te)
FocusEvent	FocusListener	focusGained(FocusEvent fe)
		focusLost(FocusEvent fe)
ItemEvent	ItemListener	itemStateChanged(ItemEvent ie)
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged(AdjustmentEvent ae)
WindowEvent	WindowListener	windowClosing(WindowEvent we)
		windowOpened(WindowEvent we)
		windowIconified(WindowEvent we)
		windowDeiconified(WindowEvent we)
		windowClosed(WindowEvent we)
		windowActivated(WindowEvent we)
		windowDeactivated(WindowEvent we)
	WindowFocusListener	windowGainedFocus(WindowEvent e)
		windowLostFocus(WindowEvent e)
	WindowStateListener	windowStateChanged(WindowEvent e)
ContainerEvent	ContainerListener	componentAdded(ContainerEvent ce)
		componentRemoved(ContainerEvent ce)

[표13-24] Event의 종류와 관련 인터페이스

[참고] MouseWheelListener, WindowFocusListener, WindowStateListener는 JDK 1.4 부터 추가되었다.

메서드	호출시기
actionPerformed(ActionEvent ae)	Button을 클릭했을 때, Menu를 클릭했을 때, TextField에서 Enter키를 눌렀을 때, List의 item하나를 선택하여 더블클릭했을 때
componentMoved(ActionEvent ae)	컴포넌트가 이동되었을 때
componentHidden(ActionEvent ae)	컴포넌트가 화면에 보이지 않게 되었을 때
componentResized(ActionEvent ae)	컴포넌트의 크기가 변경되었을 때
componentShown(ActionEvent ae)	컴포넌트가 화면에 보일 때
mouseDragged(MouseEvent me)	마우스 버튼을 누른 채로 마우스를 움직였을 때
mouseMoved(MouseEvent me)	마우스 포인터를 이동시킬 때
mousePressed(MouseEvent me)	마우스 버튼을 눌렀을 때
mouseReleased(MouseEvent me)	마우스 버튼을 떼었을 때
mouseEntered(MouseEvent me)	마우스 포인터가 이벤트 소스의 영역 안으로 들어왔을 때
mouseExited(MouseEvent me)	마우스 포인터가 이벤트 소스의 영역 안에서 밖으로 이동할 때
mouseClicked(MouseEvent me)	마우스 버튼을 눌렀다가 떼었을 때
mouseWheelMoved(MouseWheelEvent e)	마우스의 휠을 움직였을 때
keyPressed(KeyEvent ke)	키보드의 키를 눌렀을 때
keyReleased(KeyEvent ke)	키보드의 키를 떼었을 때
keyTyped(KeyEvent ke)	키보드의 키를 눌렀다 떼었을 때
textValueChanged(TextEvent te)	TextField 또는 TestArea의 내용이 변경되었을 때
focusGained(FocusEvent fe)	이벤트 소스로 focus가 이동했을 때
focusLost(FocusEvent fe)	이벤트 소스가 갖고 있던 focus가 다른 컴포넌트로 이동했을 때
itemStateChanged(ItemEvent ie)	Checkbox, CheckboxItem, List, Choice의 status가 변경되었을 때 (selected ↔ unselected)
adjustmentValueChanged(Adjustment Event ae)	Scrollbar의 값이 변경되었을 때
windowOpened(WindowEvent we)	윈도우가 열렸을 때
windowClosing(WindowEvent we)	윈도우가 닫힐 때(닫기 버튼을 눌렀을 때)
windowClosed(WindowEvent we)	윈도우가 닫혔을 때(dispose()가 호출 되었을 때)
windowIconified(WindowEvent we)	윈도우가 최소화(아이콘화) 되었을 때
windowDeiconified(WindowEvent we)	윈도우가 최소화 상태에서 다시 원래 크기로 되었을 때
windowActivated(WindowEvent we)	윈도우가 활성화 되었을 때
windowDeactivated(WindowEvent we)	윈도우가 비활성화 되었을 때
windowGainedFocus(WindowEvent e)	윈도우가 포커스를 얻을 때
windowLostFocus(WindowEvent e)	윈도우가 포커스를 잃었을 때
windowStateChanged(WindowEvent e)	윈도우의 상태가 변했을 때
componentAdded(ContainerEvent ce)	컨테이너에 컴포넌트가 추가되었을 때
componentRemoved(ContainerEvent ce)	컨테이너에 컴포넌트가 제거되었을 때

[표13-25] 이벤트 리스너를 Component에 추가/제거하는 메서드

[참고] 표13-25에 언급된 윈도우는 컨테이너인 Window와 그 자손들(Frame, Dialog, FileDialog)을 뜻한다.

ActionEvent는 고수준 이벤트로 컴포넌트에 정의된 동작이 수행되었을 때 발생한다. 예를 들어 버튼(Button)을 마우스로 클릭하거나, 버튼이 포커스(focus)를 가졌을 때 스페이스바를 입력하면 눌러진다. 이 때 MouseEvent나 KeyEvent가 발생하는 것이 아니라 ActionEvent가 발생한다. 실제로는 MouseEvent나 KeyEvent가 발생하지만 최종적으로는 ActionEvent가 발생한 것으로 처리된다.

그래서 ActionListener의 actionPerformed(ActionEvent ae)에 버튼이 눌러졌을 때 수행될 코드를 작성해야한다.

▶ ActionEvent가 발생하는 경우

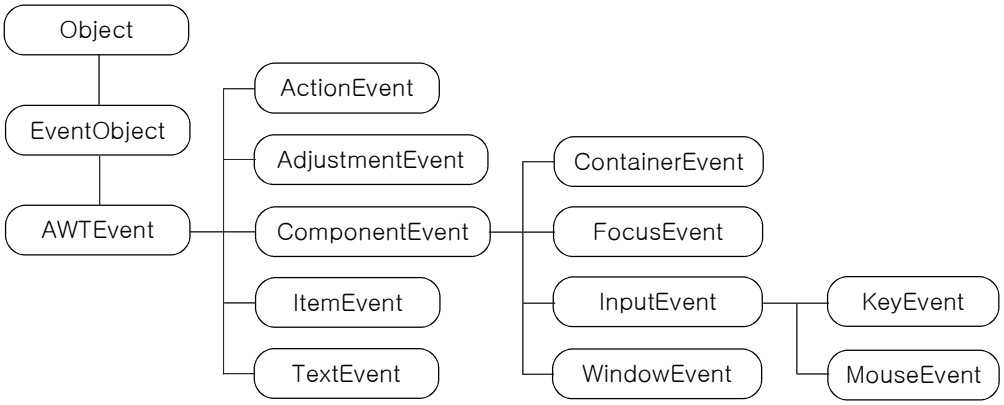
- Button이 눌러졌을 때
- Menu를 클릭했을 때
- TextField에서 Enter키를 눌렀을 때
- List의 item하나를 선택하여 더블클릭했을 때

이렇게 하면 버튼을 누르는 여러 가지 방법이 존재하더라도 ActionEvent가 발생하고 이에 대한 처리만 하면 된다는 장점이 있다.

MS윈도우와 같은 GUI OS에서는 여러 윈도우가 열려있어도 그 중의 하나만 사용자가 작업을 할 수 있다. 현재 작업 중인 윈도우(여러 윈도우 중에서 가장 위에 나타나는 윈도우)를 활성윈도우(active window)라고 하며, 사용자가 다른 작업을 위해 다른 윈도우를 선택하면, 이전에 작업 중이던 윈도우는 비활성화(deactivated)된다.

다음 그림은 이벤트 관련 클래스간의 상속계층도를 그린 것이다. AWT이벤트의 최고조상은 AWTEvent인 것을 알 수 있다.

각 이벤트간의 상속관계가 중요한 것은 아니지만 사용자 이벤트를 만들 때 참고하자.



[그림13-11] Event의 상속계층도

[참고] 이벤트 관련 클래스는 java.awt.event패키지에 속한다.

try-catch구문의 catch블럭 내에서 발생한 예외의 인스턴스를 참조할 수 있는 것처럼, 각 메서드에서는 발생한 Event의 인스턴스에 대한 참조를 사용할 수 있으므로, 이벤트 발생 시 생성된 Event인스턴스의 메서드들을 사용할 수 있다는 것을 다시 한 번 확인하자.

메서드	설명
ActionEvent	
String getActionCommand()	이벤트의 ActionCommand를 반환한다.
int getModifiers()	이벤트가 발생 때 누르고 있던 특수키를 반환한다.
long getWhen()	이벤트가 발생한 시간을 long값으로 반환한다.
AdjustmentEvent	
Adjustable getAdjustable()	이벤트 소스를 반환한다.
int getAdjustmentType()	값을 변경시킨 방법을 알려준다.
int getValue()	이벤트 발생 시의 (스크롤바의) 값을 반환한다.
boolean getValueIsAdjusting()	값이 변경되고 있는 중인지를 알려준다.
ComponentEvent	
Component getComponent()	이벤트 소스를 반환한다.
ContainerEvent	
Component getChild()	컨테이너에 추가/삭제된 컴포넌트를 반환한다.
Container getContainer()	이벤트 소스를 반환한다.
FocusEvent	
Component getOppositeComponent()	포커스를 잃은 컴포넌트를 반환한다.
ItemEvent	
Object getItem()	이벤트가 발생한 item을 반환한다.
ItemSelectable getItemSelectable()	이벤트 소스를 반환한다.
int getStateChange()	Item의 상태(선택/선택해제상태)를 반환한다.
KeyEvent	
char getKeyChar()	눌러진 키를 반환한다.
int getKeyCode()	눌러진 키의 코드를 반환한다.
String getKeyModifiersText(int modifiers)	눌러진 특수키(Alt, Ctrl) 또는 특수키 조합을 설명하는 문자열을 반환한다.
String getKeyText(int keyCode)	눌러진 키를 HOME, F1과 같이 키를 설명하는 문자열로 반환한다.
MouseEvent	
int getClickCount()	마우스를 클릭한 회수를 반환한다.
Point getPoint()	이벤트가 일어난 위치를 반환한다.
int getX()	이벤트가 일어난 위치의 x좌표를 반환한다.
int getY()	이벤트가 일어난 위치의 y좌표를 반환한다.
MouseWheelEvent	
int getScrollAmount()	스크롤되어야하는 양(amount)을 반환한다.
int getScrollType()	어떤 스크롤타입에 의해 이벤트가 발생했는지 알려준다.
int getUnitsToScroll()	단위당 스크롤해야하는 양을 반환한다.
int getWheelRotation()	마우스 휠이 이동한 방향을 알려준다.
WindowEvent	
int getNewState()	윈도우의 현재 상태를 반환한다.

int getOldState()	윈도우의 이전 상태를 반환한다.
Window getOppositeWindow()	이벤트 발생 전에 포커스를 가졌던 윈도우를 반환한다.
Window getWindow()	이벤트 소스를 반환한다.
모든 이벤트 공통	
Object getSource()	이벤트 소스를 반환한다.(EvnetObject에 정의)
String paramString()	이벤트에 관련된 정보를 문자열로 반환한다.(AWTEvent에 정의)

[표13-26] 각 Event의 주요 메서드

[참고] EventObject는 java.util패키지에, AWTEvent는 java.awt패키지에 포함되어 있다.

다음은 이벤트 소스(컴포넌트)에 Listener를 추가 또는 제거하는 메서드를 정리한 것이다. 이 메서드들은 Listener의 이름에 add 또는 remove를 붙여서 만들어졌다.

표의 맨 오른쪽의 이벤트 소스는 Listener를 추가할 수 있는 컴포넌트인데 Component는 Component자손이 대상에 포함되며, Window는 그 자손인 Frame, Panel 등이 포함됨을 의미한다.

Listener	Listener를 추가 / 제거 때 사용하는 메서드	이벤트 소스
ActionListener	void addActionListener(ActionListener l) void removeActionListener(ActionListener l)	Button,List, MenuItem, TextField
AdjustmentListener	void addAdjustmentListener(AdjustmentListener l) void removeAdjustmentListener(AdjustmentListener l)	Scrollbar
ComponentListener	void addComponentListener(ComponentListener l) void removeComponentListener(ComponentListener l)	Component
ContainerListener	void addContainerListener(ContainerListener l) void removeContainerListener(ContainerListener l)	Container
FocusListener	void addFocusListener(FocusListener l) void removeFocusListener(FocusListener l)	Component
ItemListener	void addItemListener(ItemListener l) void removeItemListener(ItemListener l)	Checkbox, CheckboxMenuItem, List, Choice
KeyListener	void addKeyListener(KeyListener l) void removeKeyListener(KeyListener l)	Component
MouseListener	void addMouseListener(MouseListener l) void removeMouseListener(MouseListener l)	Component
MouseMotionListener	void addMouseMotionListener(MouseMotionListener l) void removeMouseMotionListener(MouseMotionListener l)	Component
TextListener	void addTextListener(TextListener l) void removeTextListener(TextListener l)	TextField, TextArea
WindowListener	void addWindowListener(WindowListener l) void removeWindowListener(WindowListener l)	Window
WindowFocusListener	void addWindowFocusListener(WindowFocusListener l) void removeWindowFocusListener(WindowFocusListener l)	Window
WindowStateListener	void addWindowStateListener(WindowStateListener l) void removeWindowStateListener(WindowStateListener l)	Window

[표13-27] 이벤트 리스너를 컴포넌트에 추가 또는 제거할 때 사용하는 메서드

6.4 Adapter클래스

이벤트 핸들러를 작성할 때, 이벤트 처리에 필요한 메서드의 개수와는 관계없이 해당 이벤트 리스너(인터페이스)에 정의된 모든 추상메서드를 구현해야 한다는 것은 좀 불편하다.
이러한 불편한 점을 없애기 위해서 고안된 것이 Adapter클래스이며, 이벤트 리스너를 직접 구현하는 대신에 Adapter클래스를 상속받아서 원하는 메서드만 작성(오버라이딩)하면 된다.
아래의 코드는 KeyListener와 KeyAdapter의 경우를 예로 든 것인데, 왼 쪽의 코드에서는 이벤트 핸들러인 KeyHandler가 이벤트 리스너 KeyListener를 구현했기 때문에 KeyListener에 정의된 세 개의 추상메서드를 모두 구현해야 했지만, 오른 쪽 코드에서는 KeyListener를 구현하는 대신 KeyAdapter를 상속받았기 때문에 원하는 메서드만 작성하였다.

```
class KeyHandler implements KeyListener
{
   public void keyPressed(KeyEvent e) {
      System.out.println(e.getKeyChar());
   }
   public void keyReleased(KeyEvent e) {}
   public void keyTyped(KeyEvent e) {}
}
```

```
class KeyHandler extends KeyAdapter {
  public void keyPressed(KeyEvent e) {
    System.out.println(e.getKeyChar());
  }
}
```

이쯤에서 이미 눈치 챘겠지만 Adapter클래스는 그저 이벤트 리스너에 정의된 메서드를 아무 내용도 없이 구현해 놓았을 뿐이다.

```
interface KeyListener {
    public void keyPressed(KeyEvent e);
    public void keyReleased(KeyEvent e);
    public void keyTyped(KeyEvent e);
}
class KeyAdapter implements KeyListener {
    public void keyPressed(KeyEvent e) {}
    public void keyReleased(KeyEvent e) {}
    public void keyTyped(KeyEvent e) {}
}
```

이벤트 핸들러를 작성할 때 이벤트 리스너를 구현하는 것과 Adapter클래스를 상속받는 것은 코드를 작성하기 편리하다는 것 외에는 차이가 없다.

Adapter클래스	이벤트 리스너(interface)
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener

[표13-28] Adapter클래스가 구현한 이벤트 리스너

[예제13-29]/ch13/TextFieldTest2.java

```
import java.awt.*;
import java.awt.event.*;

class TextFieldTest2 extends Frame {
    Label lid;
    Label lpwd;
    TextField tfId;
    TextField tfPwd;
    Button ok;

    TextFieldTest2(String title) {
          super(title); // Frame(String title)을 호출한다.

          lid = new Label("ID :", Label.RIGHT); // Label의 text정렬을 오른쪽으로.
          lpwd = new Label("Password :", Label.RIGHT);

          // 약 10개의 글자를 입력할 수 있는 TextField 생성.
          tfId = new TextField(10);
          tfPwd = new TextField(10);
          tfPwd.setEchoChar('*'); // 입력한 값 대신 '*'이 보이게 한다.

          ok = new Button("OK");
          // OK버튼과 TextField에 이벤트처리를 위한 Listener를 추가해준다.
          tfId.addActionListener(new EventHandler());
          tfPwd.addActionListener(new EventHandler());
          ok.addActionListener(new EventHandler());

          setLayout(new FlowLayout()); // LayoutManager를 FlowLayout으로
          add(lid);  // 생성한 Component들을 Frame에 포함시킨다.
          add(tfId);
          add(lpwd);
          add(tfPwd);
          add(ok);
          setSize(450, 65);
          setVisible(true); // Frame이 화면에 보이게 한다.
    }

    public static void main(String args[]) {
          TextFieldTest2 f = new TextFieldTest2("Login");
    }

    class EventHandler implements ActionListener {
          public void actionPerformed(ActionEvent e){
                String id = tfId.getText(); // tfId에 입력되어있는 text를 얻어온다.
                String password = tfPwd.getText();
                if (!id.equals("javachobo")){
                      System.out.println("입력하신 id가 유효하지 않습니다. "
                                                          + " 다시 입력해 주세요.");
                      // id를 다시 입력할 수 있도록, focus를 tfId로 옮긴다.
                      tfId.requestFocus();
                      tfId.selectAll();   // tfId에 입력된 text가 선택되게 한다.
                } else if (!password.equals("asdf")) {
                      System.out.println("입력하신 비밀번호가 틀렸습니다. "
                                                      + "다시 입력해 주시기 바랍니다.");
```

```
                    // id를 다시 입력할 수 있도록 focus를 tfId로 옮긴다.
                    tfPwd.requestFocus();
                    tfPwd.selectAll();
                } else {
                    System.out.println( id + "님, 성공적으로 로그인 되었습니다.");
                }
            }
        } // class EventHandler
    }
```

[실행결과]

```
javachobo님, 성공적으로 로그인 되었습니다.
```

위의 예제는 전에 다루었던 예제13-9 TextFieldTest.java에 이벤트 처리를 추가한 것이다. 그 외에도 구조적인 부분에서 달라진 부분이 있는데, Frame을 상속받는 클래스를 구현하고 Frame에 포함되는 컴포넌트들을 인스턴스 멤버로 선언하고, Frame의 생성자에서 컴포넌트를 생성 및 설정하도록 코드를 작성하였다. 기존의 코드 보다 더 객체지향적으로 잘 정리되어 있다는 것을 느낄 수 있을 것이다.

예제를 실행한 다음 ID와 Password를 입력하고 OK 버튼을 누르면, ID와 Password가 맞는지 확인하고, 틀리면 수정하라는 메시지를 사용자에게 보낸다.

```
tfId.addActionListener(new EventHandler());
tfPwd.addActionListener(new EventHandler());
ok.addActionListener(new EventHandler());
```

OK버튼만 아니라 ID와 Password를 입력받는 TextField인 tfId,tfPwd에도 EventHandler 인스턴스를 이벤트 리스너로 등록했다.
이렇게 함으로써 버튼을 클릭할 때만 아니라 TextField에서 Enter키를 눌렀을 때도 ActionEvent가 처리되기 때문에 입력 후에 손을 키보드에서 마우스로 옮기지 않아도 되므로 편리하다.
이제 ActionEvent와 같은 고수준의 이벤트가 왜 필요한지 충분히 이해가 되었을 것이다.

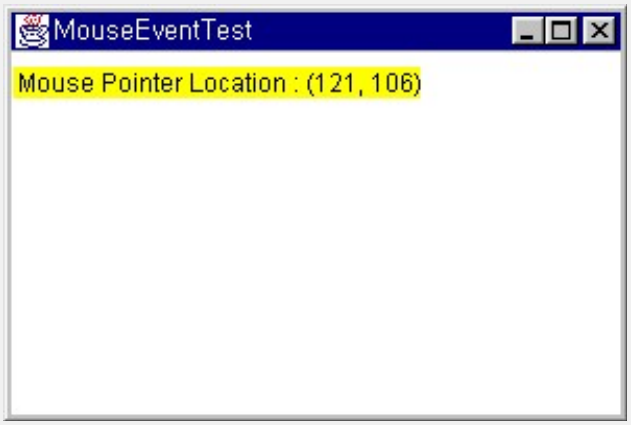
requestFocus() - 원하는 Component가 focus를 갖도록 할 수 있다.
selectAll() - TextField 또는 TextArea에 입력된 text 전체를 선택된 상태로 만든다.
setEchoChar(char c) - 실제로 입력된 값을 감추기 위해 화면에 보여질 문자를 지정한다.

이 외에도 유용한 메서드가 많이 있으니, Java API문서에서 TextField와 TextArea 그리고 그 조상인 TextComponent에 대해서 찾아보자.
[참고] 원하는 메서드를 TextField와 TextArea에서만 찾아보는 경우가 많은데 반드시 TextComponent도 찾아봐야 한다.

[예제13-30]/ch13/**MouseEventTest.java**

```
import java.awt.*;
import java.awt.event.*;

class MouseEventTest extends Frame {
    Label location;

    MouseEventTest(String title) {
          super(title); // Frame(String title)을 호출한다.
          location = new Label("Mouse Pointer Location : ");
          location.setSize(195, 15);
          location.setLocation(5,30);
          location.setBackground(Color.yellow);  // Label의 배경색을 노란색으로 한다.
          add(location);

          // EventHandler의 인스턴스를 Frame의 Listener로 등록한다.
          addMouseMotionListener(new EventHandler());

          setSize(300, 200);
          setLayout(null);
          setVisible(true);
    }

    public static void main(String args[]) {
          MouseEventTest mainWin = new MouseEventTest("MouseEventTest");
    }    // main메서드의 끝

    class EventHandler implements MouseMotionListener {
          public void mouseDragged(MouseEvent e) {}
          public void mouseMoved(MouseEvent e) {
                location.setText("Mouse Pointer Location : (" + e.getX() + ", "
                                                                  + e.getY()+ ")");
          }
    }     // EventHandler클래스의 끝
}
```

[실행결과]



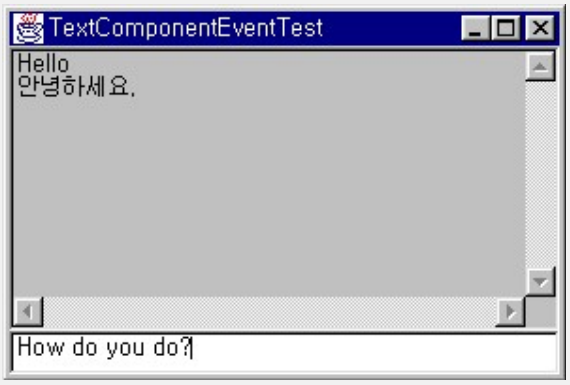
예제를 실행시킨 후, 마우스의 포인터로 Frame위를 움직이면, 화면에 포인터의 좌표가 나타나게 된다. 포인터의 좌표를 보여주기 위해서 Label을 사용했다.

Frame의 위에서 마우스의 포인터를 움직일 때, MouseEvent가 발생하며 MouseEvent의 mouseMoved메서드가 호출되며 이벤트 소스는 Frame이다. MouseEvent에는 getX()와 getY()가 있는데, 이 메서드들을 이용하면, 마우스 포인터의 좌표를 알 수 있다.

[예제13-31]/ch13/TextComponentEventTest.java

```
import java.awt.*;
import java.awt.event.*;

class TextComponentEventTest extends Frame {
    TextField tf;
    TextArea ta;

    TextComponentEventTest(String title) {
          super(title); // Frame(String title)을 호출한다.

          tf = new TextField();
          ta = new TextArea();
          add(ta, "Center");
          add(tf, "South");

          tf.addActionListener(new ActionListener(){
                public void actionPerformed(ActionEvent e) {
                     // TextField에서 Enter를 치면, tf에 입력된 text를 TextArea에 추가한다.
                      ta.append(tf.getText() + "\n");
                      tf.setText("");    // tf의 text를 지운다.
                      tf.requestFocus();
                }
          });

          ta.setEditable(false);     // TextArea의 text를 편집하지 못하게 한다.
          setSize(300, 200);
          setVisible(true);
          tf.requestFocus();         // focus가 TextField에 위치하도록 한다.
    }

    public static void main(String args[]) {
         TextComponentEventTest mainWin =
                        new TextComponentEventTest("TextComponentEventTest");
    }    // main메서드의 끝
}
```

[실행결과]



위의 예제는 TextField와 TextArea를 이용해서 채팅창과 같은 화면을 만든 것이다. TextField에 글자를 입력하고 Enter키를 누르면, TextArea에 입력한 글자들이 나타난다.

TextField에서 Enter키를 눌렀을 때, ActionEvent의 actionPerformed메서드가 호출된다는 것을 기억하고 있을 것이다. actionPerformed메서드가 호출되는 4가지 경우를 표 13-25에서 다시 한 번 확인해 보기 바란다.

TextArea의 내용을 직접 편집하지 못하도록 하기 위해서, setEditable(false)를 사용했다. 그래서 실행 시 TextArea의 text를 편집할 수 없으며, 바탕색이 회색으로 보이는 것이다.

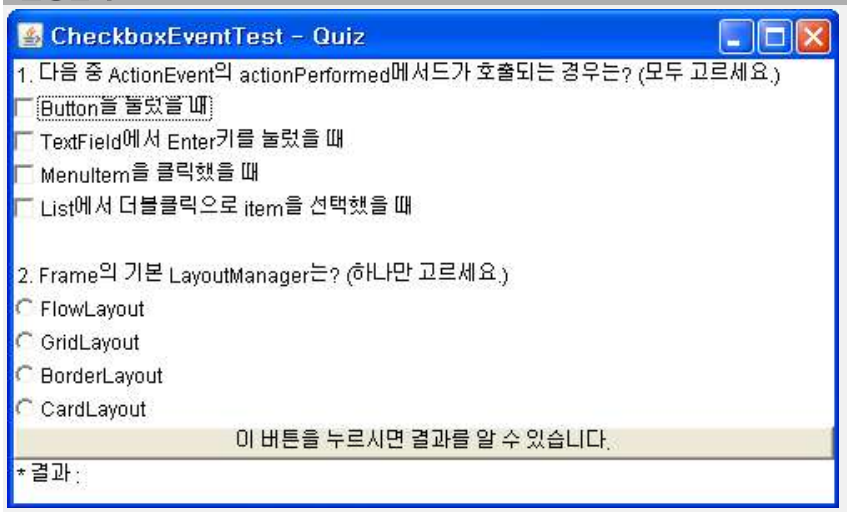
[참고] setCaretPosition()을 이용하면 TextArea내에서 커서의 위치를 변경시킬 수 있다.

[예제13-32] /ch13/**CheckboxEventTest.java**

```
import java.awt.*;
import java.awt.event.*;

class CheckboxEventTest extends Frame {
    Label q1;
    Label q2;
    Label score;
    Checkbox q1cb1, q1cb2, q1cb3, q1cb4;
    Checkbox q2cb1, q2cb2, q2cb3, q2cb4;
    CheckboxGroup group;
    Button end;

    CheckboxEventTest(String title) {
          super(title); // Frame(String title)을 호출한다.
          setSize(500, 300);
          setLayout(new GridLayout(13,1));

          q1 = new Label("1. 다음 중 ActionEvent의 actionPerformed메서드가 "
                                                  + "호출되는 경우는? (모두 고르세요.)");
          q1cb1 = new Checkbox("Button을 눌렀을 때");
          q1cb2 = new Checkbox("TextField에서 Enter키를 눌렀을 때");
          q1cb3 = new Checkbox("MenuItem을 클릭했을 때");
          q1cb4 = new Checkbox("List에서 더블클릭으로 item을 선택했을 때");

          q2 = new Label("2. Frame의 기본 LayoutManager는? (하나만 고르세요.)");
          group = new CheckboxGroup();
          q2cb1 = new Checkbox("FlowLayout", group, false);
          q2cb2 = new Checkbox("GridLayout", group, false);
          q2cb3 = new Checkbox("BorderLayout", group, false);
          q2cb4 = new Checkbox("CardLayout", group, false);

          score = new Label("* 결과 :");
          end = new Button("이 버튼을 누르시면 결과를 알 수 있습니다.");
          end.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                      float totalScore=0;
                      if(q1cb1.getState()) totalScore += 12.5;
                      if(q1cb2.getState()) totalScore += 12.5;
                      if(q1cb3.getState()) totalScore += 12.5;
                      if(q1cb4.getState()) totalScore += 12.5;
                      if(q2cb3.getState()) totalScore += 50;
                      score.setText("* 결과 : 당신의 점수는 " + totalScore
                                                                  + "점 입니다.");
```

```
                }
            });

            add(q1);
            add(q1cb1);     add(q1cb2);     add(q1cb3);     add(q1cb4);
            add(new Label("")); // 문제들 사이에 빈 줄을 하나 삽입하기 위해 사용
            add(q2);
            add(q2cb1);     add(q2cb2);     add(q2cb3);     add(q2cb4);
            add(end);
            add(score);

            setVisible(true);
        }

        public static void main(String args[]) {
            CheckboxEventTest mainWin =
                                new CheckboxEventTest("CheckboxEventTest - Quiz");
        }     // main메서드의 끝
    }
```

[실행결과]



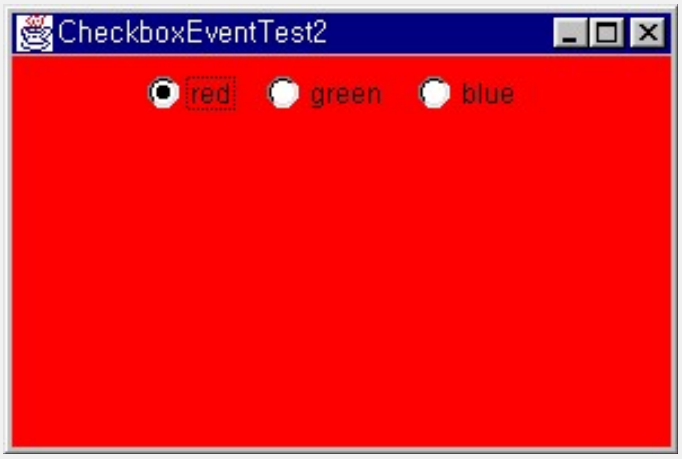
Checkbox를 이용해서 객관식 문제를 만드는 예제이다. Checkbox의 getState()를 이용해서 Checkbox의 상태(선택되었는지 선택되지 않았는지)를 알 수 있다.

[예제13-33]/ch13/CheckboxEventTest2.java

```
import java.awt.*;
import java.awt.event.*;

class CheckboxEventTest2 extends Frame {
    CheckboxGroup group;
    Checkbox cb1;
    Checkbox cb2;
    Checkbox cb3;

    CheckboxEventTest2(String title) {
        super(title);
```

```
        group = new CheckboxGroup();
        cb1 = new Checkbox("red", group, true);
        cb2 = new Checkbox("green", group, false);
        cb3 = new Checkbox("blue", group, false);

        cb1.addItemListener(new EventHandler());
        cb2.addItemListener(new EventHandler());
        cb3.addItemListener(new EventHandler());

        setLayout(new FlowLayout());
        add(cb1);
        add(cb2);
        add(cb3);
        setBackground(Color.red);
        setSize(300, 200);
        setVisible(true);
    }

    public static void main(String args[]) {
        CheckboxEventTest2 mainWin =
            new CheckboxEventTest2("CheckboxEventTest2");
    }     // main메서드의 끝

    class EventHandler implements ItemListener {
        public void itemStateChanged(ItemEvent e) {
            Checkbox cb = (Checkbox)e.getSource();
            String color = cb.getLabel();
            if(color.equals("red")) {
                setBackground(Color.red);
            } else if (color.equals("green")) {
                setBackground(Color.green);
            } else {
                setBackground(Color.blue);
            }
        }
    } // class EventHandler
}
```

[실행결과]



예제를 실행시킨 후 Checkbox(radio button)중의 하나를 클릭하면, Frame의 배경색(background color)이 변경 될 것이다. 이 때, 이벤트 소스는 클릭한 Checkbox이고, ItemEvent가 발생하며, itemStateChanged메서드가 호출되어진다. itemStateChanged메서드 내에서 getSource()를 호출하면 이벤트 소스를 얻을 수 있다. getSource()는 모든 이벤트 클래스의 조상인 EventObject클래스에 정의되어 있는 것으로 모든 이벤트 클래스에서 사용할 수 있는 유용한 메서드이다.

```
public void itemStateChanged(ItemEvent e) {
      Checkbox cb = (Checkbox)e.getSource();
      ...
}
```

[예제13-34]/ch13/**CardLayoutEventTest.java**

```
import java.awt.*;
import java.awt.event.*;

class CardLayoutEventTest extends Frame {
    Button first, prev, next, last;
    Panel buttons;
    Panel slide;
    Panel card1, card2, card3, card4, card5; //  slide에 포함될 Panel들
    CardLayout card;

    CardLayoutEventTest(String title) {
          super(title); // 조상인 Frame의 생성자 Frame(String title)을 호출

          // 화면을 담을 Panel을 담는다.
          slide = new Panel();
          card = new CardLayout();
          slide.setLayout(card);

          // 버튼을 담을 Panel을 만든다.
          buttons = new Panel();
          buttons.setLayout(new FlowLayout());

          first = new Button("<<");
          prev = new Button("<");
          next = new Button(">");
          last = new Button(">>");
          buttons.add(first);
          buttons.add(prev);
          buttons.add(next);
          buttons.add(last);

          // 버튼에 이벤트 리스너를 추가한다.
          first.addActionListener(new ActionListener(){
                public void actionPerformed(ActionEvent e) {
                      // CardLayout의 첫 번째 slide(Panel)이 보이도록 한다.
                      card.first(slide);
                }
          });
          prev.addActionListener(new ActionListener(){
                public void actionPerformed(ActionEvent e) {
                        // 현재 slide의 이전 slide(Panel)이 보이도록 한다.
```

```
                    card.previous(slide);
                }
            });
            next.addActionListener(new ActionListener(){
                public void actionPerformed(ActionEvent e) {
                     // 현재 slide의 다음 slide(Panel)이 보이도록 한다.
                    card.next(slide);
                }
            });
            last.addActionListener(new ActionListener(){
                public void actionPerformed(ActionEvent e) {
                    // CardLayout의 마지막 slide(Panel)이 보이도록 한다.
                    card.last(slide);
                }
            });

            card1= new Panel();
            card1.setBackground(Color.gray);
            card1.add(new Label("첫 번째 페이지입니다."));
            card2= new Panel();
            card2.add(new Label("두 번째 페이지입니다."));
            card2.setBackground(Color.orange);
            card3= new Panel();
            card3.add(new Label("세 번째 페이지입니다."));
            card3.setBackground(Color.blue);
            card4= new Panel();
            card4.add(new Label("네 번째 페이지입니다."));
            card4.setBackground(Color.cyan);
            card5= new Panel();
            card5.add(new Label("다섯 번째 페이지입니다."));
            card5.setBackground(Color.pink);

             // slide(Panel)에 card1(Panel)을 "1"이란 이름으로 추가한다.
            slide.add(card1, "1");
            slide.add(card2, "2");
            slide.add(card3, "3");
            slide.add(card4, "4");
            slide.add(card5, "5");

            add("South", buttons);
            add("Center", slide);

            setSize(200, 200);
            setLocation(200, 200);
            setVisible(true);

            card.show(slide,"1"); // 첫 번째 슬라이드(card1)가 나타나게 한다.
        }

        public static void main(String args[]) {
            CardLayoutEventTest mainWin =
                                    new CardLayoutEventTest("CardLayoutTest");
        }
    }
```

[실행결과]

CardLayout을 이용해서 5개의 Panel을 슬라이드 형식으로 화면에 보여주는 예제이다. 버튼을 통해 각 화면(Panel)으로 이동할 수 있다.

Frame의 Center에 CardLayout이 레이아웃 매니저인 slide를 넣고, South에는 버튼이 담긴 Panel을 넣었다.

또다시 slide에 5개의 Panel이 추가되고 버튼에 의해 5개의 Panel중 어떤 것이 보일 것인지가 달라진다.

[예제13-35]/ch13/**TextEditor**.java

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;

class TextEditor extends Frame {
    String fileName;
    TextArea  content;
    MenuBar   mb;
    Menu      mFile;
    MenuItem  miNew, miOpen, miSaveAs, miExit;

    TextEditor(String title) {
          super(title);
          content = new TextArea();
          add(content);

          mb       = new MenuBar();
          mFile    = new Menu("File");
          miNew     = new MenuItem("New");
          miOpen   = new MenuItem("Open");
          miSaveAs = new MenuItem("Save As...");
          miExit   = new MenuItem("Exit");

          mFile.add(miNew);
          mFile.add(miOpen);
          mFile.add(miSaveAs);
          mFile.addSeparator();   // 메뉴 분리선(separator)을 넣는다.
          mFile.add(miExit);

          mb.add(mFile);          // MenuBar에 Menu를 추가한다.
          setMenuBar(mb);         // Frame에 MenuBar를 포함시킨다.
```

```
        // 메뉴에 이벤트 핸들러를 등록한다.
        MyHandler handler = new MyHandler();
        miNew.addActionListener(handler);
        miOpen.addActionListener(handler);
        miSaveAs.addActionListener(handler);
        miExit.addActionListener(handler);

        setSize(300, 200);
        setVisible(true);
    }

    // 선택된 파일의 내용을 읽어서 TextArea에 보여주는 메서드
    void fileOpen(String fileName) {
        FileReader fr;
        BufferedReader br;
        StringWriter sw;

        try {
            fr = new FileReader(fileName);
            br= new BufferedReader(fr);
            sw = new StringWriter();

            int ch = 0;
            while ((ch=br.read())!=-1) {
                sw.write(ch);
            }

            br.close();
            content.setText(sw.toString());
        } catch(IOException e) {
            e.printStackTrace();
        }
    } // fileOpen메서드의 끝

    // TextArea의 내용을 지정된 파일에 저장하는 메서드
    void saveAs(String fileName) {
        FileWriter fw;
        BufferedWriter bw;

        try {
            fw = new FileWriter(fileName);
            bw= new BufferedWriter(fw);
            bw.write(content.getText());     // TextArea의 내용을 파일에 저장한다.
            bw.close();
        } catch (IOException ie) {
            ie.printStackTrace();
        } // try
    } // saveAs메서드의 끝

    public static void main(String args[]) {
        TextEditor mainWin = new TextEditor("Text Editor");
    }     // main메서드의 끝


    // 메뉴를 클릭했을 때 메뉴별 처리코드
    class MyHandler implements ActionListener {
        public void actionPerformed(ActionEvent e) {
```

```
                String command = e.getActionCommand();

                if(command.equals("New")) {
                      content.setText("");
                } else if(command.equals("Open")) {
                      FileDialog fileOpen =
                                      new FileDialog(TextEditor.this, "파일열기");
                      fileOpen.setVisible(true);
                      fileName = fileOpen.getDirectory() + fileOpen.getFile();
                      System.out.println(fileName);
                      // 선택된 파일의 내용을 TextArea에 보여준다.
                      fileOpen(fileName);
                } else if(command.equals("Save As...")) {
                      FileDialog fileSave =
                     new FileDialog(TextEditor.this,"파일저장",FileDialog.SAVE);
                      fileSave.setVisible(true);
                      fileName = fileSave.getDirectory() + fileSave.getFile();
                      System.out.println(fileName);
                      // 현재 TextArea의 내용을 선택된 파일에 저장한다.
                      saveAs(fileName);
                } else if(command.equals("Exit")) {
                      System.exit(0);      // 프로그램을 종료시킨다.
                }
          }
    } // class MyHandler
}  // TextEditor클래스의 끝
```

[실행결과]

이 예제는 FileDialog와 TextArea를 이용해서 파일 편집기를 만든 것인데, 예제가 복잡해지는 것을 피하기 위해 기능을 최소화했다. FileDialog의 사용법과 메뉴의 이벤트처리 등을 눈여겨보자. 파일을 읽고, 저장하는 부분은 후에 배우게 될 것이므로 지금은 이 부분을 제외한 나머지 부분만 이해해도 좋다.

[참고] 이 예제에서 보완되어야 할 부분을 생각해보고, 기능을 하나둘 씩 추가해가는 것도 실력향상에 많은 도움이 될 것이다.

[예제13-36]/ch13/PaintFrameTest.java

```
import java.awt.*;
import java.awt.event.*;

class PaintFrameTest {
    public static void main(String args[]) {
          PaintFrame f = new PaintFrame("PaintFrame");

          Graphics g = f.getGraphics();
          g.setColor(Color.red);

          for(int i=0; true ; i+=f.speed) {
                try {
                      Thread.sleep(100);   // 0.1초 동안 실행을 지연시킨다.
                } catch (InterruptedException e) {}

                g.clearRect(0,0,300,300);
                g.drawString("Hello", i, 150);
                i = (i<300)? i : 0;
          }
    } // main메서드의 끝
}

class PaintFrame extends Frame {
    int speed = 1;
    Scrollbar sb = new Scrollbar(Scrollbar.HORIZONTAL, 0, 5, 0, 50);
    Label lSpeed = new Label("Speed : 1");

    PaintFrame(String title) {
          super(title);
          lSpeed.setBackground(Color.yellow);
          lSpeed.setSize(65, 15);
          lSpeed.setLocation(10, 30);
          sb.setSize(260, 20);
          sb.setLocation(20, 250);
          add(sb);
          add(lSpeed);
          sb.addAdjustmentListener(new MyHandler());
          addWindowListener(new MyHandler());
          setSize(300, 300);
          setLayout(null);
          setVisible(true);
          setResizable(false); // Frame의 크기를 변경할 수 없도록 한다.
    }

    class MyHandler extends WindowAdapter implements AdjustmentListener {
          public void adjustmentValueChanged(AdjustmentEvent e){
                speed = sb.getValue();     // Scrollbar의 현재 값을 얻는다.
                lSpeed.setText("Speed : " + speed);
          }

          public void windowClosing(WindowEvent we) {
                System.exit(0);
          }
    }
} // class PaintFrame
```

[실행결과]

이 예제를 시작하면 화면 가운데 왼쪽에서 오른쪽으로 글자(Hello)가 이동한다. 아래쪽의 Scrollbar를 움직이면, 이 글자가 움직이는 속도에 변화를 줄 수 있다. 마우스를 이용하지 않고 화살표키를 눌러서 Scrollbar를 움직일 수도 있다.

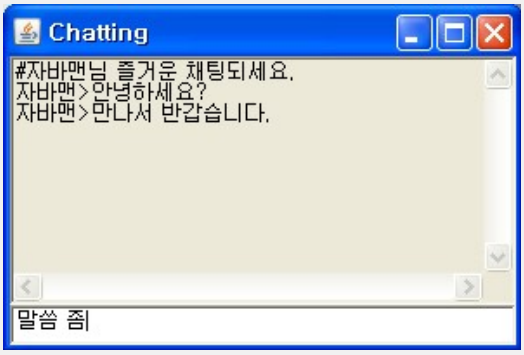
[예제13-37]/ch13/ChatWin.java

```
import java.awt.*;
import java.awt.event.*;

class ChatWin extends Frame {
    String nickname = "";

    TextArea ta = new TextArea();

    Panel p = new Panel();
    TextField tf = new TextField();

    ChatWin() {
          this("guest");
    }

    ChatWin(String nickname) {
          super("Chatting");
          this.nickname = nickname;

          setBounds(100, 100, 300, 200);

          p.setLayout(new BorderLayout());
          p.add(tf, "Center");

          add(ta, "Center");
          add(p, "South");

          addWindowListener(new WindowAdapter(){
                public void windowClosing(WindowEvent e) {
                      System.exit(0);
                }
          });
```

```
            EventHandler handler = new EventHandler();
            ta.addFocusListener(handler);
            tf.addFocusListener(handler);
            tf.addActionListener(handler);

            ta.setText("#" + nickname + "님 즐거운 채팅되세요.");
            ta.setEditable(false);

            setVisible(true);
            tf.requestFocus();
      }
      public static void main(String[] args) {
            if(args.length != 1) {
                  System.out.println("USAGE : java ChatWin NICKNAME");
                  System.exit(0);
            }

            new ChatWin(args[0]);
      } // main

      class EventHandler extends FocusAdapter implements ActionListener {
            public void actionPerformed(ActionEvent ae) {
                  String msg = tf.getText();
                  if("".equals(msg)) return;

                  ta.append("\r\n" + nickname +">"+ msg);
                  tf.setText("");
            }

            public void focusGained(FocusEvent e) {
                  tf.requestFocus();
            }
      } // class EventHandler
  } // class
```

[실행결과]

```
C:\jdk1.5\work\ch13>java ChatWin 자바맨
```



이 예제는 채팅프로그램에 사용되는 채팅창을 간단하게 만들어본 것이다. TextField에서 Enter키를 누르면 ActionEvent가 발생하고 입력한 내용이 채팅창인 TextArea에 나타난다.

```
class FocusHandler extends FocusAdapter {
    public void focusGained(FocusEvent e) {
        tf.requestFocus(); // TextField가 포커스를 잃으면 다시 포커스를 얻게 한다.
    }
}
```

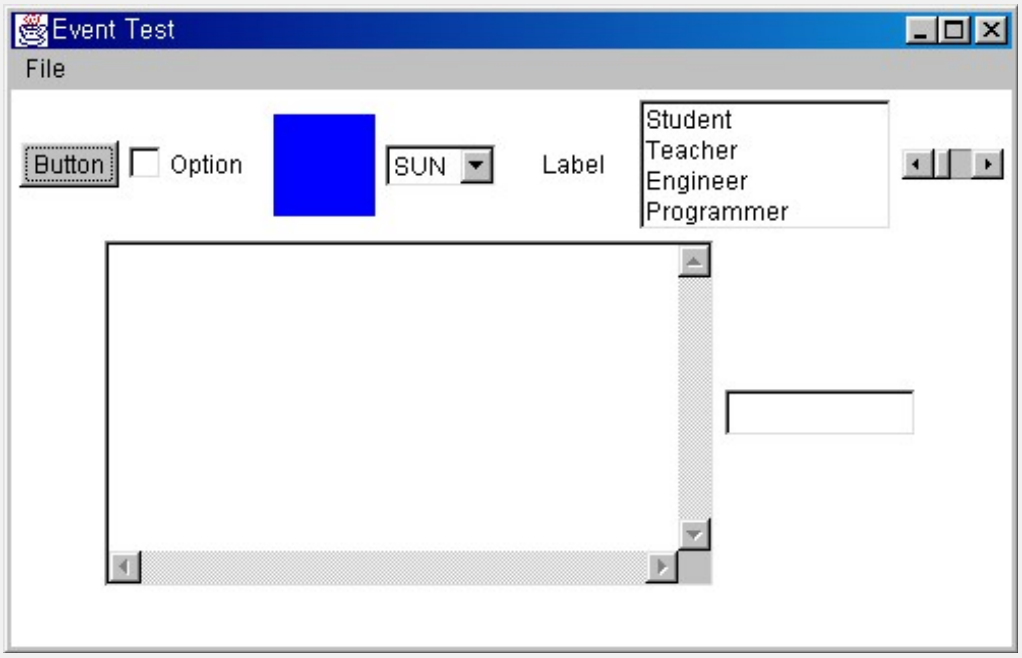
그 외에 FocusEvent에 대한 이벤트 처리도 하였는데 그 이유는 채팅창의 다른 컴포넌트를 클릭해서 TextField가 포커스를 잃더라도 다시 포커스를 TextField로 이동시켜서 사용자가 입력을 편하게 하도록 하기 위해서이다.

아래의 예제는 Frame에 여러 종류의 컴포넌트를 추가하고, 가능한 모든 종류의 Listener를 구현한 하나의 이벤트 처리 클래스를 각 컴포넌트의 Listener로 등록한다.
사용자의 동작에 의해 발생한 이벤트의 메서드는, 자신의 이름과 이벤트 소스와 관련정보를 화면(System.out)에 출력한다.

[예제13-38]/ch13/**EventTest.java**

```
// 소스는 생략합니다. java_jungsuk_src.zip에서 찾을 수 있습니다.
// java_jungsuk_src.zip는 http://www.javachobo.com 에서 다운로드 받으실 수 있습니다.
```

[실행결과]




```
C:\jdk1.5\work\ch13>java EventTest
[1] componentResized -
java.awt.Frame[frame0,200,200,500x319,hidden,layout=java.awt.FlowLayout,resizable,tit
le=Event Test]
[2] componentResized -
java.awt.Frame[frame0,200,200,500x319,hidden,layout=java.awt.FlowLayout,resizable,tit
le=Event Test]
[3] componentResized - java.awt.Button[button0,8,67,49x23,label=Button]
[4] componentResized -
java.awt.Checkbox[checkbox0,62,67,66x23,label=Option,state=false]
[5] componentResized - java.awt.Panel[panel0,248,74,10x10,layout=java.awt.FlowLayout]
```

```
[6] componentResized - java.awt.Label[label0,263,67,45x23,align=left,text=Label]
[7] componentResized - java.awt.List[list0,313,47,124x64,selected=null]
...
```

출력된 결과를 통해서 어떤 동작을 했을 때 어떤 이벤트의 메서드가 가 발생하는지 자세히 알 수 있을 것이다.

하나의 동작에도 여러분들이 생각했던 것보다 많은 메서드들이 호출된다는 것을 알 수 있을 것이다.

예를 들면, TestArea에 글자하나를 입력했을 때, keyPressed(), keyReleased(), keyTyped() 모두 3개의 메서드가 순서대로 호출된다.

여기서 중요한 것은 호출되는 메서드들의 종류와 순서, 그리고 이벤트 소스이다. 각 Component에 여러 가지 동작을 함으로써 다양한 이벤트가 발생하도록 하고 그 결과를 확인하도록 한다. 아래에는 여러분들이 직접 해보면 좋을 테스트를 몇 가지 적어보았다. 실행해 보고 그 결과를 확인하라.

[참고] 이벤트는 마우스 또는 키보드를 통한 사용자의 동작뿐만 아니라 프로그램 코드에 의해서도 발생할 수 있다는 사실을 확인하자.

[참고] 마우스를 움직일 때 마다 호출되는 MouseMoved메서드 때문에 이벤트 발생으로 인해 호출되는 메서드를 구분하기 어렵다면, MouseMoved메서드의 내용을 주석처리 하자.

1. 키보드의 키 하나를 계속 누르고 있을 때 어떤 일이 생기는지 확인하라.
2. Ctrl과 Alt같은 특수키를 눌렀을 때와 일반키를 눌렀을 때를 비교하라.
3. TextField에서 Enter키를 눌렀을 때
4. Tab키를 눌러서 focus를 이동했을 때
5. 마우스의 포인터를 Frame에서 Canvas의 영역 안으로 이동했을 때(그리고, 그 반대로)
6. List에 있는 item하나를 선택해서 더블클릭했을 때
7. Choice에서 다른 item을 선택했을 때
8. Frame의 크기를 변경했을 때

아래의 예제는 모든 종류의 Listener를 구현한 클래스이다. 이 예제를 만든 이유는 이벤트 처리를 위한 Event Handler를 작성하는데 도움을 주기 위해서이다. Listener Adapter 클래스를 사용하지 못하는 경우라면, Listener Interface에 정의된 모든 메서드들을 구현해야 하므로 단순 반복적인 코딩을 해야 하고, 오타로 인한 에러가 발생하기 쉽다.

이 예제 파일을 에디터로 열어서 필요한 부분을 복사해서 붙이는 식으로 하면 코딩이 보다 쉽고 정확해질 것이다.

[예제13-39]/ch13/AllEventHandler.java

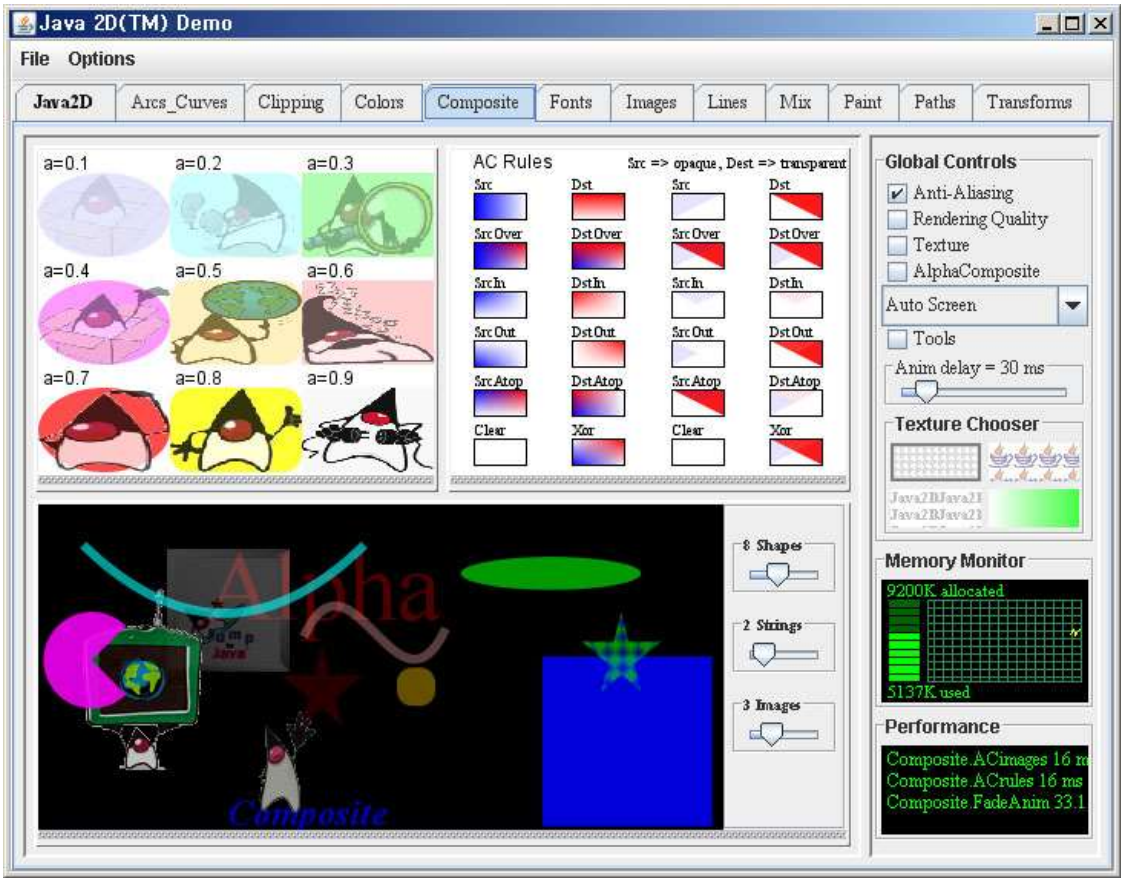
```
import java.awt.event.*;

class AllEventHandler
       implements ActionListener, ComponentListener, MouseMotionListener,
        MouseListener, MouseWheelListener, KeyListener, TextListener,
        FocusListener, ItemListener,  AdjustmentListener, WindowListener,
        WindowFocusListener, WindowStateListener, ContainerListener
{
    // ActionListener의 메서드
    public void actionPerformed(ActionEvent ae) {}

    // ComponentListener의 메서드
    public void componentMoved(ComponentEvent ce) {}
    public void componentHidden(ComponentEvent ce) {}
```

```
    public void componentResized(ComponentEvent ce) {}
    public void componentShown(ComponentEvent ce) {}

    // MouseMotionListener의 메서드
    public void mouseDragged(MouseEvent me) {}
    public void mouseMoved(MouseEvent me) {}

    // MouseListener의 메서드
    public void mousePressed(MouseEvent me) {}
    public void mouseReleased(MouseEvent me) {}
    public void mouseEntered(MouseEvent me) {}
    public void mouseExited(MouseEvent me) {}
    public void mouseClicked(MouseEvent me) {}

    // MouseWheelListener의 메서드
    public void mouseWheelMoved(MouseWheelEvent e) {}

    // KeyListener의 메서드
    public void keyPressed(KeyEvent ke) {}
    public void keyReleased(KeyEvent ke) {}
    public void keyTyped(KeyEvent ke) {}

    // TextListener의 메서드
    public void textValueChanged(TextEvent te) {}

    // FocusListener의 메서드
    public void focusGained(FocusEvent fe) {}
    public void focusLost(FocusEvent fe) {}

    // ItemListener의 메서드
    public void itemStateChanged(ItemEvent ie) {}

    // AdjustmentListener의 메서드
    public void adjustmentValueChanged(AdjustmentEvent ae) {}

    // WindowListener의 메서드
    public void windowClosing(WindowEvent we) {}
    public void windowOpened(WindowEvent we) {}
    public void windowIconified(WindowEvent we) {}
    public void windowDeiconified(WindowEvent we) {}
    public void windowClosed(WindowEvent we) {}
    public void windowActivated(WindowEvent we) {}
    public void windowDeactivated(WindowEvent we) {}

    // WindowFocusListener의 메서드
    public void windowGainedFocus(WindowEvent e) {}
    public void windowLostFocus(WindowEvent e) {}

    // WindowStateListener의 메서드
    public void windowStateChanged(WindowEvent e) {}

    // ContainerListener의 메서드
    public void componentAdded(ContainerEvent ce) {}
    public void componentRemoved(ContainerEvent ce) {}
} // End of EventHandler class
```

7. AWT의 그래픽

이제 AWT를 이용해서 그림을 그리는 방법에 대해서 알아볼 차례이다. 이 단원에서는 AWT에서 선과 도형을 어떻게 그리고 어떤 과정을 통해서 그려지는지에 대해서 배울 것이며, 이 단원을 마치고나면 간단한 그림판 정도는 만들 수 있을 것이다.

여기서는 AWT 그래픽의 기본적인 내용만을 다루기 때문에 자바를 이용해서 이미지처리나 게임과 같이 높은 수준의 그래픽기능을 요구하는 어플리케이션을 작성하는데 부적합하다는 오해를 할지도 모르겠다.
그러나 자바에서는 오래전부터 향상된 그래픽처리를 위한 API인 Java2D를 제공해왔는데. Java2D는 포토샵으로 유명한 Adobe사와 자바를 탄생시킨 썬(SUN)사가 공동 제작한 것으로 이미지 프로세싱이나 렌더링과 같은 고수준의 그래픽처리에 필요한 모든 것을 제공하는 강력한 가능의 API이다. 백문이 불여일견이니 Java2D데모프로그램을 실행해서 Java2D로 어떤 일들을 할 수 있는지 직접 확인해보자.



[그림13-12] Java2Demo.jar의 실행화면

[참고] Java2D데모프로그램은 'JDK설치디렉토리₩demo₩jfc₩Java2D'에 있으며 Java2Demo.html를 웹브라우저로 열던가, 콘솔에서 java -jar Java2Demo.jar와 같이 입력하여Java2Demo.jar를 실행시키면 볼 수 있다.

[참고] Java2D는 JDK API의 일부로 java.awt패키지와 그 하위 패키지에 포함되어 있다.

AWT는 자바초기에 작성되었기 때문에 JDK의 버전이 올라감에 따라 보다 높은 기능의 API가 필요 했다. 그래서 Java2에서부터 JFC(Java Foundation Classes)라는 이름으로 Swing과 Java2D가 추가로 제공되었다. Swing은 AWT보다 향상된 UI(User interface)를 제공하였고, Java2D는 AWT보다 향상된 그래픽 기능을 제공하였다.

그럼에도 불구하고 AWT를 배워야하는 이유는 AWT가 Swing과 Java2D의 뿌리이기 때문이다.

Swing이나 Java2D가 흥미롭고 매력적인 분야이기는 하나, 특히 국내에서는 자바가 주로 웹프로그래밍이나 서버프로그래밍에 사용되기 때문에 Swing이나 Java2D를 자바의 기본내용에 포함시키는 것 보다는 이 들의 뿌리인 AWT의 핵심내용만을 포함시키는 것이 더 낫다고 생각했다.

7.1 paint()와 Graphics

paint()는 컴포넌트에 그림을 그리기 위한 것으로 Component클래스에 정의되어 있어서 모든 컴포넌트에서 사용할 수 있다. 그림을 그리고자 하는 컴포넌트, 주로 Frame, Canvas, Panel 등의 paint()를 알맞게 오버라이딩하면 된다.

paint()의 선언부는 다음과 같이 Graphics를 매개변수로 정의하고 있으며, paint()를 작성할 때는 Graphics가 가지고 있는 다양한 그래픽 메서드를 이용해서 그림을 그리게 된다.

```
public void paint(Graphics g) {
      ...
}
```

모든 컴포넌트는 Graphics객체를 가지고 있으며, Graphics객체에는 각 컴포넌트의 그래픽 관련된 정보가 담겨져 있다. 그래서 컴포넌트에 그림을 그리고자 할 때는 getGraphics()를 호출해서 컴포넌트의 Grapphics객체에 대한 참조를 먼저 얻어낸 다음, Graphics객체가 가지고 있는 다양한 그래픽 메서드를 이용해서 작업하면 된다. 예를 들어 Panel에 선을 그리고자 한다면 다음과 같이 할 수 있다.

```
Panel p = new Panel();
Graphics g = p.getGraphics();
g.drawLine(0,0, 10, 10); //Panel에 두 점 (0,0)과 (10,10)을 잇는 선을 그린다.
```

[참고] 여기서 말하는 Graphics객체라는 것은 추상클래스인 Graphics를 완전히 구현한 클래스의 객체를 말한다.

Graphics가 제공하는 주요 메서드는 다음과 같다.

1. 문자출력 관련된 메서드 - 문자열을 출력할 수 있으며, 출력할 문자열의 폰트(font)를 지정할 수 있다. FontMetrics객체는 지정된 폰트에 대한 상세한 정보를 제공한다.

void drawString(String str, int x, int y) - (x, y)위치에 문자열을 출력한다.
Font getFont() - 현재 설정되어 있는 Font를 얻는다.
void setFont(Font font) - 출력할 문자열의 폰트를 지정한다.
FontMetrics getFontMetrics() - FontMetrics객체를 반환한다.

2. 색과 관련된 메서드 - 출력할 그림의 색을 지정하거나 지정된 색의 정보를 얻을 수 있다.
Color getColor() - 현재 설정되어 있는 Color를 얻는다.
void setColor(Color c) - 출력할 색을 지정한다.

3. 도형과 관련된 메서드 - 다양한 도형을 그리는 메서드. draw로 시작하는 것은 도형의 선만 그리고, fill로 시작하는 것은 도형의 내부를 선의 색으로 채운다.
- 선을 그리는 메서드
  void drawLine(int x1, int y1, int x2, int y2)
- 타원을 그리는 메서드
  void drawOval(int x, int y, int width, int height)
  void fillOval(int x, int y, int width, int height)
- 사각형을 그리는 메서드
  void drawRect(int x, int y, int width, int height)
  void fillRect(int x, int y, int width, int height)
- 둥근 사각형을 그리는 메서드
  void drawRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight)
  void fillRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight)
- 입체 사각형을 그리는 메서드
  void draw3DRect(int x, int y, int width, int height, boolean raised)
  void fill3DRect(int x, int y, int width, int height, boolean raised)
- 호를 그리는 메서드
  void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)
  void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle)
- 다각형을 그리는 메서드
  void drawPolygon(int[] xPoints, int[] yPoints, int nPoints)
  void fillPolygon(int[] xPoints, int[] yPoints, int nPoints)
  void drawPolyline(int[] xPoints, int[] yPoints, int nPoints)

4. 기타
void clearRect(int x, int y, int width, int height) - 지정된 영역을 지운다.
boolean drawImage(Image img, int x, int y, ImageObserver observer) - (x,y)의 위치에 이미지를 그린다.

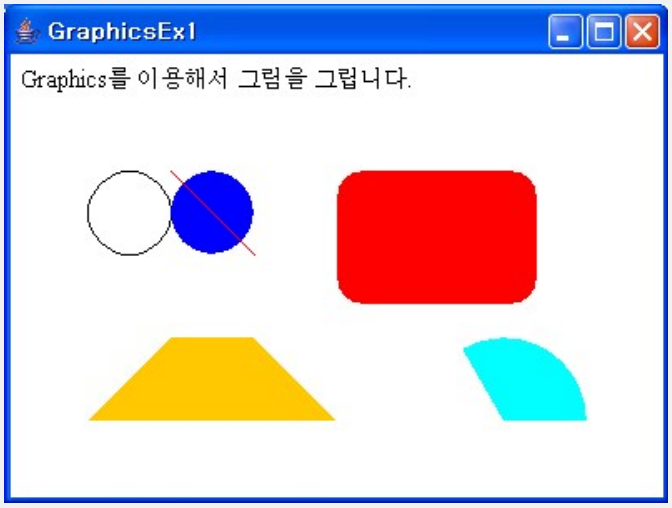
다음은 Graphics가 제공하는 다양한 메서드를 이용해서 Frame에 그림을 그리는 예제이다.

[예제13-40]/ch13/GraphicsEx1.java

```
import java.awt.*;
import java.awt.event.*;

class GraphicsEx1 extends Frame {
     public static void main(String[] args) {
```

```
            new GraphicsEx1("GraphicsEx1");
      }

      public void paint(Graphics g) {
            g.setFont(new Font("Serif", Font.PLAIN, 15));
            g.drawString("Graphics를 이용해서 그림을 그립니다." ,10, 50);

            g.drawOval(50,100, 50,50);
            g.setColor(Color.blue);
            g.fillOval(100,100, 50,50);

            g.setColor(Color.red);
            g.drawLine(100,100, 150,150);

            g.fillRoundRect(200, 100, 120, 80, 30, 30);
            g.setColor(Color.orange);
            g.fillPolygon(new int[]{ 50, 100, 150, 200},
                          new int[]{250, 200, 200, 250}, 4);
            g.setColor(Color.cyan);
            g.fillArc(250, 200, 100, 100, 0, 120);
      }

      public GraphicsEx1(String title) {
            super(title);
            addWindowListener( new WindowAdapter() {
                  public void windowClosing(WindowEvent we) {
                        System.exit(0);
                  }
            });

          // Frame을 (100,100)의 위치에 width 400, height 300 크기로 보이게 한다.
            setBounds(100,100, 400,300);
            setVisible(true);
      }
} // class
```

[실행결과]



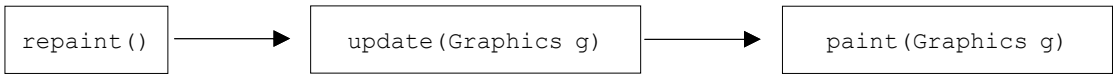
7.2 AWT쓰레드와 repaint()

바로 전 예제에서 'paint()를 오버라이딩했다는 것은 알겠는데 호출하는 부분이 없는데 어떻게 그림이 그려졌을 까?'라는 의문이 들었을 것이다.
그 것은 바로 paint()는 우리가 직접 호출하지 않아도 AWT쓰레드에 의해서 자동적으로 호출되기 때문이다.
AWT쓰레드는 GUI어플리케이션을 실행하면 자동적으로 생성되는 데몬 쓰레드로 평소에는 대기상태에 있다가 화면이 갱신되어야할 상황이 발생되면, paint()또는 update()를 호출해서 화면이 다시 그려지도록 한다.
AWT쓰레드에 의해 자동적으로 화면이 갱신되는 상황은 다음과 같다.

- 처음 화면에 나타날 때
- 다른 화면에 가려져 있던 부분이 다시 화면에 나타날 때
- 아이콘화 되어 있다가 원래 크기로 화면에 나타날 때

위의 상황 이외에도 화면이 다시 그려지도록 하려면 repaint()를 호출하면 된다. repaint()는 AWT쓰레드에게 화면을 다시 그릴 것을 요청한다. AWT쓰레드는 update()를 호출하고 update()는 다시 paint()를 호출한다.

```
repaint() - AWT쓰레드에게 화면을 갱신할 것을 요청한다.
            AWT쓰레드는 0.1초마다 확인해서 요청이 있으면 update()를 호출한다.
update(Graphics g) - 화면을 지우고 paint(Graphics g)를 호출한다.
```



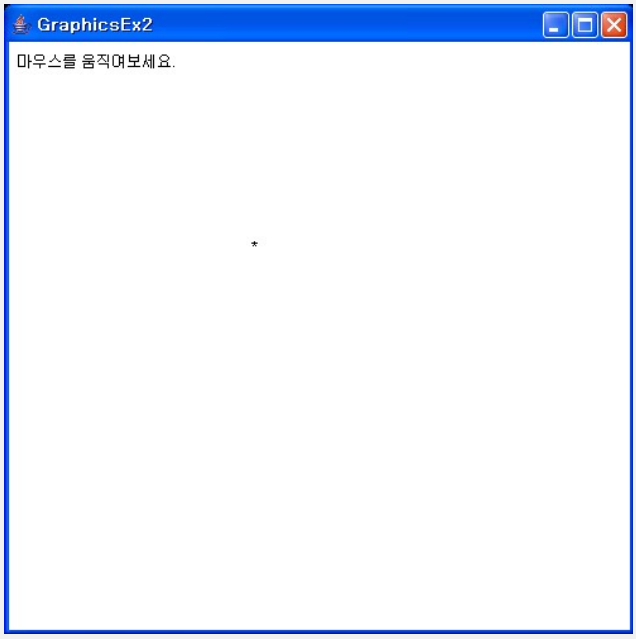
[그림13-13] repaint()에 의해 호출되는 메서드

[예제13-41]/ch13/GrapihcsEx2.java

```
import java.awt.*;
import java.awt.event.*;

class GraphicsEx2 extends Frame implements MouseMotionListener {
    int x = 0;
    int y = 0;

    public static void main(String[] args) {
          new GraphicsEx2("GraphicsEx2");
    }

    public GraphicsEx2(String title) {
          super(title);
          addMouseMotionListener(this);
          addWindowListener( new WindowAdapter() {
                public void windowClosing(WindowEvent we) {
```

```
                    System.exit(0);
                }
            });

            // Frame을 (100, 100)의 위치에 width 500, height 500 크기로 보이게 한다.
            setBounds(100,100, 500,500);
            setVisible(true);
        }

        public void paint(Graphics g) {
            g.drawString("마우스를 움직여보세요." ,10, 50);
            g.drawString("*", x, y);
        }

        public void mouseMoved(MouseEvent me) {
            x = me.getX();
            y = me.getY();
            repaint();
        }

        public void mouseDragged(MouseEvent me) {}
    } // class
```

[실행결과]



높이와 폭이 500픽셀인 Frame을 생성하고, 마우스 포인터의 위치에 '*'을 출력하는 예제이다. Frame영역 내에서 마우스 포인터를 이동시키면 '*'도 따라 움직이게 된다.

```
public void mouseMoved(MouseEvent me) {
    x = me.getX();
    y = me.getY();
    repaint();
}
```

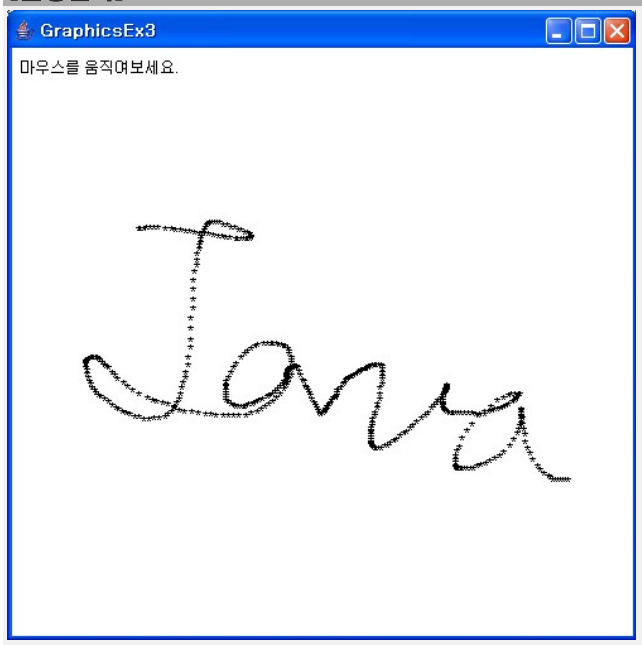
마우스를 움직이면 MouseEvent가 발생하고 이벤트 핸들러의 mouseMoved()가 호출된다. MouseEvent의 getX(), getY()를 통해 포인터의 위치를 얻어서 멤버변수 x, y에 저장한다. 그리고 repaint()를 호출해서 Frame을 화면에 다시 그리도록 해야 새로운 위치에 '*'이 출력된다.

만일 repaint()를 호출하지 않으면 마우스를 움직여도 '*'이 이동하지 않는다. repaint()를 주석처리해서 컴파일하고 실행한 다음 그 결과를 직접 확인해보자.

[예제13-42]/ch13/GrapihcsEx3.java

```
import java.awt.*;
import java.awt.event.*;

class GraphicsEx3 extends Frame implements MouseMotionListener {
    int x = 0;
    int y = 0;

    public static void main(String[] args) {
          new GraphicsEx3("GraphicsEx3");
    }

    public GraphicsEx3(String title) {
          super(title);
          addMouseMotionListener(this);
          addWindowListener( new WindowAdapter() {
                public void windowClosing(WindowEvent we) {
                      System.exit(0);
                }
          });

          // Frame을 (100, 100)의 위치에 width 500, height 500 크기로 보이게 한다.
          setBounds(100,100, 500,500);
          setVisible(true);
    }

    public void paint(Graphics g) {
          g.drawString("마우스를 움직여보세요." ,10, 50);
          g.drawString("*", x, y);
    }

    public void update(Graphics g) {
          paint(g);
    }

    public void mouseMoved(MouseEvent me) {
          x = me.getX();
          y = me.getY();
          repaint();
    }

    public void mouseDragged(MouseEvent me) {}
} // class
```

[실행결과]



전에 설명한 것과 같이 update()는 컴포넌트의 영역을 지우고배경색으로 채우고 paint()를 호출한다. 그러나 이 예제에서는 아래와 같이 update()가 컴포넌트 영역을 지우지 않고 paint()만 호출하도록 오버라이딩하였다.

```
public void update(Graphics g) {
      paint(g);
}
```

이렇게 하면 이전 예제와는 달리 마우스 포인터의 자취가 남게 된다.

이번엔 윈도우에서 'Alt+ tab'키를 눌러서 Frame이 다른 화면에 가려지게 했다가 다시 'Alt+ tab'키를 눌러서 다시 화면에 나타나게 해보자.

Frame이 다른 화면에 의해 가려졌다가 다시 나타나는 경우에는 자동적으로 Frame영역을 완전히 새로 그리기 때문에 포인터의 자취가 사라지게 된다. 그래도 이 과정에서 paint()는 호출되기 때문에 paint()에서 그린 부분은 Frame에 다시 그려진다.

```
public void paint(Graphics g) {
      g.drawString("마우스를 움직여보세요." ,10, 50);
      g.drawString("*", x, y);
}
```

화면이 가려졌다가 나타나도 지워지지 않게 하는 방법은 바로 다음 단원에서 배울 것이다.

7.3 Image를 이용해서 이미지 출력하기

Image클래스는 gif, jpg와 같은 형식의 이미지 파일을 보여주거나 가상화면(off-screen)을 생성하는데 사용된다. 예를 들어 getjava.gif라는 파일을 보여주려면 다음과 같은 코드를 사용하면 된다.

```
Toolkit tk = Toolkit.getDefaultToolkit();
Image img = tk.getImage("getjava.gif");
g.drawImage(img, 0, 0, this);
```

drawImage(Image img, int x, int y, ImageObserver obj)는 Graphics가 제공하는 것으로 이미지(img)를 지정된 위치(x, y)에 출력한다. 이미지가 로딩되는데 시간이 걸리기 때문에 로딩이 진행되는 상태에 따라 화면에 다시 그려줘야 할 필요가 있다. 그래서 이미지가 로딩되고 있는 상태를 알려줄 대상을 ImageObserver로 지정해야한다.
모든 AWT컴포넌트는 ImageObserver인터페이스를 구현했기 때문에 ImageObserver로 컴포넌트 자신(this)을 지정하는 것이 보통이다.

[그림13-14] Java API문서에서 Component를 검색한 화면

보통 실제화면에 직접 그림을 그리는 대신 가상화면에 그림을 그린 다음에 실제화면으로 복사하는 경우가 많다. 실제화면에 직접 그림을 그리면 깜빡거림이 생기기도 하고, 이전 예제에서처럼 실행 시에 그린 그림이 보존되지 않기 때문이다.

다음은 코드는 createImage()를 이용해서 폭이 500, 높이가 500인 Image객체를 생성한 다음 getGraphics()를 통해 Image객체의 Graphics객체를 얻어서 문자열을 출력한다.
모든 그래픽 작업을 마친 다음에는 drawImage()를 이용해서 가상화면에 그려진 그림을 실제화면으로 복사한다.

```
Image img = createImage(500, 500);
Graphics gImg= img.getGraphics();
gImg.drawString("왼쪽버튼을 누른 채로 마우스를 움직여보세요." ,10, 50);
// ...
g.drawImage(img,0,0,this);  // 가상화면에 그려진 그림을 복사한다.
```

다음 예제는 이전 예제를 가상화면을 사용하도록 변경한 것으로 실행 시에 마우스로 그린 그림이 다른 화면에 가려졌다가 다시 나타나도 지워지지 않는다.

[예제13-43]/ch13/GrapihcsEx4.java

```
import java.awt.*;
import java.awt.event.*;

class GraphicsEx4 extends Frame implements MouseMotionListener {
    int x = 0;
    int y = 0;

    Image img = null;
    Graphics gImg = null;

    public static void main(String[] args) {
          new GraphicsEx4("GraphicsEx4");
    }

    public GraphicsEx4(String title) {
          super(title);
          addMouseMotionListener(this);
          addWindowListener( new WindowAdapter() {
                public void windowClosing(WindowEvent we) {
                      System.exit(0);
                }
          });

          // Frame을 (100, 100)의 위치에 width 500, height 500 크기로 보이게 한다.
          setBounds(100,100, 500,500);
          setVisible(true);

          img = createImage(500, 500);
          gImg = img.getGraphics();
          gImg.drawString("왼쪽버튼을 누른 채로 마우스를 움직여보세요." ,10, 50);
          repaint();
    }

    public void paint(Graphics g) {
          if(img!=null)
                      g.drawImage(img,0,0,this); // 가상화면에 그려진 그림을 Frame에 복사
    }

    public void mouseMoved(MouseEvent me) {}

    public void mouseDragged(MouseEvent me) {
          if (me.getModifiersEx()==MouseEvent.BUTTON1_DOWN_MASK ) {
                x = me.getX();
                y = me.getY();
                gImg.drawString("*", x, y);
                repaint();
          }
    }
} // class
```

[실행결과]

Image와 Image의 Graphics를 멤버변수로 정의하고, 생성자에서 createImage()를 통해서 Frame과 같은 크기의 Image를 생성하고, 생성된 Image에서 getGraphics()를 호출하여 Image에 대한 Graphics를 얻는다. 이렇게 해서 얻어진 Graphics에 drawString()과 같은 메서드를 호출해서 작업한 내용은 Image에 그려지게 된다.

```
Image img = null;
Graphics gImg = null;

public GraphicsEx4(String title) {
    ...
    img = createImage(500, 500);
    gImg = img.getGraphics();
    gImg.drawString("왼쪽버튼을 누른 채로 마우스를 움직여보세요." ,10, 50);
}
```

paint()에서는 단순히 가상화면(img)에 그려진 내용을 Frame에 복사하기만 하면 된다.

```
public void paint(Graphics g) {
    if(img!=null)
        g.drawImage(img,0,0,this);  // 가상화면에 그려진 그림을 Frame에 복사
}
```

이전 예제와는 달리 왼쪽 버튼을 누른 상태로 포인터를 이동했을 때만 그림이 그려지도록 하기 위해 mouseDragged()를 구현하였다.

```
public void mouseDragged(MouseEvent me) {
    if (me.getModifiersEx()==MouseEvent.BUTTON1_DOWN_MASK ) {
          x = me.getX();
          y = me.getY();
          gImg.drawString("*", x, y);
          repaint();
    }
}
```

드래깅할 때 Ctrl키가 눌러졌는지 확인하려면 다음과 같이 .BUTTON1_DOWN_MASK와 CTRL_DOWN_MASK를 더해주면 된다.

```
if(me.getModifiersEx()==MouseEvent.BUTTON1_DOWN_MASK+ MouseEvent.CTRL_DOWN_MASK)
```

[예제13-44]/ch13/GrapihcsEx5.java

```
import java.awt.*;
import java.awt.event.*;

class GraphicsEx5 extends Frame implements MouseMotionListener
{
    int x = 0;
    int y = 0;

    Image img = null;
    Graphics gImg = null;

    public static void main(String[] args) {
          new GraphicsEx5("GraphicsEx5");
    }

    public GraphicsEx5(String title) {
          super(title);
          addMouseMotionListener(this);
          addWindowListener( new WindowAdapter() {
                public void windowClosing(WindowEvent we) {
                      System.exit(0);
                }
          });

          // Frame을 (100, 100)의 위치에 width 500, height 500 크기로 보이게 한다.
          setBounds(100,100, 500,500);
          setVisible(true);

          img = createImage(500, 500);
          gImg = img.getGraphics();
          gImg.drawString("왼쪽버튼을 누른 채로 마우스를 움직여보세요." ,10, 50);
          repaint();
    }
```

```
    public void paint(Graphics g) {
          if(img==null) return;
          g.drawImage(img,0,0,this);      // 가상화면에 그려진 그림을 Frame에 복사
    }

    public void mouseMoved(MouseEvent me) {
          x = me.getX();
          y = me.getY();
    }
    public void mouseDragged(MouseEvent me) {
          if (me.getModifiersEx()!=MouseEvent.BUTTON1_DOWN_MASK ) return;

          gImg.drawLine(x,y, me.getX(),me.getY());
          x = me.getX();
          y = me.getY();

          repaint();
    }
} // class
```

[실행결과]

전의 예제에서 drawString()대신 drawLine()을 사용해서 Frame에 자유롭게 선을 그릴 수 있도록 하였다. 이전 예제와 비교해보고 어떤 차이가 있는지 확인해보자.

[예제13-45]/ch13/GrapihcsEx6.java

```
import java.awt.*;
import java.awt.event.*;

class GraphicsEx6 extends Frame {
    Image img = null;
```

```
    public GraphicsEx6(String title) {
        super(title);
        addWindowListener( new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });

        Toolkit tk = Toolkit.getDefaultToolkit();
        img = tk.getImage("getjava.gif");

        setBounds(100,100, 400,300);
        setVisible(true);
    }

    public static void main(String[] args) {
        new GraphicsEx6("GraphicsEx6");
    }

    public void paint(Graphics g) {
        if(img==null) return;

        // Image의 크기를 얻는다.
        int imgWidth = img.getWidth(this);   // getWidth(ImageObserver obj)
        int imgHeight = img.getHeight(this); // getHeight(ImageObserver obj)

        // 이미지를 Frame의 중앙에 출력한다.
        g.drawImage(img,(getWidth()-imgWidth)/2,
                                        (getHeight()-imgHeight)/2, this);
    }
} // class
```

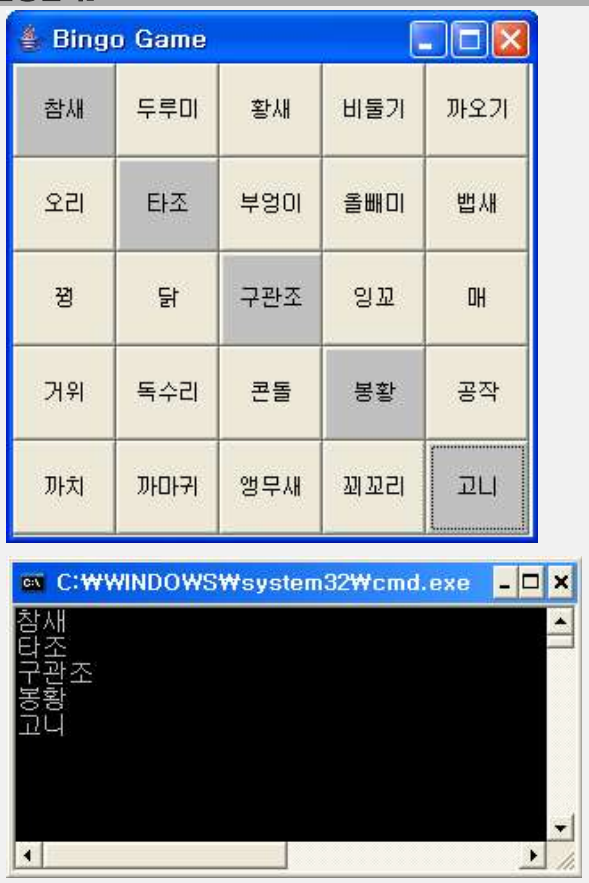
[실행결과]

getjava.gif라는 이미지를 화면의 중앙에 보여주는 예제이다. 파일명은 경로를 함께 지정해주는 것이 좋으며, 예제의 클래스파일과 같은 경로에 있는 경우 경로명을 생략할 수 있다.

[연습문제]

[13-1] 아래의 실행결과처럼 가로, 세로 크기가 5인 빙고판을 만드시오. 그리고 버튼을 누르면, 버튼의 배경색이 밝은 회색(Color.LIGHT_GRAY)으로 바뀌고 누른 버튼의 글자가 콘솔에 출력되게 하시오.

[실행결과]



[13-2] 다음 중 기본 레이아웃 매니저가 BorderLayout이 아닌 컨테이너는? (모두 고르시오)

a. Window
b. Panel
c. Applet
d. Frame
e. Dialog

[13-3] 다음 중 ActionEvent가 발생하는 경우가 아닌 것은? (모두 고르시오)

a. Button을 클릭했을 때
b. Menu를 클릭했을 때
c. TextArea에서 Enter키를 눌렀을 때
d. List에서 item을 선택하여 클릭했을 때
e. Label을 더블 클릭했을 때

[13-4] 타이틀 바에 현재 시간을 출력하는 시계를 작성하시오. 단, 쓰레드를 이용해서 시간이 1초마다 계속 변경되도록 해야 한다.

[Hint] SimpleDateFormat을 사용하라.

[실행결과]

[13-5] 다음의 코드는 현재 년과 월을 보여주고 왼 쪽의 버튼을 누르면 이전 달로, 오른쪽 버튼을 누르면 다음 달로 이동하는 프로그램이다. 알맞은 코드를 넣어 프로그램을 완성하시오.

[연습문제]/ch13/Exercise13_5.java

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.*;

class Exercise13_5 extends Frame {
    Panel  pUp        = new Panel();
    Button btnPrevMon = new Button("◀");
    Button btnNextMon = new Button("▶");
    Label  lblYearMon = new Label();

    Calendar curMon = Calendar.getInstance();

    Exercise13_5(String title) {
          super(title);

          pUp.setBackground(Color.yellow);
          pUp.setLayout(new FlowLayout(FlowLayout.CENTER));
          pUp.add(btnPrevMon);
          pUp.add(lblYearMon);
          pUp.add(btnNextMon);

          btnPrevMon.addActionListener(new BtnEventHandler());
          btnNextMon.addActionListener(new BtnEventHandler());
          addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent we) {
                      we.getWindow().setVisible(false);
                      we.getWindow().dispose();
                      System.exit(0);
                }
          });

          add(pUp);
          setBounds(200,200,200,70);
          setDays(curMon);
          setVisible(true);
    } // Exercise13_5

    void setDays(Calendar date) {
          int year  = date.get(Calendar.YEAR);
          int month = date.get(Calendar.MONTH);

          lblYearMon.setText(year+"년 "+(month+1)+"월");
    }

    class BtnEventHandler implements ActionListener {
          public void actionPerformed(ActionEvent ae) {
```

```
                /*
                        (1) 아래의 로직에 맞게 코드를 작성하시오.
                        1. 눌러진 버튼이 btnPrevMon이면, curMon을 한 달 이전으로 변경한다.
                        2. 눌러진 버튼이 btnNextMon이면, curMon을 한 달 이후로 변경한다.
                        3. setDays()를 호출해서 변경된 내용이 레이블에 보이게 한다.
                */
            }
        }

        public static void main(String[] args) {
            Exercise13_5 mainWin = new Exercise13_5("Scheduler");
        } // main
    }
```

[실행결과]

[13-6] 다음의 코드는 문제13-5를 발전시켜서 해당 월의 날짜를 버튼으로 추가하는 코드이다. (1)~(2)에 알맞은 코드를 넣어 프로그램을 완성하시오.

[연습문제]/ch13/Exercise13_6.java

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.*;

class Exercise13_6 extends Frame {
    Panel  pDate      = new Panel();
    Panel  pUp        = new Panel();

    Button btnPrevMon = new Button("◀");
    Button btnNextMon = new Button("▶");
    Label  lblYearMon = new Label();

    Button[] btnArr = new Button[42];

    Calendar curMon = Calendar.getInstance();

    Exercise13_6(String title) {
         super(title);

         pUp.setBackground(Color.yellow);
         pUp.setLayout(new FlowLayout(FlowLayout.CENTER));
         pUp.add(btnPrevMon);
         pUp.add(lblYearMon);
         pUp.add(btnNextMon);


         pDate.setLayout(new GridLayout(6,7));
         for(int i=0; i < btnArr.length;i++) {
              btnArr[i] = new Button("");
              pDate.add(btnArr[i]);
         }

         btnPrevMon.addActionListener(new BtnEventHandler());
         btnNextMon.addActionListener(new BtnEventHandler());
         addWindowListener(new WindowAdapter() {
              public void windowClosing(WindowEvent we) {
                   we.getWindow().setVisible(false);
                   we.getWindow().dispose();
                   System.exit(0);
              }
         });

         add(pUp,"North");
         add(pDate,"Center");
         setBounds(200,200,500,300);
         setDays(curMon);
         setVisible(true);
```

```
    } // MyScheduler

    void setDays(Calendar date) {
          int year  = date.get(Calendar.YEAR);
          int month = date.get(Calendar.MONTH);

          lblYearMon.setText(year+"년 "+(month+1)+"월");

          Calendar sDay = Calendar.getInstance(); // 시작일

          /*
             (1) 아래의 로직에 맞게 코드를 작성하시오.
             1. 시작일(sDay)을 해당 월의 1일이 포함된 일요일로 설정한다.
             2. 반복문을 사용해서 sDay의 값을 1씩 증가시키면서 버튼에 날짜를 넣는다.
             3. 만일 날짜가 해당 월에 속하면 버튼의 배경색을 흰색(Color.white)으로 하고
                그렇지 않으면 밝은 회색(Color.lightGray)으로 설정한다.
          */
    } // setDays(Calendar date)

    class BtnEventHandler implements ActionListener {
          public void actionPerformed(ActionEvent ae) {
                /*
                      (2) 아래의 로직에 맞게 코드를 작성하시오.
                      1. 눌러진 버튼이 btnPrevMon이면, curMon을 한 달 이전으로 변경한다.
                      2. 눌러진 버튼이 btnNextMon이면, curMon을 한 달 이후로 변경한다.
                      3. setDays()를 호출해서 변경된 내용이 레이블에 보이게 한다.
                */
          }
    }

    public static void main(String[] args) {
          Exercise13_6 mainWin = new Exercise13_6("Scheduler");
    } // main
}
```

[실행결과]

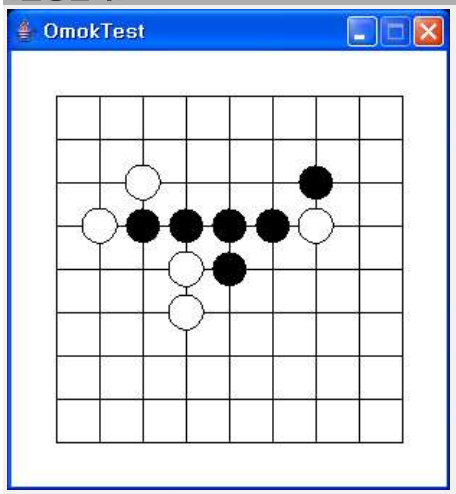
[13-7] 다음의 코드는 오목게임이다. Frame에 오목판을 그리고, 그 위에 왼쪽 버튼을 누르면 검은 돌이 오른쪽 버튼을 누르면 흰 돌이 그려지도록 알맞은 코드를 완성하시오. 단, 돌은 클릭한 곳의 가장 가까운 교차점에 놓여야 한다.

[연습문제]/ch13/Exercise13_7.java

```
import java.awt.*;
import java.awt.event.*;

class Exercise13_7 extends Frame implements MouseListener  {
    final int LINE_NUM   = 9;   // 오목판 줄 수
    final int LINE_WIDTH = 30; // 오목판 줄 간격
    final int BOARD_SIZE = LINE_WIDTH * (LINE_NUM-1); // 오목판의 크기
    final int STONE_SIZE = (int)(LINE_WIDTH * 0.8);    // 돌의 크기

    final int X0; // 오목판 시작위치 x좌표
    final int Y0; // 오목판 시작위치 y좌표

    final int FRAME_WIDTH;   // Frame의 폭
    final int FRAME_HEIGHT;  // Frame의 높이

    Image img = null;
    Graphics gImg = null;

    public Exercise13_7(String title) {
          super(title);

          // Event Handler를 등록한다.
          addMouseListener(this);
          addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent we) {
                      System.exit(0);
                }
          });

          setVisible(true);     // Frame을 화면에 보이게 한다.
          Insets insets = getInsets(); // 화면에 보이기 전에는 Insets의 값을 얻을 수 없다.

          // 오목판 그려질 위치(LEFT, TOP)의 좌표 X0, Y0를 지정한다.
          X0 = insets.left + LINE_WIDTH;
          Y0 = insets.top  + LINE_WIDTH;

          // Frame의 크기를 계산한다.
          FRAME_WIDTH  = BOARD_SIZE+LINE_WIDTH*2+insets.left+insets.right;
          FRAME_HEIGHT = BOARD_SIZE+LINE_WIDTH*2+insets.top+insets.bottom;

          // Frame을 화면의 (100,100)의 위치에 계산된 크기로 보이게 한다.
          setBounds(100,100, FRAME_WIDTH, FRAME_HEIGHT);

          img = createImage(FRAME_WIDTH, FRAME_HEIGHT);
          gImg = img.getGraphics();

          setResizable(false); // Frame의 크기를 변경하지 못하게 한다.
          drawBoard(gImg);
    } // Exercise13_7(String title)

    public void drawBoard(Graphics g) {
```

```
        for(int i=0; i<LINE_NUM;i++) {
            g.drawLine(X0,Y0+i*LINE_WIDTH,X0+BOARD_SIZE, Y0+i*LINE_WIDTH);
            g.drawLine(X0+i*LINE_WIDTH,Y0, X0+i*LINE_WIDTH, Y0+BOARD_SIZE);
        }
    }

    public void paint(Graphics g) {
        if(img==null) return;

        g.drawImage(img,0,0,this); // 가상화면에 그려진 그림을 Frame에 복사
    }

    public void mousePressed(MouseEvent e) {          // MouseListener
        int x = e.getX();      // 마우스 포인터의 x좌표
        int y = e.getY();      // 마우스 포인터의 y좌표

        /*
         (1) 아래의 로직에 맞게 코드를 작성하시오.
          1. x와 y의 값이 오목판의 밖을 벗어난 곳이면 돌을 그리지 않는다.
          2. x와 y의 값을 클릭한 곳에서 가장 가까운 교차점으로 변경한다.(반올림)
          3. x와 y의 값에서 돌의 크기(STONE_SIZE)의 절반을 빼야 클릭한 곳에 돌이 그려진다.
          4. 눌러진 버튼이 마우스 왼쪽 버튼이면, (x,y)의 위치에 검은 돌을 그리고
             눌러진 버튼이 마우스 오른쪽 버튼이면, (x,y)의 위치에 흰 돌을 그린다.
             (흰 돌을 그릴 때 검은 색 테두리도 같이 그린다.)
          5. repaint()를 호출한다.
        */
    }

    public void mouseClicked(MouseEvent e) {}   // MouseListener
    public void mouseEntered(MouseEvent e) {}   // MouseListener
    public void mouseExited(MouseEvent e) {}    // MouseListener
    public void mouseReleased(MouseEvent e) {} // MouseListener

    public static void main(String[] args) {
        new Exercise13_7("OmokTest");
    }
}
```

[실행결과]



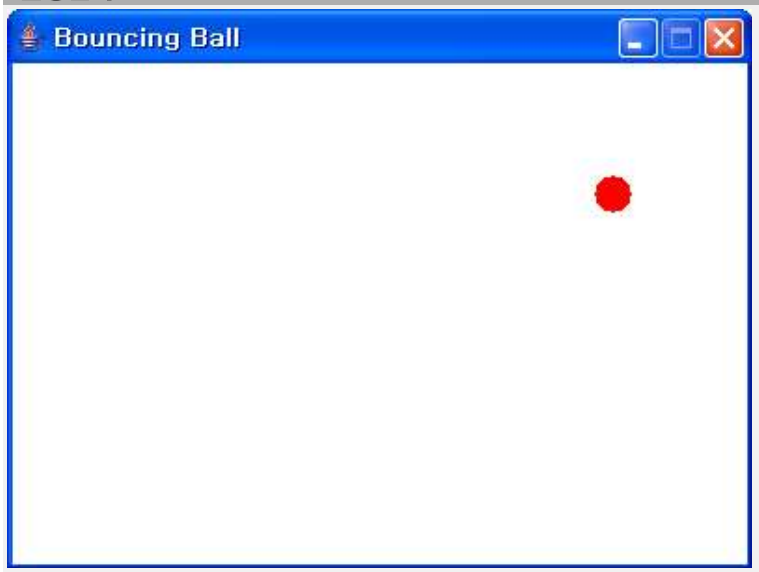
[13-8] 다음은 Frame의 내부를 튀어 다니는 공을 그리는 프로그램이다. 공이 Frame의 벽에 부딪히면 방향을 바꾸어 이동한다. 알맞은 코드를 넣어 완성하시오.

[연습문제]/ch13/Exercise13_8.java

```
import java.awt.*;
import java.awt.event.*;


class BouncingBall extends Frame {
    final int BALL_SIZE = 20;
    final int FRAME_WIDTH  = 400;
    final int FRAME_HEIGHT = 300;

    final int TOP;
    final int BOTTOM;
    final int LEFT;
    final int RIGHT;
    final int SPEED = 3;

    int x = 100;
    int y = 100;

    int xStep = SPEED;
    int yStep = SPEED;

    BouncingBall(String title) {
          super(title);

          addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent e) {
                      System.exit(0);
                }
          });

          setBounds(300,200, FRAME_WIDTH, FRAME_HEIGHT);
          setResizable(false);
          setVisible(true);

          Insets insets = getInsets(); // Frame의 테두리의 두께를 얻어온다.

          TOP   = insets.top;
          LEFT  = insets.left;
          BOTTOM = FRAME_HEIGHT - insets.bottom ;
          RIGHT  = FRAME_WIDTH  - insets.right;
    }

    void start() {
          while(true) {
                x +=xStep;
                y +=yStep;

          /*
            (1) 아래의 로직에 맞게 코드를 작성하시오.
            1. x의 값이 왼쪽 테두리(LEFT)보다 작거나 같으면
               x의 값을 LEFT로 바꾸고 x축 이동방향(xStep)을 반대로 한다.
```

```
            2. x의 값이 오른쪽 테두리(RIGHT-BALL_SIZE)보다 작거나 같으면
               x의 값을 RIGHT-BALL_SIZE로 바꾸고 x축 이동방향(xStep)을 반대로 한다.
            3. y의 값이 윗 쪽 테두리(TOP)보다 작거나 같으면
               y의 값을 TOP으로 바꾸고 y축 이동방향(yStep)을 반대로 한다.
            4. y의 값이 아래 쪽 테두리(BOTTON-BALL_SIZE)보다 작거나 같으면
               y의 값을 BOTTON-BALL_SIZE로 바꾸고 y축 이동방향(yStep)을 반대로 한다.
         */

               repaint();

               try {
                     Thread.sleep(100);
               } catch (Exception e) {}
         }
    } // start()

    public void paint(Graphics g) {
         g.setColor(Color.RED);
         g.fillOval(x, y, BALL_SIZE, BALL_SIZE);
    }
}

class Exercise13_8 {
    public static void main(String[] args) {
         new BouncingBall("Bouncing Ball").start();
    }
}
```

[실행결과]



[13-9] 문제13-8을 발전시켜서 BallGenerator라는 쓰레드를 생성시켜서 3초마다 새로운 공을 하나씩 생성하는 프로그램이다. 알맞은 코드를 넣어 프로그램을 완성하시오.

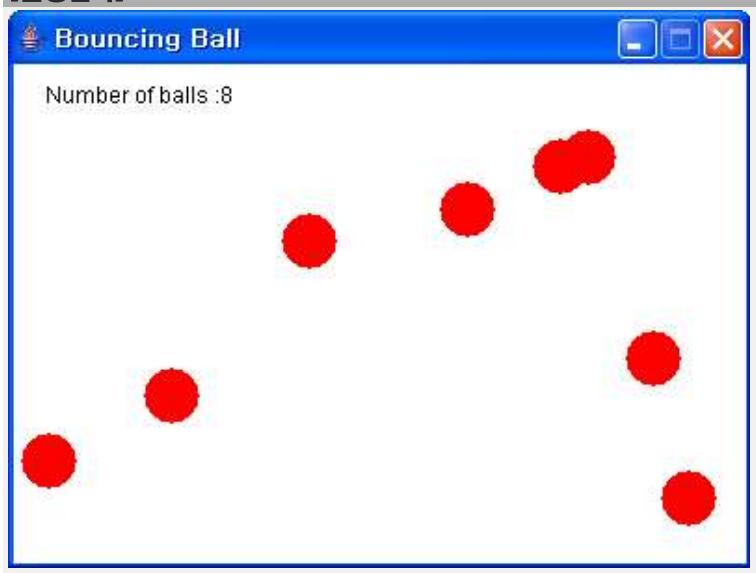
[연습문제]/ch13/Exercise13_9.java

```
import java.util.*;
import java.awt.*;
import java.awt.event.*;

class BouncingBall extends Frame {
    final int FRAME_WIDTH  = 400;
    final int FRAME_HEIGHT = 300;

    final int TOP;
    final int BOTTOM;
    final int LEFT;
    final int RIGHT;

    ArrayList balls = new ArrayList();

    BouncingBall(String title) {
          super(title);

          setBounds(300,200, FRAME_WIDTH, FRAME_HEIGHT);
          setResizable(false);
          setVisible(true);

          Insets insets = getInsets();

          TOP   = insets.top;
          LEFT  = insets.left;
          BOTTOM = FRAME_HEIGHT - insets.bottom;
          RIGHT  = FRAME_WIDTH  - insets.right;

          addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent e) {
                      System.exit(0);
                }
          });
    }

    void start() {
          new BallGenerator().start();

          while(true) {
                int size = balls.size();

                for(int i=0 ; i < size; i++) {
                      Ball b = (Ball)balls.get(i);

                      b.x +=b.xStep;
                      b.y +=b.yStep;

          /*
             (1) 아래의 로직에 맞게 코드를 작성하시오.
             1. x의 값이 왼쪽 테두리(LEFT)보다 작거나 같으면
```

```
                x의 값을 LEFT로 바꾸고 x축 이동방향(xStep)을 반대로 한다.
            2. x의 값이 오른쪽 테두리(RIGHT-BALL_SIZE)보다 작거나 같으면
                x의 값을 RIGHT-BALL_SIZE로 바꾸고 x축 이동방향(xStep)을 반대로 한다.
            3. y의 값이 윗 쪽 테두리(TOP)보다 작거나 같으면
                y의 값을 TOP으로 바꾸고 y축 이동방향(yStep)을 반대로 한다.
            4. y의 값이 아래 쪽 테두리(BOTTON-BALL_SIZE)보다 작거나 같으면
                y의 값을 BOTTON-BALL_SIZE로 바꾸고 y축 이동방향(yStep)을 반대로 한다.
        */
            }

            repaint();

            try {
                Thread.sleep(100);
            } catch (Exception e) {}
        }
    } // start()

    public void paint(Graphics g) {
        g.drawString("Number of balls :"+balls.size(),20,50);
        g.setColor(Color.RED);

        int size = balls.size();

        for(int i=0 ; i < size; i++) {
            Ball b = (Ball)balls.get(i);
            g.fillOval(b.x, b.y, Ball.SIZE, Ball.SIZE);
        }
    }

    class BallGenerator extends Thread {
        public void run() {
            /*
              (2) 아래의 로직에 맞게 코드를 작성하시오.
              1. 3초마다 Frame의 임의의 위치에 공을 생성해서 balls에 추가한다.
            */
        } // run()
    }

    class Ball {
        int x = 100;
        int y = 100;

        static final int SIZE = 30;

        final int SPEED = 5;
        int xStep = SPEED;
        int yStep = SPEED;

        Ball(int x, int y) {
            this.x = x;
            this.y = y;
        }
    }
} // class BouncingBall
```

```
class Exercise13_9 {
    public static void main(String[] args) {
          new BouncingBall("Bouncing Ball").start();
    }
}
```

[실행결과]



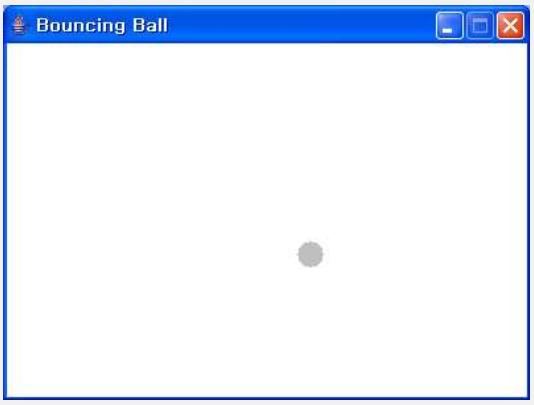
[13-10] 다음은 키보드의 화살표키를 이용해서 공을 Frame의 테두리 내에서 상하좌우로 이동시키는 프로그램이다. 알맞은 코드를 넣어 프로그램을 완성하시오.

[연습문제]/ch13/Exercise13_10.java

```
import java.awt.*;
import java.awt.event.*;

class BouncingBall extends Frame {
    final int BALL_SIZE = 20;
    final int FRAME_WIDTH  = 400;
    final int FRAME_HEIGHT = 300;

    final int TOP;
    final int BOTTOM;
    final int LEFT;
    final int RIGHT;

    int x;
    int y;

    final int SPEED = 3;
    int xStep = SPEED;
    int yStep = SPEED;

    BouncingBall(String title) {
          super(title);

          x = FRAME_WIDTH/2  - BALL_SIZE/2;
          y = FRAME_HEIGHT/2 - BALL_SIZE/2;

          setVisible(true);
          setBounds(300,200, FRAME_WIDTH, FRAME_HEIGHT);

          Insets insets = getInsets();
          TOP   = insets.top;
          LEFT  = insets.left;
          BOTTOM = FRAME_HEIGHT - insets.bottom ;
          RIGHT  = FRAME_WIDTH  - insets.right;

          registerEventHandler();
          setResizable(false);
    }

    void registerEventHandler() {
          addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent e) {
                      System.exit(0);
                }
          });

          addKeyListener(new KeyAdapter() {
                public void keyPressed(KeyEvent ke){
                      int key = ke.getKeyCode();

                      if(key==KeyEvent.VK_UP){
                        /*
```

```
                    (1) 알맞은 코드를 넣으시오.
                  */
                }else if(key==KeyEvent.VK_DOWN){
                  /*
                    (2) 알맞은 코드를 넣으시오.
                  */
                }else if(key==KeyEvent.VK_LEFT){
                  /*
                    (3) 알맞은 코드를 넣으시오.
                  */
                }else if(key==KeyEvent.VK_RIGHT){
                  /*
                    (4) 알맞은 코드를 넣으시오.
                  */
                }
            } // public void keyPressed(KeyEvent ke)
        });
    }

    void start() {
        while(true) {
            repaint();

            try {
                Thread.sleep(100);
            } catch (Exception e) {}
        }
    } // start()

    public void paint(Graphics g) {
        g.setColor(Color.LIGHT_GRAY);
        g.fillOval(x, y, BALL_SIZE, BALL_SIZE);
    }
}

class Exercise13_10 {
    public static void main(String[] args) {
        new BouncingBall("Bouncing Ball").start();
    }
}
```

[실행결과]



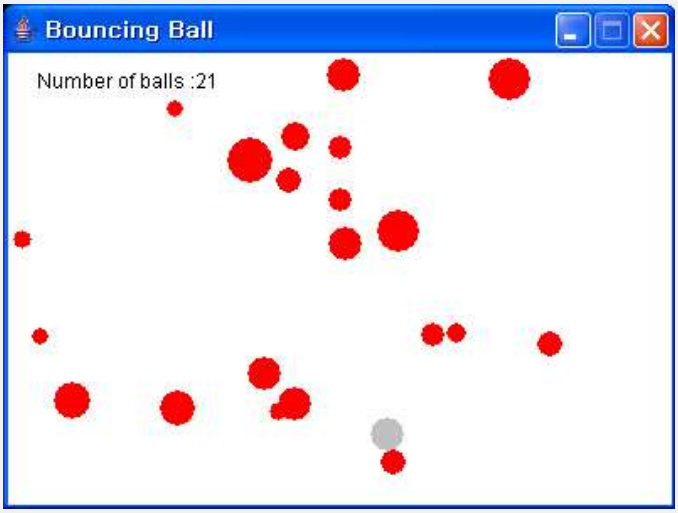
[13-11] 다음은 문제13-9와 13-10을 결합해서 발전시킨 것으로 회색공을 키보드로 움직여서 빨간공들을 피해다니다 빨간공과 회색공이 충돌하면 중단되는 프로그램이다. 알맞은 코드를 넣어 코드를 완성하시오.

[연습문제]/ch13/Exercise13_11.java

```
import java.util.*;
import java.awt.*;
import java.awt.event.*;

class BouncingBall extends Frame {
    final int BALL_SIZE = 20;
    final int FRAME_WIDTH  = 400;
    final int FRAME_HEIGHT = 300;

    final int TOP;
    final int BOTTOM;
    final int LEFT;
    final int RIGHT;
    final int SPEED = 7;

    int x;
    int y;
    boolean isPlaying = true;
    ArrayList balls = new ArrayList();

    BouncingBall(String title) {
          super(title);

          x = FRAME_WIDTH/2  - BALL_SIZE/2;
          y = FRAME_HEIGHT/2 - BALL_SIZE/2;

          setBounds(300,200, FRAME_WIDTH, FRAME_HEIGHT);
          setResizable(false);
          setVisible(true);

          Insets insets = getInsets();

          TOP   = insets.top;
          LEFT  = insets.left;
          BOTTOM = FRAME_HEIGHT - insets.bottom;
          RIGHT  = FRAME_WIDTH  - insets.right;

          registerEventHandler();
    }

    void registerEventHandler() {
          addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent e) {
                      System.exit(0);
                }
          });

          addKeyListener(new KeyAdapter() {
                public void keyPressed(KeyEvent ke){
                      if(!isPlaying) return;
```

```
                int key = ke.getKeyCode();

                if(key==KeyEvent.VK_UP){
                  /*
                    (1) 알맞은 코드를 넣으시오.
                  */
                }else if(key==KeyEvent.VK_DOWN){
                  /*
                    (2) 알맞은 코드를 넣으시오.
                  */
                }else if(key==KeyEvent.VK_LEFT){
                  /*
                    (3) 알맞은 코드를 넣으시오.
                  */
                }else if(key==KeyEvent.VK_RIGHT){
                  /*
                    (4) 알맞은 코드를 넣으시오.
                  */
                }
            } // public void keyPressed(KeyEvent ke)
        });
    }

    void start() {
        new BallGenerator().start();

        while(isPlaying) {
            int size = balls.size();

            for(int i=0 ; i < size; i++) {
                Ball b = (Ball)balls.get(i);

                b.x +=b.xStep;
                b.y +=b.yStep;

            /*
          (5) 아래의 로직에 맞게 코드를 작성하시오.
          1. b.x의 값이 왼쪽 테두리(LEFT)보다 작거나 같으면
             b.x의 값을 LEFT로 바꾸고 x축 이동방향(b.xStep)을 반대로 한다.
          2. b.x의 값이 오른쪽 테두리(RIGHT-b.size)보다 작거나 같으면
             b.x의 값을 RIGHT-b.size로 바꾸고 x축 이동방향(b.xStep)을 반대로 한다.
          3. b.y의 값이 윗 쪽 테두리(TOP)보다 작거나 같으면
             b.y의 값을 TOP으로 바꾸고 y축 이동방향(b.yStep)을 반대로 한다.
          4. b.y의 값이 아래 쪽 테두리(BOTTON-b.size)보다 작거나 같으면
             b.y의 값을 BOTTON-b.size로 바꾸고 y축 이동방향(b.yStep)을 반대로 한다.
            */
            } // for

            repaint();

            try {
                Thread.sleep(100);
            } catch (Exception e) {}
```

```
        }
    } // start()

    boolean collisionCheck(Ball b) {
        /*
          (6) 아래의 로직에 맞게 코드를 작성하시오.
          1. 회색공의 중심과 빨간공의 중심간의 거리(distance)가 회색공의
             반지름(BALL_SIZE/2)과 빨간공의 반지름(b.size/2)보다 같거나 작으면
             true를 반환하고 그렇지 않으면 false를 반환한다.

             회색공의 위치가 (x, y)일 때,
             중심 좌표는 (x+BALL_SIZE/2, y+BALL_SIZE/2)이다.
             두 점 (x1, y1)과 (x2, y2)간의 거리는 아래의 공식으로 구할 수 있다.
             Math.sqrt((x1-x2)*(x1-x2) + (y1-y2)*(y1-y2))
        */
    }

    public void paint(Graphics g) {
        g.drawString("Number of balls :"+balls.size(),20,50);
        g.setColor(Color.LIGHT_GRAY);
        g.fillOval(x, y, BALL_SIZE, BALL_SIZE);

        g.setColor(Color.RED);

        int size = balls.size();

        for(int i=0 ; i < size; i++) {
            Ball b = (Ball)balls.get(i);
            g.fillOval(b.x, b.y, b.size, b.size);
        }
    }

    class BallGenerator extends Thread {
        public void run() {
            while(isPlaying) {
              int x=(int)(Math.random()*(RIGHT-LEFT-Ball.MAX_SIZE))+LEFT;
              int y=(int)(Math.random()*(BOTTOM-TOP-Ball.MAX_SIZE))+TOP;

              balls.add(new Ball(x,y));

              try {
                 Thread.sleep(3*1000);
              } catch(Exception e) {}
            }
        } // run()
    }

    class Ball {
        int x = 100;
        int y = 100;
        int size = 30;

        static final int MAX_SIZE = 30;
        static final int MIN_SIZE = 10;
```

```
            final int SPEED = 5;
            int xStep = SPEED;
            int yStep = SPEED;

            Ball(int x, int y) {
                  this(x, y,(int)(Math.random()*(MAX_SIZE-MIN_SIZE))+MIN_SIZE);
            }

            Ball(int x, int y, int size) {
                  this.x = x;
                  this.y = y;
                  this.size = size;
            }
      }
} // class BouncingBall

class Exercise13_11 {
      public static void main(String[] args) {
            new BouncingBall("Bouncing Ball").start();
      }
}
```

[실행결과]